

Hochschule Ruhr West
Institute of Computer Science

Fine-Tuning Large Language Models for Structured Dashboard Design Recommendations Based on User Goals and Data Context

Master Thesis

Author:	Ramin Saeedi Valashani
Degree programme:	M.Sc. Computer Science
Institution:	Hochschule Ruhr West, Bottrop, Germany
Submission date:	26 September 2026
Academic supervisor:	Prof. Dr. Marc Jansen
Industry tutor:	M.Sc. Alexander Pawlak, adesso

Abstract

This thesis investigates whether retrieval-augmented generation and parameter-efficient fine-tuning improve structured dashboard-design recommendations produced by instruction-tuned language models. A provenance-aware dataset was constructed from source-grounded analytical records and constrained, explicitly marked LLM-generated enrichment. Four model profiles were evaluated under prompt-only, retrieval-augmented, QLoRA-fine-tuned, and combined conditions. The executed matrix contains 32 complete runs on 274 held-out items: three seeds for Qwen3-1.7B and OLMo-2-1B-Instruct, and one seed for Qwen3-8B and Qwen3.8-27B.

The evaluation separates JSON recovery, runtime-contract validity, completeness, reference agreement at three scoring layers, per-class performance, robustness, grounding, and latency. This decomposition is necessary because end-to-end chart accuracy confounds the chart decision with output formatting and truncation. At the text level, fine-tuning improves agreement with the frozen chart reference in all eight model-by-seed blocks by 0.36–10.58 percentage points, although output validity falls for Qwen3-8B and OLMo under the executed 512-token generation budget. Retrieval is slightly beneficial for Qwen3-1.7B, improves recoverability but not chart choice for OLMo, and significantly lowers reference agreement for the 8B and 27B Qwen3 models. A traceable conflict between the knowledge base and the frozen reference explains most pie-to-donut substitutions at both larger scales. Adding retrieval after fine-tuning lowers reference agreement in eleven of twelve blocks.

The automatic results are conditional on an internally constructed reference, asymmetric seed coverage, heterogeneous code states, a small three-document knowledge base, and text-only recommendations. A completed exploratory human evaluation of the four Qwen3.8-27B conditions contributed 81 rating pages from ten complete sessions across eight briefs. On a five-point rubric, retrieval-only achieved the highest equal-brief means for chart appropriateness (4.48), rationale quality (4.52), and overall usefulness (4.52); prompt-only was marginally higher for layout and interaction. Fine-tuned conditions did not improve the human-rated outcomes in this sample. Because recruitment was by convenience, coverage was two to four pages per output, and ordinal inter-rater agreement was low for five of six dimensions, these observations are descriptive rather than population-level or causal conclusions. The thesis contributes a reproducible, artifact-anchored evaluation of structured recommendation generation and demonstrates why format conformance, semantic agreement, and human judgement must be reported as separate evidence layers.

Keywords: large language models; dashboard recommendation; QLoRA; retrieval-augmented generation; structured generation; provenance; evaluation

Acknowledgements

I thank Prof. Dr. Marc Jansen for his academic supervision and M.Sc. Alexander Pawlak at adesso for his practical guidance and feedback during this thesis.

Contents

Abstract	i
1 Introduction	1
1.1 Motivation	1
1.2 The task studied in this thesis	2
1.3 Research gap	2
1.4 Research questions	2
1.5 Approach	3
1.6 Contributions	3
1.7 Scope and delimitation	4
1.8 Structure of the thesis	4
2 Background and Foundations	5
2.1 Dashboard Design	5
2.2 Visualization Design Principles	6
2.3 Analytical Tasks and Chart Types	7
2.4 Large Language Models	8
2.5 Parameter-Efficient Fine-Tuning	9
2.6 Retrieval-Augmented Generation	11
2.7 Structured LLM Outputs	11
3 Related Work	13
3.1 Visualization Recommendation Systems	13
3.2 Natural-Language Interfaces and NL-to-Visualization	14
3.3 Datasets and Benchmarks for Visualization Generation	15
3.4 LLMs for Visualization and Dashboard Assistance	16
3.5 Parameter-Efficient Fine-Tuning for Structured Generation	17
3.6 Retrieval-Augmented Generation for Domain-Grounded Generation	19
3.7 Evaluation of Structured and Visualization Outputs	20
3.8 Research Gap and Positioning of This Thesis	21
4 Dataset Construction and Provenance	23
4.1 Task Definition and Dataset Requirements	23
4.2 Source Corpus and Source Registration	24
4.3 Target Schema and Evidence Classes	26
4.4 Source-Faithful Transformation of nvBench	29
4.5 Iterative Validation and Quality Filtering	30
4.6 Selection, Deduplication, and Evaluation Isolation	32
4.7 Constrained LLM Enrichment and the <code>dashboard_v3</code> Freeze	33
4.8 Controlled <code>dashboard_v4</code> Augmentation	35
4.9 <code>dashboard_v4_1</code> Semantic Repair	36
4.10 Final Dataset Composition and Mixed Lineage	37
4.11 Dataset Freeze, Versioning, and Reproducibility	39
4.12 Limitations and Threats to Validity	40
5 System Design and Methods	42
5.1 Methodological Scope and Research Design	42
5.2 Formal Task and Output Contract	44
5.3 Model Matrix and Experimental Factors	45
5.4 Shared Prompt and Structured Generation Pipeline	48
5.5 Method A — Prompt-Only Baseline	50
5.6 Method B — Retrieval-Augmented Generation	50
5.7 Method C — QLoRA Fine-Tuning	51

5.8	Method D — QLoRA Fine-Tuning with Retrieval	53
5.9	Alternative Fine-Tuning Methods and Selection Rationale	54
5.10	Inference and Output Processing	55
5.11	Evaluation Protocol	55
5.12	Reproducibility and Provenance	57
5.13	Methodological Limitations and Threats to Validity	58
6	Experimental Design and Evaluation	61
6.1	Executed experiment matrix	61
6.2	Models	63
6.3	Data and inputs	65
6.4	Executed training configuration	66
6.5	Inference configuration and execution environment	68
6.6	Metrics	69
6.7	Statistical analysis plan	71
6.8	Human evaluation	72
6.9	Reproducibility of the reported analysis	73
7	Results	74
7.1	Structured-output reliability (RQ2: structure)	74
7.2	Chart agreement at three levels of strictness (RQ1)	78
7.3	Supporting analysis: model scale and model family	88
7.4	Stability across seeds and under perturbation (RQ3)	89
7.5	Failure modes	92
7.6	Retrieval grounding	94
7.7	Computational cost	96
7.8	Human evaluation (RQ2 and RQ3)	98
8	Discussion and Threats to Validity	99
8.1	Interpretation of Main Results	99
8.2	Effect of Retrieval	99
8.3	Effect of Fine-Tuning and Measurement Choice	100
8.4	Human-Rated Actionability	101
8.5	Model Scale and Cross-Family Observations	101
8.6	Error Analysis	102
8.7	Comparison with Related Work and Methodological Lessons	103
8.8	Practical Implications	105
8.9	Threats to Validity	105
9	Conclusion and Future Work	108
9.1	Overall Findings	108
9.2	Answers to the Research Questions	108
9.3	Contributions	109
9.4	Limitations in Brief	109
9.5	Future Work	109

Eidesstattliche Erklärung

Hiermit erkläre ich, dass ich die vorliegende Masterarbeit selbstständig angefertigt habe. Es wurden nur die in der Arbeit ausdrücklich benannten Quellen und Hilfsmittel benutzt. Wörtlich oder sinngemäß übernommenes Gedankengut habe ich als solches kenntlich gemacht. Die vorgelegte Arbeit hat weder in der gegenwärtigen noch in einer anderen Fassung bereits einer anderen Prüfungsbehörde vorgelegen.

Bottrop, 26. September 2026
Ramin Saeedi Valashani

Declaration of AI-Assisted Tools

OpenAI Codex was used as an assistive tool for language revision, LaTeX organization, consistency checks, and support in drafting and auditing analysis code, following documented procedural guidance for scientific-agent workflows [Kassis et al. \(2026\)](#). The controlled dataset-enrichment workflow described in Chapter 4 used the explicitly recorded model `gpt-5.6-luna`; all generated fields remain labelled as LLM-generated and not expert gold. AI tools were not treated as scientific sources for empirical claims. The author selected the research design, executed and verified the experiments, checked the cited literature and numerical results, and accepts full responsibility for the submitted text and conclusions.

Chapter 1

Introduction

This chapter introduces the research problem, defines the task that the thesis studies, states the research gap and the research questions, and explains how the remaining chapters answer them. It first describes why the design of a dashboard is a decision problem rather than a rendering problem, and why that decision problem is a plausible target for a language model. It then narrows the general idea into the concrete task used throughout this work: mapping a short natural-language dashboard brief to a machine-readable design recommendation. The chapter closes with the contributions, the scope of the study, and an overview of the thesis structure. Readers who finish this chapter should know what is being predicted, which four system configurations are compared, which evidence the study can produce, and which questions it deliberately leaves open.

1.1 Motivation

Dashboards are one of the most common ways in which organisations present quantitative information. They combine several coordinated views so that an audience can monitor a situation, compare alternatives, or decide on an action. The visualization literature has repeatedly pointed out that a dashboard is defined less by the number of charts it shows than by the fit between the displayed information and the decision context of its audience. A review of dashboards in performance management describes them as systems that combine performance information, visual display, and interaction in support of management decisions [Yigitbasioglu and Velcu \(2012\)](#). A systematic analysis of how the term is actually used in research and practice finds a broad and partly inconsistent design space, ranging from static reporting displays to interactive analytic environments [Sarikaya et al. \(2019\)](#). More recent work argues that recurring solutions in this space can be described as design patterns, for example patterns for overview, comparison, filtering, and detail [Bach et al. \(2023\)](#).

Producing a good dashboard therefore involves a sequence of connected decisions. Which measures matter for this audience and this goal? What analytical task does each measure imply — a comparison, a trend, a distribution, a part-to-whole relation? Which chart type expresses that task in a way that people can decode accurately? How should the fields be mapped onto visual channels? How should the views be arranged so that the most important information is found first? Which styling and accessibility conventions apply? Which interactions are needed, and what justifies each choice? Each of these questions is informed by an established body of knowledge. Controlled studies of graphical perception show that some encodings support more accurate quantitative judgements than others [Cleveland and McGill \(1984\)](#). Expressiveness and effectiveness criteria distinguish a representation that states the intended information from one that also communicates it efficiently [Mackinlay \(1986\)](#). Task-based studies show that the effectiveness of a basic visualization depends on the analytical task and on the data distribution rather than on a universal ranking of chart types [Saket et al. \(2019\)](#); [Kim and Heer \(2018\)](#).

The practical problem is that this knowledge is not evenly distributed. The people who know the data and the decision are frequently not the people who know the visualization literature. Existing recommendation systems address part of the gap. Constraint-based approaches encode design knowledge as explicit rules that can rank or reject encodings [Moritz et al. \(2019\)](#), and mixed-initiative tools suggest encodings during exploratory analysis [Wongsuphasawat et al. \(2016, 2017\)](#). Natural-language interfaces let an analyst request a chart in ordinary words and translate that request into a visualization specification [Narechania et al. \(2021\)](#); [Luo et al. \(2022\)](#). These systems are valuable, but almost all of them stop at the level of one chart for one query. They do not produce a coordinated dashboard design that also fixes layout, styling, interaction, and the reasons behind each decision.

Large language models change what is feasible here, because they can read an unstructured description of a situation and emit a structured object in one step. Two established mechanisms make it possible to adapt such a model to a specialised task without training it from scratch. Retrieval-augmented generation attaches relevant external passages to the prompt so that a frozen model can use knowledge that is not reliably present in its parameters [Lewis et al. \(2020\)](#). Parameter-efficient fine-tuning freezes the pretrained weights and learns a small low-rank update instead [Hu et al. \(2022\)](#), and its quantized variant makes this affordable on a single accelerator by keeping the frozen base model in 4-bit precision [Dettmers et al. \(2023\)](#). Both mechanisms are widely used, but they intervene at different points: one changes what the model sees, the

other changes what the model is. Whether either of them — or their combination — actually helps on a structured dashboard-design task is an empirical question, and it is the question this thesis addresses.

1.2 The task studied in this thesis

The thesis studies one narrowly defined prediction problem. The input is a *dashboard brief*: a short specification containing the intended users, the analytical goals, the key performance indicators (KPIs), the available data columns with their types, and any constraints. The output is a *structured dashboard-design recommendation*: a single JSON object with six required top-level fields — `context_summary`, `kpi_chart_mapping`, `layout`, `styling`, `interactions`, and `rationales`. Each entry of `kpi_chart_mapping` links one KPI to an analytical task type, a primary chart type, optional alternatives, and an encoding that names the fields placed on the visual channels. Chapter 4 documents how the data for this task was built and frozen; Chapter 5 gives the formal contract and the implementation.

The output is deliberately a structured object rather than free prose or a rendered image. A structured object can be parsed deterministically, validated against a schema, compared field by field, and consumed by a later rendering layer. It also makes a controlled comparison between systems possible, because every condition is scored against the same field names and the same controlled vocabularies. This choice has a cost that the thesis takes seriously throughout: format compliance and design quality are different properties. A response can be syntactically valid JSON and still recommend the wrong chart, and a response can recommend a defensible chart inside a record that fails the format contract. Structured-generation research shows both that schema conformance is itself a non-trivial capability [Geng et al. \(2025\)](#) and that imposing format restrictions can measurably change what a model produces [Tam et al. \(2024\)](#). The evaluation design in Chapter 6 therefore keeps parse success, contract validity, completeness, and chart agreement as separate quantities.

1.3 Research gap

Three gaps motivate the study.

The first gap is in task scope. Public resources for natural-language-to-visualization research supply a query, a database schema, and a target chart specification. `nvBench`, for example, was synthesised from a text-to-SQL benchmark and provides cross-domain natural-language-to-visualization pairs at large scale [Luo et al. \(2021b\)](#); [Yu et al. \(2018\)](#), and its successor addresses ambiguity in that mapping [Luo et al. \(2025\)](#). These resources cover the analytical core of the present task well. They do not cover the dashboard-level fields — users, layout, styling, interactions, rationales — that distinguish a coordinated dashboard from a single chart. No public benchmark maps a dashboard brief of the kind defined above onto the full structured recommendation used here. A dataset for this task therefore has to be constructed, and its construction has to be transparent about which fields are source-grounded and which were generated.

The second gap is in controlled comparison. Retrieval augmentation and parameter-efficient fine-tuning are usually studied separately, on different tasks, with different base models and different prompts. Reports that compare them on one task, with one frozen dataset, one prompt contract, one output parser, one evaluation code path, and matched seeds are rare — and rarer still across several model sizes and more than one model family. Without those controls it is difficult to attribute an observed difference to the intervention rather than to an incidental change in prompt, data, or decoding settings.

The third gap is in evaluation. Many reported results for structured generation compress several distinct properties into a single accuracy number. If a metric requires a fully valid record before it will score the chart decision, then a formatting failure and a design failure become indistinguishable. The present study needs an evaluation protocol that separates these layers explicitly, that states which reference labels are independent evidence and which are the project’s own construction, and that marks what has not been measured.

1.4 Research questions

The thesis retains the three research questions stated in the exposé. Their wording is reproduced here without adding new research questions:

RQ1 (Accuracy). How accurately can the system map dashboard tasks/KPIs to suitable chart types (Top-1/Top-3 where reference labels exist) across prompt-only, RAG, fine-tuning, and fine-tuning+RAG?

RQ2 (Actionability & Structure). Do RAG and/or fine-tuning improve actionability and structure, measured via schema compliance and human rubric ratings?

RQ3 (Rationale & Robustness). Do RAG and/or fine-tuning improve heuristic-aligned rationales and robustness under paraphrased or partially specified briefs?

The evidence does not answer every component with the same strength. RQ1 is evaluated with the available primary-chart reference; the artifacts do not contain a defensible ranked Top-3 reference set. RQ2 combines

automatic structure measures with a completed exploratory human evaluation of eight briefs and four system conditions. RQ3 is evaluated through controlled robustness tests, a lexical grounding proxy, and human rationale ratings. Model scale, model family, seed variability, and failure modes support these questions rather than forming additional research questions. The human study reports its sampling, assignment, missingness, aggregation, and agreement explicitly, as recommended for human evaluation of generated outputs [van der Lee et al. \(2019\)](#).

1.5 Approach

The study compares exactly four system conditions for a fixed base model and a fixed seed:

- **A — Prompt-only.** The base instruction-tuned model receives the shared system and user prompt and nothing else. This is the reference condition.
- **B — Retrieval-augmented generation.** The same base model additionally receives up to three passages retrieved from a small, frozen corpus of visualization and dashboard design guidelines.
- **C — QLoRA fine-tuning.** The base model is adapted with a quantized low-rank adapter trained on the frozen training split. No retrieval is used.
- **D — Fine-tuning with retrieval.** The adapter produced by condition C for the *same* base model, dataset, training configuration, and seed is loaded, and the retrieval procedure of condition B is applied on top of it.

Condition D reuses the C adapter rather than training a second one. Training D independently would introduce a second stochastic training trajectory and would make the incremental effect of retrieval impossible to isolate. The contrasts that the design supports are therefore B – A for retrieval alone, C – A for adaptation alone, D – C for retrieval added after adaptation, and D – B for adaptation added under retrieval. The direct D – A contrast compares the complete systems but changes two factors at once and cannot attribute the difference to either one.

Everything not named by a condition is held fixed: the frozen dataset release and its hashes, the prompt builder, the controlled task and chart vocabularies, the output parser, the generation settings, the held-out test split, and the evaluation code. The experiment was executed across four base models — three Qwen3 profiles spanning roughly 1.7B to 27B parameters and one OLMo 2 profile at roughly 1.49B parameters [Yang et al. \(2025\)](#); [OLMo Team et al. \(2025\)](#) — with three seeds where the compute budget allowed and a single seed for the more expensive profiles. Chapter 6 states the executed coverage precisely and explains why the coverage is asymmetric.

1.6 Contributions

The thesis makes the following contributions, each stated at the strength the available evidence supports.

1. **A frozen, lineage-annotated dataset for the dashboard-brief-to-recommendation task.** The release combines nvBench-derived, source-grounded analytical records with LLM-generated dashboard-level enrichment and additional generated records. Every field carries an explicit evidence class, the generated material is labelled as generated rather than as gold, the held-out test is byte-identical across the freeze steps, and the package is published with a manifest and file hashes. Chapter 4 documents the construction; the limitations of generated supervision are stated there and revisited in Chapter 8.
2. **A configuration-driven implementation of the four conditions with enforced adapter provenance.** One prompt source, one parser, one evaluation path, and a validator that refuses to load an adapter into condition D unless the base model, model revision, model-configuration hash, training-configuration hash, dataset version, and seed match the producing condition-C run.
3. **A layered evaluation protocol that keeps evidence types apart.** Parse success, contract validity, completeness, chart agreement, robustness under two perturbation families, and retrieval grounding are reported as separate quantities, each with an explicit statement of what it does and does not establish. Reference-based chart agreement is labelled as agreement with the project’s own reference, not as evidence of visualization effectiveness.
4. **An executed A/B/C/D comparison across three Qwen3 scales and one cross-family OLMo 2 model,** with paired per-item statistics — Cochran’s Q, exact McNemar tests with Holm correction, and percentile bootstrap confidence intervals for paired differences — computed from the stored per-item records [Cochran \(1950\)](#); [McNemar \(1947\)](#); [Holm \(1979\)](#); [Efron \(1979\)](#).
5. **A diagnostic decomposition of the headline chart metric.** The stored raw model outputs are re-scored at three levels of strictness so that a failure of the output contract, a failure of the generation-length budget, and a failure of the chart decision can be separated. This decomposition is reported in Chapter 7 and is the basis for several of the conclusions in Chapter 8.

6. **A transparent record of what was not measured.** The independent chart-effectiveness scorer, the realism layer, and the human-rating layer are specified but not executed. They are reported as open, not silently omitted.

1.7 Scope and delimitation

The system predicts a structured design recommendation. It does not render a dashboard, does not execute queries against a database, and does not measure how quickly a human completes a task with the resulting display. Claims about decision quality, adoption, or business value are therefore outside what the current evidence can support.

The adaptation method is fixed to one QLoRA configuration. The thesis compares prompting, retrieval, and adaptation; it does not compare adaptation algorithms against each other. Alternative parameter-efficient methods are discussed in Chapters 2, 3, and 5 to justify the selection, and they are explicitly not part of the experiment matrix.

The retriever is a sparse TF-IDF retriever over a small frozen guideline corpus. Dense retrieval is available in the repository as an optional configuration and was not used for the reported conditions; it is discussed only where it helps explain the design choice.

Model coverage is limited to four base models and to the seed coverage that the compute budget allowed. Seed replication is complete for the two smaller models and partial for the larger ones. The consequence — that variability statements are weaker for the larger models — is stated wherever it applies rather than being generalised away.

Finally, the reference recommendations against which the automatic metrics score contain LLM-generated fields. Agreement with them measures how well a system reproduces the project’s frozen representation. It is not, by itself, evidence that a recommendation is a good dashboard design. This boundary is maintained in every results statement.

1.8 Structure of the thesis

Chapter 2 introduces the background: dashboard design, visualization principles, analytical tasks and chart types, language models and instruction tuning, parameter-efficient fine-tuning including LoRA and QLoRA, retrieval-augmented generation, and structured outputs.

Chapter 3 reviews related work on visualization recommendation, natural-language interfaces, datasets and benchmarks for visualization generation, LLM-assisted visualization, parameter-efficient fine-tuning for structured generation, retrieval augmentation, and the evaluation of structured outputs, and positions the present study against them.

Chapter 4 documents the dataset: source selection, the target schema and its evidence classes, the source-faithful transformation of nvBench, quality filtering, deduplication and split isolation, the constrained LLM enrichment, the controlled augmentation and semantic repair, the freeze, and the resulting limitations.

Chapter 5 describes the system: the formal task contract, the model matrix and the controlled factors, the shared prompt and generation pipeline, the four methods, the reasons for selecting QLoRA over the alternatives, inference and output processing, reproducibility and provenance, and methodological threats to validity.

Chapter 6 defines the experimental setup and evaluation protocol: the executed run matrix and seed coverage, each metric and its limits, the perturbation conditions, the statistical analysis, and the completed exploratory human study.

Chapter 7 reports the results from the generated artifacts only: structured-output reliability, chart agreement at three levels of strictness, the paired method contrasts with confidence intervals and corrected p-values, seed variability, robustness, retrieval grounding, and cost.

Chapter 8 interprets the results, analyses the failure modes item by item, and states the threats to validity that bound each conclusion.

Chapter 9 summarises what the study established, what it did not, and which next steps follow directly from the open items.

Chapter 2

Background and Foundations

This chapter defines the dashboard-design concepts, visualization principles, language-model mechanisms, QLoRA adaptation, retrieval augmentation, and structured-output terminology required by the study. Detailed implementation choices and alternatives are reserved for Chapter 5.

2.1 Dashboard Design

A dashboard is an interactive view that brings together several related measures and visual elements for monitoring, analysis, or decision support. Dashboards are not defined only by the number of charts that they contain. Their value depends on whether the displayed information supports the needs of a particular audience and decision context. Reviews of dashboard research describe dashboards as systems that combine performance information, visual displays, and interaction in order to support monitoring and management decisions [Yigitbasioglu and Velcu \(2012\)](#). Practitioner-oriented work makes the same point from the design side, arguing that a dashboard should present the information needed to achieve one or more objectives on a single screen and that most dashboard failures are failures of visual communication rather than of data [Few \(2006\)](#). More recent work also shows that recurring dashboard design problems can be described through design patterns, for example patterns for overview, comparison, filtering, and detail [Bach et al. \(2023\)](#), and a survey of how the term is used across research and practice documents a design space that is considerably broader than a single canonical dashboard type [Sarikaya et al. \(2019\)](#).

In this thesis, a dashboard is understood as a coordinated design rather than as a random collection of individual charts. A coherent dashboard connects the intended users, their goals, the relevant KPIs, the available data, the analytical tasks, the visual encodings, the layout, the styling, and the interactions. These elements should support the same decision context. A chart can be correct on its own and still be unsuitable when it is placed in the wrong dashboard, given too much visual emphasis, or separated from the information needed to interpret it.

A KPI is a measurable value used to represent progress or performance in relation to a goal. The value of a KPI depends on its context. The same measure may be useful for one user and irrelevant for another user because the decision, time period, comparison group, or available level of detail is different. A dashboard design should therefore make clear what is measured, for whom it is measured, over which period, and why the measure matters. This view also prevents a common mistake in automated design systems: treating a KPI name as enough information to choose a chart.

The design problem in this thesis can be described as a translation problem. A dashboard brief contains natural-language information about users and goals, KPI descriptions, data columns, and constraints. The system must translate this brief into a structured design recommendation. The recommendation does not only contain a chart type. It also contains the mapping from KPIs to analytical tasks and encodings, a dashboard layout, styling, interactions, and rationales. This formulation is related to the general visualization-design problem of mapping data and user tasks to graphical representations [Mackinlay \(1986\)](#); [Munzner \(2014\)](#). Constraint-based systems such as Draco make a similar distinction by representing design knowledge as explicit constraints that can be used to rank or reject visual encodings [Moritz et al. \(2019\)](#).

The project implements this design problem as structured generation. Its input is a dashboard brief with users, goals, KPIs, data columns, and constraints. Its target is a typed recommendation with the fields `context_summary`, `kpi_chart_mapping`, `layout`, `styling`, `interactions`, and `rationales`. This structure is a project-specific design contract. It is not presented as a universal definition of a dashboard. It makes the target explicit and allows the methods in the thesis to be compared on the same output interface.

2.1.1 Dashboards versus Individual Visualizations

A single visualization normally addresses a focused analytical question, such as whether a value changes over time or whether two variables are related. A dashboard combines several views and must also manage the relationships between them. These relationships include visual hierarchy, consistent scales and labels, shared filters, spatial grouping, interaction, and the amount of information shown at one time.

This distinction matters for the present research. Many visualization datasets and systems focus on one query and one chart specification. They can provide useful examples for analytical tasks, data fields,

aggregations, and encodings. They do not necessarily provide a complete dashboard with a coordinated layout, styling, interaction design, and rationale. The project therefore treats chart selection as one part of a larger recommendation. The generated object is evaluated as a structured dashboard design, not only as a single chart label.

2.1.2 Users, Goals, KPIs, and Data Context

A dashboard brief is a compact specification of the design problem. The user description identifies the intended audience and its level of expertise. The goals describe what the audience wants to understand, compare, monitor, or decide. The KPIs identify the measures that are relevant to these goals. The data context describes the available fields, their types, their possible groupings, the time dimension, and any constraints on the design. These parts should be interpreted together because none of them is sufficient on its own.

For example, the instruction “show sales” does not determine a good visualization. A brief such as “show monthly total sales by product category for a regional manager” gives more useful information. It identifies a time grain, an aggregation, a grouping dimension, and an audience. It can therefore support a more precise decision about the analytical task, chart type, encoding, and level of detail. Explicit context also reduces the risk that a model invents a field, a KPI, or an interaction that is not present in the input.

2.2 Visualization Design Principles

Visualization design is based on both empirical findings and practical design guidance. These two forms of knowledge should be kept separate. Controlled studies can provide evidence about how accurately people decode a visual encoding. Design frameworks can provide useful concepts for describing tasks, data, and interfaces. Neither type of source turns every chart choice into a universal rule. The relevant principle for this thesis is that a chart should be selected in relation to the task and the context in which it will be read.

2.2.1 Graphical Perception

Cleveland and McGill [Cleveland and McGill \(1984\)](#) studied how people judge elementary graphical quantities. Their work showed that some visual encodings support more accurate judgments than others. In particular, position along a common scale is generally judged more accurately than angle or area. This result gives a perceptual reason to prefer position- and length-based encodings when users must make precise quantitative comparisons.

The result should not be interpreted as a complete ranking of all chart types. The usefulness of an encoding also depends on the task, the data distribution, the number of categories, the display size, and the user’s prior knowledge. A chart recommendation should therefore ask what the user needs to compare or detect. Precision is important for some tasks, while overview, composition, or recognition may be more important for others.

2.2.2 Expressiveness and Effectiveness

Mackinlay [Mackinlay \(1986\)](#) distinguishes between expressiveness and effectiveness. An expressive visualization represents the intended information without adding an unsupported meaning. An effective visualization uses graphical encodings that help users interpret the information accurately and efficiently. These criteria are useful for automated recommendation because a chart can be technically valid and still be a poor design choice.

For example, a pie chart can represent category proportions without a data error. It may nevertheless be ineffective when many categories are present or when users need to compare values that are close to one another. A bar chart may support that comparison more directly because the values can be judged by aligned positions and lengths. This example illustrates why the output of the system should include a rationale and not only a chart name.

2.2.3 Task–Chart Fit

The fit between an analytical task and a chart is central to visualization design. Saket et al. [Saket et al. \(2019\)](#) show that the effectiveness of basic visualizations changes across tasks. Kim and Heer [Kim and Heer \(2018\)](#) likewise show that task and data distribution can affect the effectiveness of visual encodings. These findings support a context-dependent recommendation process instead of a universal ranking such as “bar charts are always best.”

In practice, an ordered temporal trend is often represented with a line chart because connected positions make continuity and direction visible. A categorical comparison or ranking is often represented with bars because aligned lengths support comparison. A relationship between two quantitative variables is often represented with a scatter plot. These mappings are useful starting points, but they still require checks on data semantics, scale, cardinality, and the user’s goal. The project consequently represents task type and chart type as explicit fields and leaves the final choice to the model under the constraints of the brief.

2.2.4 Visual Hierarchy and Information Density

A dashboard must guide the user's attention. High-priority information should be easy to find, while secondary detail should remain available without competing with the main message. Visual hierarchy can be created through position, size, grouping, whitespace, contrast, and consistent alignment. The dashboard design-pattern literature describes these choices as recurring solutions to common interface problems [Bach et al. \(2023\)](#).

Information density requires a balance. A dashboard with too little information may not support the intended decision. A dashboard with too much information may increase search time and make important changes harder to detect. Density is therefore not simply a matter of counting charts. It also depends on the complexity of each view, the number of encodings, the amount of text, the interaction model, and the user's familiarity with the domain. In this thesis, layout and styling are part of the output because they influence how the selected charts are interpreted together.

2.2.5 Readability, Labels, and Scales

Readable charts use titles, labels, legends, annotations, and number formats that match the target audience. Axis labels should communicate the meaning and unit of a measure. Category labels should be understandable and should not expose database names without explanation. A scale should preserve the intended comparison and should not exaggerate or hide a difference through an inappropriate axis range or transformation.

Readability also depends on the available space. A design that works on a large screen may become difficult to read when several views are placed in a small dashboard. This is another reason to treat layout as a design decision rather than as a final decoration. The system should make a reasonable connection between the amount of information, the expected display, and the level of detail required by the user.

2.2.6 Color and Accessibility

Color can distinguish categories, show magnitude, communicate status, or draw attention to an important value. It can also create problems when too many colors are used, when similar colors are difficult to separate, or when a critical distinction is communicated only through color. The Web Content Accessibility Guidelines (WCAG) 2.2 require, among other things, that information and relationships are not communicated through color alone and that text has sufficient contrast for reading [World Wide Web Consortium \(2024\)](#).

For dashboard design, this means that color should usually be supported by labels, position, shape, text, or another redundant cue. Color palettes should be selected with the audience and display conditions in mind. Accessibility is not a separate visual layer that can be added after chart selection. It can change the suitability of an encoding and should therefore be included in the recommendation and its rationale.

2.3 Analytical Tasks and Chart Types

An analytical task describes what a user wants to do with data. Brehmer and Munzner [Brehmer and Munzner \(2013\)](#) distinguish the purpose of a task from the means used to perform it and from the data involved. This distinction is useful because a column type alone does not reveal the user's intention. A date column can support a trend, a comparison between periods, a lookup, or a deviation from a target. The same field can therefore lead to different visual designs depending on the question.

The task vocabulary used in this thesis is informed by visualization task literature and adapted to the project's structured schema. It includes trend, comparison, composition, distribution, correlation, ranking, deviation, part-to-whole, and flow. The vocabulary is an implementation choice that makes evaluation possible. It should not be interpreted as a complete or universal taxonomy of all analytical work.

2.3.1 Comparison and Ranking

Comparison asks how values differ between categories, groups, or periods. Ranking adds the requirement that the values should be ordered. Bar charts are common choices because the lengths share a baseline and the categories can be sorted. Grouped bars can support comparison between several series, while a table may be more appropriate when exact values are more important than visual pattern recognition. The number of categories and the available screen space can change which option is readable.

2.3.2 Trend

Trend tasks examine change along an ordered dimension, most often time. Line charts are often useful because the connected marks emphasize continuity and direction. Area charts can add a sense of volume, but overlapping or stacked areas can make individual series harder to compare. The time grain must also be considered. Daily, monthly, and yearly aggregation can show different patterns and can lead to different decisions.

2.3.3 Composition and Part-to-Whole

Composition tasks ask how components contribute to a total. Stacked bars can show composition across several groups, and normalized stacked bars can show relative proportions. Treemaps can use space to represent parts of a hierarchy. Pie and donut charts can be suitable for a small number of clearly different parts, but they are less suitable when categories are numerous or values are close. The choice is therefore influenced by whether the user needs to compare individual parts, inspect a total, or understand changes in composition.

2.3.4 Distribution

Distribution tasks focus on frequency, spread, shape, and outliers. Histograms can show the frequency of binned values, while box plots provide a compact view of median, spread, and possible outliers. Density plots can show the shape of a distribution when the display and audience support that level of detail. A categorical bar chart may be enough when the input already contains aggregated frequencies. The data representation and the intended question must be checked before selecting the chart.

2.3.5 Correlation and Relationship

Correlation or relationship tasks examine how two or more variables change together. A scatter plot is a standard choice for two quantitative variables because each observation can be represented by a position in a two-dimensional coordinate system. Additional encodings may show groups or a third variable, but they also increase visual complexity. Identifier-like fields should not be treated as measurements only because they are stored as numbers.

2.3.6 Deviation

Deviation tasks focus on the difference from a reference value, target, baseline, or zero. A diverging bar chart can show positive and negative differences, while an annotated line chart can show when a measure moves above or below a target. KPI cards can provide a compact status view when the deviation itself is the primary message. The reference must be explicit; otherwise, the viewer cannot determine what the displayed difference means.

2.3.7 Flow and Network Relationships

Flow tasks concern movement between stages, categories, or entities. Sankey diagrams can represent quantities moving between connected nodes, and node-link diagrams can represent relationships in a network. These charts can be useful when the connections are the main subject of analysis, but they can become difficult to read as the number of nodes and links grows. In the current project, flow is part of the task vocabulary, while the available training and evaluation data determine how often this task can be assessed.

These examples are literature-informed design guidance, not fixed rules. A line chart is not automatically correct for every temporal field, and a bar chart is not automatically correct for every comparison. A final recommendation should take the user's goal, the data semantics, the number of values, accessibility, and dashboard context into account. This distinction is important for the thesis because the system is evaluated on context-aware structured recommendations rather than on memorization of a chart lookup table.

2.4 Large Language Models

Large language models are neural models that learn statistical patterns in sequences of tokens and use those patterns to generate text. Modern LLMs are commonly based on the Transformer architecture introduced by Vaswani et al. [Vaswani et al. \(2017\)](#). The model receives a sequence of tokens, represents their relationships, and predicts a continuation. In this thesis, the LLM is used as a conditional generator: it receives a dashboard brief and produces a structured design recommendation.

2.4.1 Transformer Architecture

The main operation in a Transformer is self-attention. In simplified form, each token representation is transformed into a query, a key, and a value. The similarity between a query and the keys determines how strongly the corresponding values contribute to the updated representation. Multi-head attention allows the model to learn several types of relationships in parallel. Transformer blocks also contain feed-forward layers, residual connections, and normalization.

The thesis does not require a mathematical treatment of every Transformer component. The important point is that self-attention allows a model to condition its output on different parts of the input sequence. A dashboard brief can therefore contain user information, goals, KPIs, data columns, and constraints in the same context. The model can use these elements when generating a recommendation, but the architecture alone does not guarantee that the recommendation is correct or that it follows the available data.

2.4.2 Autoregressive Generation

Decoder-only LLMs usually generate text autoregressively. At each step, the model predicts a probability distribution for the next token based on the tokens that have already been provided or generated. The selected token is appended to the context, and generation continues until a stopping condition is reached. Sampling settings can change the output, which is relevant when the same prompt is evaluated under different methods or random seeds.

Autoregressive generation is flexible because the output can contain several related fields instead of one classification label. It also creates failure modes. A model can stop too early, repeat content, omit a field, or generate text that looks plausible but is unsupported by the input. These failures motivate the use of explicit output constraints and separate validation steps later in the thesis.

2.4.3 Instruction Tuning

Instruction tuning adapts a pretrained language model using examples in which tasks are described through natural-language instructions. Wei et al. [Wei et al. \(2022\)](#) show that instruction tuning can improve performance on tasks that were not included in the same form during training. The method is important for this thesis because a dashboard brief is expressed as an instruction rather than as a fixed classification input.

Instruction tuning should not be confused with a guarantee of domain expertise. An instruction-tuned model may follow the requested format while still making a poor chart choice or inventing a field. Prompt-only generation therefore provides a useful baseline for measuring how much of the task can be solved from the pretrained and instruction-tuned model without retrieval or task-specific adapter training.

2.4.4 Limitations and Hallucination

LLMs can produce fluent statements that are not supported by the input or by an external source. The literature commonly refers to this problem as hallucination in natural-language generation [Ji et al. \(2023\)](#). In the present use case, a hallucination can be an invented data column, an unsupported KPI, an interaction that is not possible with the given data, or a rationale that cites a principle without applying it to the current brief.

A syntactically valid JSON object does not remove this problem. It only means that the output can be parsed according to a format. The project therefore separates structural validity from semantic correctness and grounding. These distinctions are needed when comparing prompt-only, RAG, fine-tuned, and combined methods.

2.5 Parameter-Efficient Fine-Tuning

Full fine-tuning updates all trainable parameters of a pretrained model for a downstream task. This can be expensive for LLMs because the optimizer must maintain additional states and because a separate model copy may be needed for each task. PEFT reduces this cost by keeping most pretrained parameters fixed and learning only a small task-specific component. Adapter-based transfer learning established this general idea by inserting small trainable modules while sharing the original model parameters across tasks [Houlsby et al. \(2019\)](#).

PEFT is especially relevant when a study compares several models, methods, or random seeds. The learned task-specific component can be stored separately from the base model, and the same base model can be reused across conditions. This reduces storage and makes the intervention easier to describe. It does not mean that PEFT is always as accurate as full fine-tuning. The result depends on the task, the data, the model, and the chosen PEFT method.

2.5.1 LoRA

Low-Rank Adaptation (LoRA) learns a low-rank update to selected weight matrices while keeping the pretrained weights frozen [Hu et al. \(2022\)](#). Let W_0 be a pretrained weight matrix. A LoRA update can be written as

$$W' = W_0 + \frac{\alpha}{r}BA,$$

where $A \in \mathbb{R}^{r \times d_{\text{in}}}$ and $B \in \mathbb{R}^{d_{\text{out}} \times r}$ are trainable matrices, r is the adapter rank, and α is a scaling factor. The rank r is chosen to be much smaller than the dimensions of the original matrix. During training, W_0 is frozen and only A and B are updated.

This factorization reduces the number of trainable parameters and allows a task-specific adapter to be saved without copying the entire base model. It also defines a clear experimental intervention: the base model remains the same, while the learned low-rank update changes the model's behaviour for the target task. LoRA

was originally evaluated in several language-model settings, but its performance is not independent of rank, target modules, data, and training configuration.

2.5.2 QLoRA

QLoRA combines LoRA with low-bit storage of the frozen base model [Dettmers et al. \(2023\)](#). The base weights are stored in a 4-bit representation and are dequantized for the computation needed during forward and backward passes. The gradients are used to update the LoRA adapters rather than the quantized base weights. QLoRA therefore does not mean that the adapter itself is trained as a 4-bit copy of the model; the important distinction is between the frozen quantized base and the trainable adapter parameters.

The method introduces several techniques to reduce memory use. NormalFloat4 (NF4) is used for the quantized base weights, double quantization reduces the storage needed for quantization constants, and paged optimizers help manage memory peaks during training. The original QLoRA paper reports that this combination can make fine-tuning much larger models possible on a single GPU while retaining strong task performance. Those reported results motivate the method, but they are not a guarantee for every model, dataset, or hardware setup.

QLoRA is therefore best understood as a resource-aware implementation of low-rank adaptation. It retains the parameter-sharing and adapter-storage properties of LoRA while lowering the memory required to load the frozen base model. Quantization can also introduce approximation effects, and the final outcome remains sensitive to rank, learning rate, sequence length, data quality, and hardware precision.

2.5.3 Why PEFT for This Thesis

The research task has three properties that support a PEFT design. First, the target is a structured recommendation task rather than the training of a new language model. The model must learn a project-specific mapping from a dashboard brief to a multi-field design object. Second, the study compares separate interventions, including prompt-only generation, retrieval, fine-tuning, and fine-tuning with retrieval. Third, the project is intended to run several model and seed conditions under finite GPU memory and storage. PEFT addresses these properties by keeping the base model reusable and by representing the task-specific intervention as a separate adapter. The decision is therefore supported by both the transfer-learning literature and the experimental design of this thesis.

2.5.4 Why QLoRA for This Thesis

QLoRA is selected as the main PEFT method because it combines the low-rank task update with a quantized frozen base model. This combination is directly relevant to the resource constraint of the project. The 4-bit base loading, NF4 representation, double quantization, and paged optimizer techniques reduce the memory needed during training, while the LoRA adapter keeps the learned task component small and separable [Dettmers et al. \(2023\)](#). This makes the planned comparison of models, methods, and seeds more feasible than a full-parameter training setup under the same hardware budget. The claim is a feasibility and experimental-control argument; it is not a claim that QLoRA is the best fine-tuning algorithm for all tasks.

QLoRA also supports a clean comparison between the main conditions. Method A uses the base model with the shared prompt. Method B adds retrieved context without an adapter. Method C adds a QLoRA adapter to the base model. Method D reuses the compatible adapter from Method C and adds the same retrieval procedure used by Method B. This factorization makes the intended comparison explicit: retrieval is an inference-time source of context, while QLoRA changes the learned parameters. Reusing an adapter with the same seed for C and D removes one unnecessary source of variation.

The alternative methods remain scientifically relevant. DoRA may change the capacity of the low-rank update by separating magnitude and direction. RSLoRA changes the rank scaling and is relevant when different ranks are compared. AdaLoRA changes the distribution of the parameter budget, and GaLore changes the memory strategy while allowing full-parameter learning. Choosing one of these methods as the main intervention would answer a different research question or introduce a different control problem. They are therefore better treated as optional ablations in this thesis unless the experimental plan is expanded explicitly.

The choice also has limitations. QLoRA can be sensitive to quantization settings, adapter rank, target modules, learning rate, sequence length, and the quality of the supervision. A small adapter may reproduce the formatting and recurring patterns of the training data without learning a robust design principle. In addition, the project-specific training data may contain regularities that make internal diagnostic scores look strong without proving dashboard quality. These limitations motivate the separate evaluation of format, chart and encoding behaviour, grounding, robustness, and usefulness in human evaluation.

2.6 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) combines a generative model with an external knowledge source. Lewis et al. [Lewis et al. \(2020\)](#) describe RAG as a framework in which a generator conditions its output on information retrieved from non-parametric memory in addition to its learned parameters. The central idea is that information needed for a response does not have to be stored only in the model weights. It can be selected from a collection of documents at inference time.

A typical RAG pipeline starts with a document or guideline corpus. The documents are cleaned and divided into passages, and the passages are stored in an index. For a new input, a retriever scores the passages and selects a small set of candidates. These passages are inserted into the model context together with the original user input. The LLM then generates an answer conditioned on both sources. Each step affects the final result: missing documents, poor chunk boundaries, an unsuitable query, or irrelevant retrieved passages can reduce the usefulness of the generated answer [Lewis et al. \(2020\)](#).

The current project uses a small, repository-local collection of visualization guidance and a lexical retrieval procedure. This choice is appropriate for a controlled experiment because the corpus can be inspected, versioned, and kept constant across runs. It also makes the retrieved evidence visible in the run artifacts. The use of a simple retriever is not a claim that lexical retrieval is universally superior to dense retrieval. It is a controlled implementation decision for testing whether explicit design guidance changes the generated recommendation.

2.6.1 RAG and Fine-Tuning as Different Interventions

Fine-tuning and RAG add information to a model in different ways. Fine-tuning changes model parameters and can teach recurring task formats, domain patterns, and output behaviour. RAG leaves the base parameters unchanged during inference and supplies explicit context for the current input. The two mechanisms can therefore complement each other, but they can also fail differently. A fine-tuned model may learn a useful pattern but apply it incorrectly, while a RAG system may retrieve the right passage but fail to use it in the final answer.

This distinction motivates the A/B/C/D design of the thesis. The prompt-only condition provides a base reference. The RAG condition tests the addition of retrieved guidance. The fine-tuned condition tests the addition of a QLoRA adapter. The combined condition tests whether retrieval remains useful after task-specific adaptation. The comparison is meaningful only when the prompt construction, model, generation settings, corpus, retriever, and adapter provenance are controlled as described in the methods chapter.

2.6.2 RAG Limitations and Grounding

RAG does not guarantee that an answer is grounded. A relevant passage may be absent from the corpus, or retrieval may return an irrelevant passage. The model may ignore a relevant passage, combine several passages incorrectly, or add an unsupported claim. A response can also mention retrieved concepts without applying them to the actual data columns and goals in the brief.

For this reason, retrieval availability and retrieval usefulness should be reported separately. The project records retrieved passages and evaluates grounding as a separate dimension. A grounding score should be interpreted as evidence about support for claims under the chosen corpus and metric. It should not be treated as a complete measure of visual-design quality or as proof that RAG improves every recommendation [Lewis et al. \(2020\)](#).

2.7 Structured LLM Outputs

Many applications cannot use free-form text directly. They require an output that can be parsed and passed to another software component. JSON is widely used for this purpose because it represents nested objects and arrays in a machine-readable format. A structured output contract makes the expected interface explicit, but it does not make the content automatically correct.

2.7.1 JSON Schema and Schema-Constrained Generation

A JSON Schema describes the expected structure of a JSON instance. It can specify property names, data types, required fields, arrays, nested objects, and allowed values. The JSON Schema Draft 2020-12 specification defines the vocabulary and validation rules used to describe such instances [JSON Schema \(2022\)](#). A schema can therefore be used after generation to check whether an output is structurally valid.

Schema-constrained generation applies these restrictions during decoding or through a generation library so that the model is less likely to produce invalid structures. Post-hoc validation applies the schema after the model has generated text. The two approaches are related but not identical. Constrained decoding can reduce

syntax errors, while post-hoc validation remains necessary when the output is produced by an unconstrained model or when application-specific semantic checks are required.

Recent work on structured generation, including JSONSchemaBench, evaluates structured-output systems across many real-world schemas and separates schema coverage, efficiency, and output quality [Geng et al. \(2025\)](#). This distinction is important for the thesis. A system can satisfy the formal shape of a schema while producing a chart type that does not fit the task or a rationale that is not supported by the brief.

2.7.2 Parsing, Validation, and Semantic Correctness

The evaluation should distinguish between an output that cannot be parsed, an output that parses but violates the schema, and an output that is schema-valid but semantically incorrect. The first case is a generation or parsing failure. The second case is a structural failure. The third case is a content failure that requires task-specific checks or human judgment.

In this project, the typed models in `src/core/schemas.py` define the expected recommendation object and the parser records failures. The prompt also restricts task and chart values to the project’s controlled vocabulary. These checks make the output comparable across methods, but they do not prove that the model selected a good chart, used an available column, or gave a useful rationale. Those questions require additional evaluation layers.

2.7.3 Structured Outputs in This Thesis

The target of the thesis is a structured dashboard recommendation, not an open-ended design essay. The six top-level fields provide a stable interface for comparing prompt-only generation, RAG, QLoRA fine-tuning, and the combined condition. They also make it possible to evaluate different aspects separately: chart and encoding choices, layout and styling, interactions, rationales, and structural validity.

The schema should therefore be understood as an interface contract. It controls how the recommendation is represented and how it can be evaluated or consumed by software. It does not replace visualization knowledge, retrieval evidence, or human assessment. This separation between structure and meaning is carried into the evaluation and results chapters.

Chapter 3

Related Work

This chapter reviews the research that is most closely related to the dashboard-design recommendation task studied in this thesis. It first discusses visualization recommendation systems that encode design knowledge as rules, constraints, or interactive recommendations. It then covers natural-language interfaces and neural approaches for translating language into visualization specifications. The next section examines the datasets and benchmarks that support this research area. After that, the chapter reviews recent uses of large language models (LLMs) for analytic specifications, chart generation, and visualization assistance. The final technical sections discuss parameter-efficient fine-tuning (PEFT), retrieval-augmented generation (RAG), and the evaluation of structured visualization outputs. The chapter ends by identifying the remaining gap and positioning the present study within this literature.

The review makes a distinction between a visualization recommendation and a complete dashboard-design recommendation. A visualization recommendation usually selects one view or one visualization specification. The task in this thesis is broader: a dashboard brief is mapped to several coordinated design decisions, including KPI-to-chart mappings, layout, styling, interactions, and rationales. This distinction is important because a method can perform well on chart selection without solving the other parts of dashboard design. The discussion below therefore compares prior systems according to their input, output, design knowledge, and evaluation scope rather than treating all of them as equivalent.

3.1 Visualization Recommendation Systems

Visualization recommendation systems are an established research area. Their central goal is to reduce the manual effort required to choose data transformations and visual encodings. The main approaches differ in how they represent design knowledge and how much control remains with the analyst. Some systems formulate recommendation as a search problem over possible graphical presentations. Others use explicit constraints to filter or rank views. A third group integrates recommendations into an interactive interface so that users can explore and refine the results.

3.1.1 Explicit design knowledge and constraint-based recommendation

Mackinlay’s APT system is an early and influential example of automated visualization design. It treats the construction of a graphical presentation as a systematic search over possible encodings and evaluates candidates using the ideas of expressiveness and effectiveness [Mackinlay \(1986\)](#). Expressiveness concerns whether a presentation represents the intended data without adding unsupported information. Effectiveness concerns how well the chosen graphical properties support the required perceptual task. This formulation is relevant to the present thesis because it shows that visualization choice can be described as a structured design problem rather than as an arbitrary stylistic decision.

The same line of work was later integrated into a practical visual-analysis environment. Show Me added automatic presentation commands and defaults to Tableau, including automatic mark-type selection and the generation of views for one or more selected fields [Mackinlay et al. \(2007\)](#). The system did not remove the analyst from the process. Instead, it provided a fast starting point while allowing the analyst to continue the analysis and change the result. This human-in-the-loop character is important: a recommender can support design without claiming that one automatically generated view is the only valid answer.

Draco provides a more explicit formal representation of visualization design knowledge. Moritz et al. [Moritz et al. \(2019\)](#) express design principles as constraints over visualization specifications. Hard constraints can exclude invalid or unsuitable specifications, while soft constraints can be used to rank alternatives. The framework also supports the use of empirical evidence when assigning weights to design preferences. Draco therefore connects a visualization grammar with a knowledge model that can be inspected and extended. This is close to the motivation for a structured dashboard recommendation: design knowledge should be available as a reason for a recommendation, not only as an unexplained model output.

These systems also illustrate a limitation of rule-based recommendation. The quality of a result depends on the coverage and correctness of the encoded design knowledge. A rule can express that a temporal variable is often useful for a trend view, but it cannot by itself understand every way in which a user may describe a business goal, nor can it reliably infer missing context. A rule system must also decide how to behave when

several views satisfy the same constraints. These issues motivate the later development of natural-language interfaces and learned generation methods.

3.1.2 Mixed-initiative exploration and visualization galleries

Voyager addresses the problem from the perspective of exploratory analysis. Instead of requiring the analyst to specify every view manually, Voyager presents a gallery of automatically generated visualizations that can be browsed and refined [Wongsuphasawat et al. \(2016\)](#). The system combines visualization recommendation with faceted browsing and uses statistical and perceptual measures to organize the recommended views. Its user study showed that this interaction model can support the exploration of unfamiliar data and increase the number of variables considered by users.

Voyager 2 extends the same general idea to partial view specifications [Wongsuphasawat et al. \(2017\)](#). An analyst can specify some parts of a view and leave other parts open. The system then recommends compatible completions. This makes the interaction more flexible than a fully manual specification process and more controlled than a system that generates a complete view without user guidance. The work demonstrates that recommendation can be useful at different stages of analysis: it can support initial exploration, but it can also complete a partially formed design.

The mixed-initiative perspective provides two lessons for the thesis. First, recommendation should be understood as support for a user task, not only as an optimization over chart types. Second, several candidate views can be useful when the user’s goal is exploratory or when the input does not determine one unique encoding. These lessons are consistent with later benchmark work on ambiguity and with the project decision to record chart and rationale information separately. They also show why a dashboard recommendation should be evaluated in relation to users, goals, and constraints rather than by chart-type accuracy alone.

3.1.3 Relation to the present dashboard task

The systems described above mainly operate on structured data tables, view specifications, or a limited set of user selections. Their outputs are usually individual views, view galleries, or completions of a visualization specification. They do not normally generate one coordinated object that combines KPI mappings, multiple views, dashboard layout, styling, interaction behaviour, and textual rationales.

This difference does not make the earlier systems less relevant. They provide the design concepts that a dashboard recommendation must respect. Their constraints can describe valid encodings, their perceptual criteria can inform chart choice, and their mixed-initiative interfaces show how recommendations can remain understandable to users. The contribution of the present thesis is positioned at a different level: it studies whether an LLM can produce a structured, multi-component recommendation from a dashboard brief while preserving the design and evaluation distinctions established by this earlier work.

3.2 Natural-Language Interfaces and NL-to-Visualization

Natural-language interfaces for visualization reduce the need for users to know the syntax of a visualization grammar or the details of a data-processing language. A user can state an analytical intention in ordinary language, and the system translates the request into data attributes, analytic tasks, transformations, and visual encodings. This translation is difficult because natural-language requests are often incomplete, ambiguous, or dependent on the structure of the data.

3.2.1 Semantic parsing and conversational interfaces

NL4DV is a prominent toolkit for natural-language visualization interfaces. Given a tabular dataset and a natural-language query, it identifies data attributes and analytic tasks and returns a structured analytic specification with relevant visualizations [Narechania et al. \(2021\)](#). The output makes intermediate decisions visible. A developer or user can inspect which phrases were linked to which fields, which task was inferred, and which visualization was recommended. This explicit intermediate representation improves debuggability compared with a system that only returns a final chart.

The toolkit also shows the value of separating language understanding from visualization rendering. A query may mention a measure, a grouping field, a filter, and an operation such as a comparison or trend. These elements can be represented before a chart is selected. The separation makes it possible to evaluate task recognition and chart recommendation as related but distinct steps. It also provides a useful conceptual basis for the structured output used in this thesis, where chart mappings and rationales are stored alongside the broader dashboard fields.

Mitra et al. [Mitra et al. \(2022\)](#) extend NL4DV toward conversational interaction. Their system maintains multiple conversations, incorporates follow-up information into the analytic specification, and supports ambiguity resolution over several turns. This work is important because a single utterance does not always contain

enough information for a reliable visualization. A follow-up question can specify a missing field, clarify a comparison, or change the intended view without requiring the user to restate the entire request.

Parser-based systems have a clear advantage in transparency. Their intermediate structures can be inspected and their errors can often be connected to a particular parsing rule or mapping. Their limitation is that extending the rule set to cover new language patterns, complex transformations, and domain-specific terminology requires substantial manual work. This limitation is one reason why later systems use neural generation and, more recently, instruction-tuned LLMs.

3.2.2 Neural generation of visualization specifications

Data2Vis treats visualization generation as a sequence-to-sequence problem. Dibia and Demiralp [Dibia and Demiralp \(2019\)](#) use a recurrent neural architecture to generate Vega-Lite specifications from data descriptions. The work shows that a model can learn recurring relationships between data characteristics, transformations, and visualization syntax. It also demonstrates the attraction of generating a formal specification rather than an image: the result can be rendered, inspected, and evaluated programmatically.

Luo et al. [Luo et al. \(2022\)](#) study natural language to visualization through neural machine translation. Their approach treats the translation from a natural-language request to a visualization specification as a learned mapping. This direction reduces the need to write a separate rule for every linguistic variation. At the same time, it makes the training data more important because the model can learn regularities, errors, and biases from the examples provided to it.

NL2Viz introduces constrained syntax-guided synthesis for natural-language visualization [Wu et al. \(2022\)](#). The system combines the language input with data and program context and uses synthesis constraints to produce visualization programs that satisfy structural requirements. This is relevant to the present work because it treats generation as a combination of language interpretation and constrained program construction. It also shows that constraints remain useful even when a learned model is responsible for a large part of the translation process.

3.2.3 Common limitations of NL-to-visualization systems

The systems in this section generally focus on one query and one visualization, or on a conversation that incrementally refines one analytic specification. This scope is valuable, but it does not cover the complete dashboard problem. A dashboard recommendation must coordinate several KPIs and views and must also make decisions about spatial arrangement, visual hierarchy, colour use, interaction behaviour, and explanation. These decisions can depend on the audience and on the purpose of the dashboard, even when the underlying chart specifications are valid.

Natural-language input also creates a second problem: under-specification. A phrase such as “show performance” does not determine a measure, time grain, comparison group, or suitable level of aggregation. A system may infer a plausible choice, ask for clarification, or return several alternatives. The correct behaviour depends on the application and on the risk of making an unsupported assumption. This issue becomes central in the benchmark literature discussed next and motivates the use of explicit constraints and validation in the present thesis.

3.3 Datasets and Benchmarks for Visualization Generation

Datasets determine which parts of the visualization task a model can learn and which claims an evaluation can support. A dataset may contain natural-language queries, analytic-task labels, table schemas, visualization specifications, code, rendered images, or human judgements. These components are not interchangeable. A dataset that is suitable for chart-type prediction may not contain the information needed to train a model to recommend layout or interactions. The following benchmarks are therefore discussed according to their task and output coverage.

3.3.1 Quda and analytic-task recognition

Quda was introduced by Fu et al. [Fu et al. \(2020\)](#) for recognizing analytic tasks from free-form natural-language queries. It contains 14,035 queries, and each query is annotated with one or more analytic tasks. The authors first collected seed queries with data analysts and then used crowdsourced paraphrase generation and validation to increase linguistic diversity. The resulting data is useful for studying how different phrasings express tasks such as comparison, trend, or distribution.

Quda addresses an important subproblem, but it does not describe a complete visualization or dashboard design. It provides evidence about the language of analytical intent, not about the final selection of fields, encodings, layout, styling, or interaction. It can therefore support task-recognition research or data augmentation, but it cannot by itself serve as gold data for the full structured output required by this thesis.

3.3.2 nvBench and cross-domain NL-to-visualization

The nvBench benchmark was constructed to provide larger-scale supervision for natural-language to visualization research. Luo et al. [Luo et al. \(2021b\)](#) report 25,750 natural-language/visualization pairs derived from 750 tables across 105 domains. The benchmark is synthesized from natural-language-to-SQL resources, which provides broad coverage of domains and table structures. Its construction was also evaluated by experts and crowd workers, which is important because a synthetic benchmark still needs quality checks.

nvBench is useful because it links language with data context and a visualization target. It can support the study of field selection, aggregation, and basic chart specifications across domains. However, its synthetic construction also defines its scope. The benchmark is designed for a query-to-visualization task, not for the design of a complete dashboard for a named audience. It does not provide the full set of contextual and presentational decisions required by the target object in this thesis.

3.3.3 nvBench 2.0 and ambiguity

nvBench 2.0 addresses a limitation of single-target benchmarks: one query can have more than one reasonable interpretation. The published version contains 7,878 natural-language queries and 24,076 corresponding visualizations derived from 780 tables across 153 domains [Luo et al. \(2025\)](#). Its controlled ambiguity-injection pipeline starts from seed visualizations and creates alternative interpretations together with step-wise reasoning paths.

This one-to-many structure is important for visualization evaluation. Ambiguity can occur at the data level, for example when a word may refer to more than one column or when an aggregation is not stated. It can also occur at the visualization level, for example when “trend” could be represented by more than one chart type depending on the data and the purpose. A benchmark that stores only one target may incorrectly mark a defensible alternative as wrong. nvBench 2.0 does not solve the complete dashboard problem, but it provides a stronger basis for discussing uncertainty and multiple valid outputs.

3.3.4 What existing benchmarks cover

Together, Quda, nvBench, and nvBench 2.0 cover several important components: analytic-task recognition, natural-language variation, data and schema context, chart specifications, cross-domain supervision, and ambiguity. They do not cover the full mapping from a dashboard brief containing users, goals, KPIs, data fields, and constraints to a coordinated, multi-component dashboard-design recommendation.

This conclusion is a scope statement about the sources reviewed for this thesis, not a claim that no other visualization dataset exists. Other resources may cover chart code, chart images, visual question answering, or conversational changes to a visualization. The relevant point is that those resources do not automatically provide labels for every field in the target dashboard schema. The project dataset and its provenance are therefore described separately in Chapter 4 rather than being treated as a direct copy of one public benchmark.

3.4 LLMs for Visualization and Dashboard Assistance

Instruction-tuned LLMs have changed the design space for natural-language visualization systems. Instead of building a separate parser for every intent and language pattern, a general model can be prompted to interpret a brief and generate an analytic or visualization specification. This flexibility is useful for complex inputs, but it also makes it easier for unsupported assumptions and invalid structures to pass unnoticed. Recent work therefore combines LLMs with explicit prompts, intermediate representations, constraints, fine-tuning, or visual feedback.

3.4.1 LLMs for analytic specifications

Sah et al. [Sah et al. \(2024\)](#) study the generation of analytic specifications from natural-language queries with LLMs. Their prompt produces detected data attributes, inferred analytic tasks, and recommended visualizations. It also records mappings between input phrases and detected entities and states the design principles used for recommendations. The authors include conversational interaction and ambiguity detection in the prompt design and evaluate the approach with GPT-4.

The work is relevant to this thesis because it does not treat the LLM as an opaque chart generator. Instead, it asks the model to expose intermediate decisions that can be inspected and debugged. This approach is compatible with structured outputs and rationales. It also shows a practical limitation: a detailed prompt can improve the form of the result, but the model still depends on the data context and may fail when the request is ambiguous or when a requested field is not present.

3.4.2 Chart generation with LLMs

ChartGPT focuses on generating charts from abstract natural-language input [Tian et al. \(2025\)](#). The authors decompose chart generation into several steps so that each step addresses a more specific decision. They

also create task-specific training data and fine-tune a model to provide visualization knowledge that is not guaranteed to be present in a general language model. An interactive interface allows users to inspect and modify intermediate results.

ChartGPT demonstrates why decomposition and intermediate representations are useful when natural-language requests do not specify every visual parameter. It is closely related to the chart-selection part of this thesis, but its main target remains chart generation. The dashboard problem adds coordinated multi-view design and requires the system to express decisions about layout, styling, interactions, and rationales in the same structured response.

Text2Chart31 extends chart-generation research to a wider range of plot types and richer training records. Pesaran Zadeh et al. [Pesaran Zadeh et al. \(2024\)](#) introduce a dataset with 31 Matplotlib plot types and approximately 11.1 thousand tuples that combine descriptions, code, data tables, and plots. They also propose instruction tuning with automatic feedback, including reinforcement-learning components for chart-generation tasks. This work shows that domain-specific adaptation can be used not only to learn a chart label, but also to connect language, data, code, and the resulting plot.

Text2Chart31 is nevertheless different from the present target. Its central output is a chart or chart-generating program, whereas this thesis studies a structured dashboard-design recommendation. The distinction matters for both training and evaluation. A model can generate executable chart code and still provide an unsuitable dashboard hierarchy, inaccessible styling, or an interaction that is not supported by the data.

3.4.3 LLM-assisted visualization design

Visualization studies LLMs as design-feedback assistants rather than only as generators. Shin, Hong, and Elmqvist [Shin et al. \(2025\)](#) combine a preamble of visualization guidelines with perceptual filters that extract salient measures from a visualization image. Their longitudinal study with 13 visualization designers suggests that natural-language feedback can help designers refine visualizations. The work is relevant because it connects generated advice with measurable properties of a visual result and with an iterative design process.

LLM-assisted visualization retargeting addresses a related but different problem: adapting existing visualization implementations to new data. Snyder, Wang, and Drucker [Snyder et al. \(2025\)](#) compare direct code generation with a more constrained program-synthesis approach that provides structural information such as visual encodings. Their analysis reports that both approaches can struggle when the new data has not been transformed appropriately. This finding is a useful warning for the present thesis. Correct language generation cannot compensate for a mismatch between the requested design and the data representation on which that design depends.

3.4.4 General lessons from the LLM literature

The recent literature supports the use of LLMs as flexible interfaces for visualization work, but it does not support treating fluent output as reliable design evidence. Successful systems add structure in different ways: they expose analytic specifications, decompose chart generation, use task-specific data, apply constraints, or provide feedback based on the rendered result. These approaches address different failure modes and are not substitutes for one another.

For the present thesis, this leads to a layered system design. The LLM is used as a generation component, while the prompt specifies the output contract, retrieval supplies optional external guidance, fine-tuning provides task adaptation, and validation checks the generated structure. The evaluation must then distinguish syntactic validity from chart and dashboard quality. This separation is the basis for the comparison developed in the following sections.

3.5 Parameter-Efficient Fine-Tuning for Structured Generation

The background chapter introduced PEFT and its main variants. In related work, the important question is how these adaptation methods change the comparison between a general model and a task-specific model. Fine-tuning can help a model learn the vocabulary, field relations, and output patterns of a visualization task. It can also make the model reproduce regularities of a particular dataset without learning a general design principle. The choice of adaptation method therefore affects both resource use and scientific interpretation.

3.5.1 LoRA, QLoRA, and alternative adaptation methods

LoRA learns low-rank updates to selected weight matrices while keeping the pretrained model weights frozen [Hu et al. \(2022\)](#). The adapter can be stored separately from the base model, which makes it possible to reuse one base model across several task-specific conditions. This property is useful for controlled experiments because the intervention can be described as a learned update rather than as a completely different model.

QLoRA combines the LoRA update with low-bit storage of the frozen base model [Dettmers et al. \(2023\)](#). The method uses 4-bit quantization, NormalFloat4 (NF4), double quantization, and paged optimizers to

reduce memory use during fine-tuning. The base model remains frozen while the LoRA parameters are trained. QLoRA therefore changes the memory representation of the base model without changing the basic idea of a separate low-rank task update. The reported results in the original paper motivate its use as a resource-aware method, but they do not guarantee the same outcome on a different model, dataset, or hardware setup.

Other adaptation methods make different trade-offs. Adapter tuning inserts small trainable bottleneck modules into the network [Houlsby et al. \(2019\)](#). Prefix tuning and prompt tuning learn continuous task-specific vectors that condition a frozen model through virtual tokens or soft prompt embeddings [Li and Liang \(2021\)](#); [Lester et al. \(2021\)](#). BitFit updates only bias parameters, while IA3 learns vectors that rescale internal activations [Ben Zaken et al. \(2022\)](#); [Liu et al. \(2022\)](#). These methods can require fewer trainable parameters, but their update structure is different from a LoRA adapter.

Several methods modify the allocation or scaling of low-rank updates. AdaLoRA allocates different ranks to different matrices according to their estimated importance [Zhang et al. \(2023\)](#). DoRA decomposes the weight update into magnitude and direction and applies low-rank adaptation to the directional component [Liu et al. \(2024\)](#). Rank-Stabilized LoRA changes the scaling rule for the low-rank update and is intended to improve behaviour at higher ranks [Kalajdzievski \(2023\)](#). These methods address specific limitations of standard LoRA, but they also introduce additional choices that must be controlled in an ablation.

GaLore is related to memory-efficient training but is not PEFT in the strict sense used here. It projects gradients into a low-rank subspace while still allowing full-parameter training [Zhao et al. \(2024\)](#). A GaLore comparison therefore changes not only the adapter algorithm but also the set of parameters that can be updated. Treating GaLore as an interchangeable name for QLoRA would make the experimental interpretation incorrect.

3.5.2 Fine-tuning in visualization-generation research

ChartGPT and Text2Chart31 provide visualization-specific evidence that task adaptation can be useful for chart generation. In both cases, the training data contains information about visualization tasks that is not represented by a generic language-instruction corpus alone. The results support the idea that a model may need examples of data fields, chart parameters, chart code, or intermediate decisions before it can reliably generate domain-specific visualization outputs.

The present thesis extends this idea from one chart to a structured dashboard recommendation. The required output contains several types of information, and the model must learn their names, relations, and allowed structure. Fine-tuning is therefore not evaluated only by whether a model produces fluent text. It is evaluated through parsing, schema validation, field completeness, chart and encoding behaviour, robustness, grounding, and human usefulness. This broader evaluation is necessary because a model can learn the output format without learning appropriate dashboard design.

3.5.3 Scientific reasons for selecting QLoRA in this thesis

QLoRA is selected as the main fine-tuning method for this thesis for four connected reasons. The first reason is resource feasibility. The project compares several controlled conditions and may use more than one model size. Storing the frozen base model in a 4-bit representation reduces the memory required during training, while the adapter keeps the task-specific parameters small. This makes the planned experiments more realistic under a limited GPU budget than full-parameter fine-tuning with the same base models. This is a feasibility argument, not a claim that QLoRA is the most accurate adaptation method for every task.

The second reason is experimental control. A QLoRA run leaves the base model identifiable and stores the learned task intervention as an adapter. In the project design, the prompt-only condition does not use an adapter, the RAG condition adds retrieved context without changing the model parameters, the QLoRA condition adds the adapter, and the combined condition uses the compatible adapter together with the same retrieval procedure. This factorization makes the intended comparison clearer than a design in which the retrieval mechanism and the training algorithm are changed at the same time.

The third reason is the structure of the research task. The goal is to adapt an existing instruction-tuned causal language model to produce a project-specific structured object. The task is not to train a new language model or to learn a new optimizer. Low-rank adaptation is a suitable intervention because it can change the model's behaviour for the target output while preserving the reusable base model. The adapter can also be checked for compatibility with the model, dataset, training configuration, and seed before it is used by the combined method.

The fourth reason is comparability with the alternatives that were considered. DoRA changes the parameter decomposition, AdaLoRA changes the rank allocation, RSLORA changes the scaling behaviour, and GaLore changes the memory strategy while allowing full-parameter learning. Each of these methods could support a useful ablation, but choosing one as the main method would answer a different question or introduce a different control problem. The project therefore keeps QLoRA as the main fine-tuning intervention

and treats the other algorithms as optional ablation paths rather than presenting them as universally worse or better.

This choice has clear limitations. QLoRA can be sensitive to the quantization configuration, adapter rank, target modules, learning rate, sequence length, and supervision quality. A small adapter can learn recurring formatting patterns without learning robust visualization principles. Quantization also introduces an approximation that may affect the result differently across model families. These limitations are addressed by recording configuration and adapter provenance and by evaluating the output at several levels instead of relying on training loss alone.

The seed design is part of the same control principle. In this thesis, three seeds are reserved for the smallest final model, Qwen3-1.7B, to check whether the main result is stable under different random conditions. These runs are a stability check; three seeds do not by themselves prove generalization. If the predefined stability criterion is met, the remaining planned model and method configurations will be run with Seed 42. If it is not met, the result must be reported as unstable and the plan must be reconsidered rather than silently treating Seed 42 as representative. The thesis will not describe the remaining single-seed runs as multi-seed evidence.

3.6 Retrieval-Augmented Generation for Domain-Grounded Generation

RAG provides a second way to add task-relevant information to a language model. Fine-tuning changes model parameters and stores knowledge in a learned task component. RAG leaves the model parameters unchanged at inference time and retrieves passages from an external knowledge base for each input. The two interventions can therefore be compared separately and can also be combined.

3.6.1 Retrieval and generation

Lewis et al. [Lewis et al. \(2020\)](#) define RAG as a combination of a parametric generator and a non-parametric memory. A document collection is indexed, a retriever selects passages for a new input, and the generator produces an answer conditioned on the input and the retrieved context. The approach is useful when relevant information is too specific, too recent, or too large to depend only on the model parameters.

For visualization design, an external knowledge base can contain design guidelines, perceptual findings, accessibility recommendations, and domain-specific documentation. Retrieved passages can provide a source for a rationale or can remind the model of a design constraint. This is especially useful when a recommendation must be explained to a user. The retrieved text is not itself a guarantee of a correct design; it is an additional condition under which the model generates its output.

3.6.2 RAG and fine-tuning as different interventions

RAG and fine-tuning solve different parts of the adaptation problem. Fine-tuning can teach stable output conventions, field names, and recurring relations across the training examples. RAG can provide explicit information that can be changed without retraining the model. Fine-tuning is therefore a parameter intervention, while RAG is a context intervention. Combining them may be useful, but it also makes it necessary to verify which component is responsible for an observed change.

This distinction motivates the four conditions used in the project comparison. Method A is prompt-only generation and serves as the base condition. Method B adds retrieval while keeping the base model unchanged. Method C adds QLoRA fine-tuning without retrieval. Method D combines the QLoRA adapter with retrieval. The design is meaningful only if the prompt, tokenizer, model, generation settings, retrieval procedure, dataset, and adapter provenance are controlled. A difference between methods cannot be attributed to retrieval if the context length or post-processing rules also change.

3.6.3 RAG limitations in the present task

RAG introduces its own sources of error. A retriever may miss a relevant passage, return text that is only loosely related to the brief, or return several passages that do not agree. Chunk boundaries and query wording affect which evidence is available. The context may also exceed the model’s input budget, or the generator may ignore a relevant passage even when it is present. These are system-level issues rather than simple model-quality issues.

The project therefore records retrieval results and knowledge-base identity separately from the generated recommendation. Grounding is evaluated as a separate dimension and must be interpreted together with its measurement mode. A lexical word-overlap score can indicate that terms from a retrieved passage appear in a rationale, but it is not a complete faithfulness assessment. The presence of retrieved text also does not prove that the chart, layout, or interaction recommendation is appropriate for the user’s task.

3.7 Evaluation of Structured and Visualization Outputs

Evaluation is difficult because a dashboard recommendation can be correct in one dimension and weak in another. A response may be valid JSON but select the wrong field. It may select a defensible chart but propose an unusable layout. It may provide a plausible rationale that is not supported by the input or by the retrieved evidence. A meaningful evaluation must therefore separate structural validity, semantic correctness, robustness, grounding, and human usefulness.

3.7.1 Structural validity

The first level of evaluation asks whether the output can be used by software. The evaluation should distinguish successful extraction of a JSON object from validity against the required JSON Schema. It should also distinguish schema validity from completeness: all required keys may be present even when some values are empty or unusable. These checks answer different questions and should not be merged into one score.

JSON Schema Draft 2020-12 defines a formal language for describing JSON instances, including properties, types, required fields, arrays, and allowed values [JSON Schema \(2022\)](#). Structured-output research such as JSONSchemaBench evaluates more than parse success and separates schema coverage, generation efficiency, and output quality [Geng et al. \(2025\)](#). This distinction is directly relevant to the thesis. A model can satisfy the formal structure while producing an unsupported KPI, an invalid chart choice, or an empty rationale.

3.7.2 Semantic and visualization correctness

The second level concerns the meaning of the recommendation. Relevant checks include whether the model identifies the intended analytical task, uses available fields, preserves constraints, chooses an appropriate chart type, and assigns fields to the expected visual channels. Aggregation and temporal granularity may also be important. These checks should be reported separately when a single exact-match score would hide the source of an error.

Task-based visualization studies show that the effectiveness of an encoding depends on the analytical task and on the distribution of the data, not only on the name of the chart [Saket et al. \(2019\)](#); [Kim and Heer \(2018\)](#). This supports an evaluation design that considers task and data context. It also explains why an automatically generated chart should not be judged only by whether its label matches one synthetic target.

Exact-match evaluation has a further limitation when more than one visualization is defensible. nvBench 2.0 makes this problem explicit by associating ambiguous queries with multiple valid visualizations [Luo et al. \(2025\)](#). The present thesis therefore treats independent human-effectiveness evidence and human evaluation as important complements to internal synthetic-gold metrics. A synthetic target can be useful for checking pipeline behaviour, but it should not automatically be interpreted as the only correct design.

3.7.3 Robustness and grounding

A useful system should not change its main recommendation because a brief was harmlessly paraphrased. It should also react appropriately when essential information is missing. Behavioural testing with CheckList argues for systematic tests of capabilities and failure modes instead of relying only on one aggregate accuracy value [Ribeiro et al. \(2020\)](#). For the dashboard task, this means testing paraphrases, incomplete briefs, unsupported fields, and other controlled changes.

Robustness is not the same as correctness. A model can consistently repeat the same wrong chart under several paraphrases. Similarly, a model can mention words that overlap with a retrieved source without using that source correctly. The project therefore keeps paraphrase consistency, missing-information behaviour, and grounding support separate from chart and dashboard quality. This separation makes it possible to identify whether an intervention improves reliability, content, or only the surface form of the answer.

3.7.4 Human evaluation and LLM-based judges

Human evaluation is needed for dimensions that are difficult to reduce to a formal exact match. These dimensions include the appropriateness of the chart for the stated goal, the quality of the dashboard layout, styling and accessibility, interaction design, rationale quality, and overall usefulness. Best-practice guidance for evaluating generated language recommends a clear rubric, controlled instructions, independent ratings, and agreement analysis [van der Lee et al. \(2019\)](#). The same principles apply when people evaluate structured dashboard recommendations.

An LLM can also be used as an auxiliary judge, but its score should not automatically replace human judgement. A model-based judge may be sensitive to prompt wording, may prefer fluent explanations, and may not detect an error in a chart-field mapping. In this thesis, an LLM judge is therefore treated as supportive evidence only. Claims about usefulness and actionability depend on the human-evaluation protocol and its reliability, not on an unvalidated automatic judge score.

3.7.5 Evaluation layers for this thesis

The project evaluation protocol assembles evidence across four layers. The first layer evaluates chart choice against an independent human-effectiveness reference where coverage exists. The second layer evaluates parsing, schema validity, completeness, robustness, and grounding on held-out and perturbed inputs. The third layer, if retained in the final claim scope, compares structural properties of generated dashboards with a real-dashboard corpus and is interpreted descriptively rather than as evidence of optimality. The fourth layer uses blind human ratings for quality and usefulness.

This layered design also protects against circular conclusions. The project’s generated chart labels and internal synthetic test targets can be useful for diagnosing the pipeline, but they share part of the same task construction and are not sufficient as the primary evidence for chart quality. Independent references, real briefs, and human ratings are kept separate from the training and augmentation lineage. The detailed dataset and evaluation procedures are described in the following chapters.

3.8 Research Gap and Positioning of This Thesis

3.8.1 Gap in task scope

Prior work provides strong solutions for individual parts of visualization design. Constraint-based systems formalize visual encoding knowledge. Natural-language interfaces identify analytic tasks and relevant fields. Neural and LLM-based systems generate chart specifications or chart code. RAG can provide external guidance, and PEFT can adapt a model to a task. The remaining problem is how to combine these capabilities for a dashboard brief that contains user context, goals, KPIs, data fields, and constraints and that requires a coordinated multi-component output.

The target of this thesis is not a claim that LLMs replace visualization recommendation research. It is a narrower study of whether an LLM-based pipeline can produce a structured dashboard-design recommendation that is useful beyond a single chart label. The output includes KPI-to-chart mappings together with layout, styling, interactions, and rationales. This makes the task broader than the public query-to-visualization benchmarks reviewed above and creates a need for separate checks of each output component.

3.8.2 Gap in controlled intervention comparisons

The literature often studies one system, one adaptation method, or one generation pipeline. Such studies are valuable for introducing a method, but they do not always isolate the contribution of retrieval from the contribution of task-specific fine-tuning. The project addresses this issue with four controlled conditions: prompt-only generation, retrieval-augmented generation, QLoRA fine-tuning, and QLoRA fine-tuning combined with retrieval.

The comparison is designed as a factorization of two interventions. Retrieval changes the context given to the generator, whereas QLoRA changes the task-specific parameter state. This does not remove every possible confounder, and the model family and seed coverage must still be reported. It does, however, provide a clearer basis for asking whether retrieval remains useful after task adaptation and whether fine-tuning improves the structured output without retrieval.

3.8.3 Gap in evaluation and provenance

A second gap concerns evaluation breadth and provenance. A valid JSON response is not necessarily a good dashboard recommendation, and a high internal chart score is not necessarily independent evidence. The present study therefore records dataset versions, source lineage, knowledge-base identity, model configuration, seed, adapter provenance, raw outputs, parsing results, and evaluation status. Training and augmentation data are kept separate from the independent evidence used for final quality claims.

The seed policy follows the same principle of bounded interpretation. Three seeds are used only for the Qwen3-1.7B stability check. If the predefined stability criterion is met, the remaining planned configurations use Seed 42. Results from those runs will be reported as single-seed results unless additional seeds are actually executed. This wording prevents a stability check on one model from being presented as variance evidence for every model and method.

3.8.4 Position of the thesis

The thesis is positioned at the intersection of visualization recommendation, natural-language visualization, structured LLM generation, PEFT, RAG, and human-centred evaluation. Its contribution is an experimentally controlled study of these components for a dashboard-level output contract. QLoRA is used as the main fine-tuning method because it offers a resource-aware, adapter-based intervention that is compatible with the project hardware and comparison design. The alternative methods remain relevant as possible ablations,

while the main scientific conclusions are restricted to the models, data, seeds, and evaluation layers that are actually run and verified.

This positioning leads to the next chapters. Chapter 4 describes the construction and provenance of the data used for training and evaluation. Chapter 5 formalizes the output schema and the four method conditions. The later evaluation chapters then test structure, chart behaviour, robustness, grounding, and human usefulness separately.

Chapter 4

Dataset Construction and Provenance

This chapter describes how the data for the dashboard-design recommendation task was constructed and how its provenance was preserved. The task is implemented as a structured transformation from a dashboard brief to a machine-readable recommendation. A brief contains information about the intended users, their goals, relevant key performance indicators (KPIs), available data columns, and analytical constraints. The target is not a rendered dashboard image and not only a chart label. It is a structured object that contains a context summary, KPI-to-chart mappings, encoding information, layout, styling, interactions, and rationales. The dataset must therefore preserve analytical evidence while also representing the presentation fields needed by the downstream models.

Provenance is central to this thesis because the four evaluated systems—prompt-only generation, retrieval-augmented generation (RAG), QLoRA fine-tuning, and QLoRA fine-tuning combined with RAG—must be compared on the same task and under the same data boundary. A dataset that silently mixes source observations, deterministic transformations, and model-generated annotations would make such a comparison difficult to interpret. The construction process consequently separates the source-grounded analytical core from derived labels and generated design annotations. It also protects the held-out test set from the later augmentation steps. These controls reduce the circularity risk that affected the earlier synthetic-only approach, although they do not make the final dataset fully human-annotated or externally independent.

The chapter first defines the dataset requirements and explains why the original nvBench corpus was selected as the primary source. It then describes the target schema, the evidence classes, and the source-faithful transformation of natural-language queries, relational schemas, SQL/VQL, and visualization encodings. The following sections explain the progressive quality checks, the Tier A/B/C policy, deduplication, group-aware splitting, and the construction of the source-grounded `dashboard_v3` corpus. The chapter then documents the constrained LLM enrichment of six presentation fields, the controlled `dashboard_v4` augmentation, and the `dashboard_v4_1` semantic repair. It closes by reporting the final mixed lineage, freeze and versioning procedure, and the threats to validity that bound the interpretation of the dataset.

4.1 Task Definition and Dataset Requirements

The dataset represents the supervised transformation

dashboard brief $\longrightarrow$ structured dashboard recommendation.

The input side of this transformation is a compact dashboard brief. In the project schema, the brief contains `users`, `goals`, `kpis`, `columns`, and `constraints`. The user field describes the intended audience in a short textual form. The goals describe what that audience should understand, compare, monitor, or decide. KPIs identify the measures that are relevant to those goals. The columns field records the available data fields and their types or roles. Constraints describe conditions such as filtering, grouping, sorting, temporal granularity, refresh expectations, or restrictions on the interpretation of the data. These elements are not independent. For example, the same numerical field may support a different design for a comparison task than for a trend or distribution task, and a KPI may be unsuitable when the available field is an identifier rather than a meaningful measure.

The output is a structured recommendation with six required top-level concepts: `context_summary`, `kpi_chart_mapping`, `layout`, `styling`, `interactions`, and `rationales`. The mapping field connects one or more KPIs with an analytical task, a chart type, alternatives, and an encoding. The layout field describes how the views should be arranged and prioritized. Styling covers typography, contrast, palette, labels, and related accessibility decisions. Interactions describe possible actions such as filtering, sorting, tooltips, or selection. Rationales explain why a mapping or design decision is related to the goal and the available data. The output is therefore broader than a single chart recommendation, even when a record contains only one KPI mapping.

This distinction is important for the choice of a construction source. A rendered dashboard or chart image is not sufficient for the target task because an image does not directly expose the source query, aggregation, data field roles, or constraint scope. Conversely, a source query and a chart label are not sufficient to provide

authoritative users, layout, styling, interactions, or rationales. The dataset was therefore designed as a layered object. Source evidence supports the analytical part of a record, while later stages add presentation-oriented fields under a separate lineage label. The final object can be consumed by software, but its schema is a project-specific contract rather than a universal definition of dashboard design.

The construction requirements followed from this task definition. First, every source-derived analytical choice had to remain traceable to the external corpus, its database schema, its query, or a deterministic transformation with a recorded rule. Second, the representation had to be stable enough that all four methods received comparable inputs and were evaluated against the same field names and controlled vocabularies. Third, unsupported or ambiguous source expressions had to fail closed. A transformed record should not silently lose a filter, a group, an aggregation, or a time grain, because such a change would create a different analytical task. Fourth, the data had to support a strict separation between trainable material, validation material, the held-out test, and the separate human-evaluation item list. Finally, the release had to be documented with manifests, hashes, validation reports, and field-level lineage so that the dataset could be inspected independently of the model results.

These requirements also explain why the dataset is not described simply as a collection of gold labels. The source-derived fields are evidence-backed, but some task labels are deterministic abstractions and some presentation fields were generated by an LLM. The generated material can provide useful supervision for learning the output format and for broadening task coverage, but it cannot be treated as expert annotation without independent evidence. This distinction is maintained throughout the chapter.

4.2 Source Corpus and Source Registration

4.2.1 Why nvBench was selected

The primary source was the original nvBench corpus. nvBench was introduced as a cross-domain natural-language-to-visualization (NL2VIS) benchmark containing 25,750 published natural-language/visualization pairs from 750 tables and 105 domains [Luo et al. \(2021a\)](#). Its construction connects natural-language visualization requests with natural-language-to-SQL resources, including Spider, so that a record can retain both linguistic intent and relational query evidence [Luo et al. \(2021b\)](#); [Yu et al. \(2018\)](#). This relationship was a close fit for the present dataset because the project needed more than a chart name. It needed a source query, fields, aggregation intent, grouping or filtering information, and a visualization encoding that could be inspected after transformation.

The original nvBench records provide several useful evidence layers. A natural-language query describes the analytical request. The associated database identifier links the record to a relational schema. SQL and VQL express the data operation and the intended visual form. The visualization object provides fields such as the chart label, x- and y-axis information, classification or series information, and a short description. Together, these elements make it possible to preserve the relation between an analytical goal, a data field, an aggregation, and a chart. The source also represents operations such as grouping, ordering, filtering, and temporal binning in a form that can be parsed or checked against the database.

nvBench was not used as complete dashboard gold. It is an NL2VIS benchmark, not an expert-curated corpus of multi-view dashboards. It does not provide authoritative personas for the intended users, complete dashboard layouts, styling decisions, interaction designs, or full design rationales for the target schema. These missing fields were not filled by pretending that they were present in the source. Instead, the project preserves the source-backed analytical content and adds the six presentation fields later under explicit LLM-generated lineage. This decision keeps the source contribution useful without overstating what nvBench can support.

Other resources were inspected during source selection, as discussed in Chapter 3. Table 4.1 records the candidates and the explicit criterion on which each was accepted or rejected. The decisive criteria were the modality of the primary task, the presence of a natural-language statement of intent, the presence of structured visualization information that survives transformation, the availability of the underlying data fields and analytical operations, and whether the source can be scaled and split without leakage.

Table 4.1: Candidate source corpora and the criterion that decided their use in this project.

Candidate	Primary task	Natural-language intent	Structured visualization spec	Data fields and operations	Decision and reason
nvBench Luo et al. (2021a,b)	Natural language to visualization	Yes	Yes (VQL, chart, axes, classification)	Yes (SQL over a known relational schema)	Selected as the primary source: the only inspected corpus that supplies intent, a structured specification, and the underlying query together
nvBench 2.0 Luo et al. (2025)	Natural language to visualization under ambiguity	Yes	Yes	Yes	Inspected and not used in the final lineage; its ambiguity annotations are relevant to future work but were not needed for the transformation used here
Quda Fu et al. (2020)	Analytic-task recognition from free-form queries	Yes	No chart specification	No relational schema	Rejected as a primary source: it labels the analytical task but does not carry the encoding or the data operation the target schema requires
ChartGPT Tian et al. (2025)	Chart generation from abstract utterances	Yes	Yes	Partly	Related system rather than a reusable corpus for this transformation
FigureQA Kahou et al. (2018)	Question answering over synthetic figure images	Question, not design intent	No	No	Rejected on modality: the primary artifact is a rendered image, which does not expose the query, aggregation, field roles or constraint scope
DVQA Kaffe et al. (2018)	Question answering over bar-chart images	Question, not design intent	No	No	Rejected on modality, as above
PlotQA Methani et al. (2020)	Question answering over scientific plots	Question, not design intent	No	Partly (underlying tables)	Rejected on modality and on task alignment: the target is answering about a given chart, not choosing one
ChartQA Masry et al. (2022)	Question answering over real-world charts	Question, not design intent	No	Partly	Rejected on modality and on task alignment, as above

The four chart question-answering corpora were excluded for the same structural reason rather than for a judgement about their quality. Their supervision maps *from* an existing chart to an answer, while this project needs supervision that maps *to* a chart specification from a description of the data and the goal. A rendered image does not expose the source query, the aggregation, the role of each field, or the scope of a constraint, so a record built from it cannot be validated against the operations that the target schema encodes. Inspecting a dataset during source selection does not make it a training source, and none of the rejected corpora contributed any record to `dashboard_v3` or `dashboard_v4`.

The selection was therefore task-specific rather than a general claim that nvBench is the best visualization dataset. Its value for this thesis is that it provides a traceable analytical starting point. The dataset can be transformed conservatively while keeping the original query and visualization evidence available for validation. This property was more important for the present research question than the availability of rendered images or a large set of human ratings for a different task.

4.2.2 Registration, license, version, and source counts

The source was registered before the transformation stage in `data/raw_external/nvbench/source_manifest.json`. The manifest records the repository URL as `https://github.com/TsinghuaDatabaseGroup/nvBench`, the downloaded reference as `main`, and the local archive as `nvBench-main.zip`. No upstream commit was pinned. The exact local archive is identified by the SHA-256 digest `2c95244aca93aaca689fc954f8ae228c6c17fd47c81e1d7b265c4191cb012e4c`. The extracted README contains the MIT license statement, and the source manifest records the local license status as confirmed on that basis. The license record documents the evidence found in the project; it is not a substitute for checking the conditions of reuse when the data is redistributed.

The source manifest records 7,247 top-level visualization objects and 25,762 natural-language query records. These values must be distinguished from the 25,750 published natural-language/visualization pairs reported in the nvBench paper [Luo et al. \(2021a\)](#). The difference does not by itself indicate a failed download. A top-level visualization object can contain more than one natural-language query, while the published paper reports the benchmark pair count. The local query count is the count used by the extraction process, whereas the published pair count describes the benchmark as presented in the scholarly source. The chapter therefore reports both counts with their units instead of silently normalizing them.

The local archive also records the number of files and total bytes in the source manifest. These values identify the downloaded package, but the archive digest is the more useful integrity identifier for the transformation because it binds the local source to a specific byte sequence. The use of the `main` branch remains a reproducibility limitation: a future download of the branch may contain different bytes even though the repository URL is unchanged. The archive hash makes the bytes used in this project identifiable, but it does not replace a pinned upstream commit.

The database files distributed with nvBench were treated as validation evidence rather than as a second annotation source. A rebuildable cache was created from the registered archive. Cache preparation checks the source archive digest, protects extraction against path traversal, and leaves the raw archive unchanged. Database access is read-only. It is used to inspect schemas, field types, key information, uniqueness, cardinality, and selected query results. This distinction matters because database profiles can help decide whether a field is a meaningful measure, but they do not create a new ground-truth label when the source query is ambiguous.

4.3 Target Schema and Evidence Classes

4.3.1 Canonical GoldItem contract

Each modelable example is represented as a `GoldItem`. At the top level, the object contains an `item_id`, a `brief`, a `recommendation`, and an optional split assignment. The brief contains the fields described in Section 4.1 and an extensible `extra` object for provenance, lineage, quality, generation, and repair metadata. The recommendation contains the six required concepts `context_summary`, `kpi_chart_mapping`, `layout`, `styling`, `interactions`, and `rationales`. A KPI-to-chart mapping carries a KPI name, an enumerated `task_type`, an enumerated `chart_type`, possible alternatives, and an encoding dictionary. A rationale contains a claim and a principle. This structure allows the evaluation code to inspect the recommendation at field level rather than treating it as one undifferentiated text string.

The controlled task vocabulary contains nine values: `trend`, `comparison`, `composition`, `distribution`, `correlation`, `ranking`, `deviation`, `part_to_whole`, and `flow`. The chart vocabulary in the schema contains 17 values: `line`, `bar`, `stacked_bar`, `grouped_bar`, `area`, `pie`, `donut`, `scatter`, `heatmap`, `histogram`, `box`, `kpi_card`, `table`, `gauge`, `sankey`, `treemap`, and `map`. The schema is intentionally broader than the original nvBench-derived subset. A permitted value is not evidence that a chart or task occurs in every release.

For example, the source-grounded v3 corpus contains five primary chart families, while the later generated augmentation uses a larger controlled vocabulary.

Validation was applied in layers. Strict JSONL reading first rejects malformed JSON and non-object rows. Pydantic models and the frozen JSON schema then check the declared field types and the required task and chart enumerations. A completeness check requires the required brief and recommendation fields to be present and non-empty. Semantic checks then inspect whether the selected fields, KPI mappings, encodings, constraints, and chart-specific requirements are mutually compatible. This order separates a parsing failure from a schema failure and from a semantic failure. A record may be valid JSON while still containing an unsupported chart value, an empty recommendation field, an unavailable column, or an invalid analytical mapping.

The contract permits additional metadata at defined extension points. This is useful for retaining raw source evidence, rule versions, quality scores, and generation metadata without changing the core recommendation interface. The extension fields do not weaken the validation boundary. Raw task and chart values are checked against the declared vocabularies, and the required recommendation concepts must still be present. The final dataset therefore has one stable interface for the four downstream systems while preserving the information needed to audit how each record was constructed.

4.3.2 Source-grounded, derived, and generated fields

The dataset uses three evidence classes. Source-grounded fields are values that can be traced to nvBench, its source query, its visualization object, or its database/query evidence. Deterministically derived fields are values produced by a documented project rule, such as the normalized chart vocabulary, the task inference, or the KPI-selection abstraction. LLM-generated fields are values produced or rewritten by a language model. The third class includes the six presentation fields in the enriched v3 training and validation records and the broader generated content in the v4 augmentation.

This classification is field-level rather than record-level. A single record may combine several classes. In the source-grounded v3 train and validation records, for example, the goals, KPIs, columns, constraints, provenance, and original chart/encoding evidence remain source-backed, while the task type and KPI selection are derived and the six presentation fields are LLM-generated. The v3 test retains source-grounded analytical lineage but was not sent through the enrichment workflow; its presentation fields remain the values stored in the held-out artifact. The v4 augmentation is different: its brief and analytical specification were generated by the project’s controlled generator, and its presentation fields were later repaired by an LLM. These generated records are not new nvBench observations.

The main field-level distinction is summarized in Table 4.2. The table describes provenance, not quality. A generated field can pass all repository checks and still not be human gold. Similarly, a source-backed field can preserve the original query while remaining subject to the limitations or ambiguity of the upstream corpus.

Table 4.2: Field-level provenance across the preserved v3 and generated v4/v4.1 record families.

Field or field group	Preserved v3 train/validation	Preserved v3 test	Added v4/v4.1 records	Evidence interpretation
Goals, KPIs, columns, constraints, and source provenance	Source-grounded	Source-grounded	LLM-generated within the v4 generator	Source evidence for v3; generated supervision for v4
Original chart and encoding evidence	Source-provided	Source-provided	Generated and deterministically validated	A v3 source value is not equivalent to a v4 generated value
Task type and KPI-selection abstraction	Deterministically derived	Deterministically derived	Deterministically validated from generated content	Rule-based abstraction, not an original nvBench label
<code>users</code> , <code>context_summary</code> , <code>layout</code> , <code>styling</code> , <code>interactions</code> , and <code>rationales</code>	LLM-generated enrichment	Held-out artifact values; not enriched in this workflow	LLM-generated and repaired in v4.1	Generated annotations, not expert gold
Item identifiers, split, and lineage metadata	Deterministically assigned or recorded	Deterministically assigned or recorded	Deterministically assigned or recorded	Release and audit metadata

This separation follows the general principle that dataset documentation should describe origin, transformation, intended use, and limitations rather than presenting all fields as if they had the same status [Gebru et al. \(2021\)](#); [Pushkarna et al. \(2022\)](#). It is also necessary for interpreting later model results. A high score on a generated training target cannot be read as performance against an independent expert reference, and a source-grounded test result cannot be generalized automatically to task families introduced only through the v4 generator.

4.4 Source-Faithful Transformation of nvBench

The source transformation was designed to keep the meaning of the original record visible. The implementation reads the raw visualization/query structure, resolves database fields, parses SQL and VQL where possible, and stores the original evidence inside the record’s provenance object. Project-specific normalizations are added alongside that evidence. This avoids replacing the source with a cleaned text-only representation in which it would be impossible to see which values were observed and which were inferred.

4.4.1 Stable record and group identifiers

The builder traverses `NVBench.json` in sorted-key order and emits one candidate per natural-language query. This choice treats the query as the modeling unit while retaining the relation to the top-level visualization object. A source record is assigned a stable `source_record_id`, such as `nvbench:{key}:query:{query_index}` or the corresponding axis-qualified form used by the source. A separate `source_group_id` identifies related records derived from the same base visualization or source family. The group identifier is derived from the base key after removing axis-name or sort-specific suffixes where appropriate.

The distinction between record and group is important because one source object can generate several natural-language query records. Two queries may use different wording while referring to the same underlying visualization or analytical configuration. If they were split independently, a model could see a close source counterpart during training and evaluation. Group identifiers are therefore established before selection and split assignment. The group-safe split is complemented by exact and near-duplicate checks, but group assignment is the primary protection against related source records crossing the evaluation boundary.

4.4.2 Column and type resolution

The transformer keeps physical source columns separate from derived aggregate expressions. It resolves fields from the relational schema and profiles relevant database columns in read-only mode. The profile can contain a field’s observed type, row-level null and distinct ratios, uniqueness, numerical ratio, range, sign information, and representative values. The transformation also records whether an axis type comes directly from database metadata, from an aggregate expression, or from a documented fallback. These details are retained because a field’s storage type is not always the same as its analytical role.

This distinction is especially important for identifiers. A field may be stored as an integer and still represent a customer, account, invoice, or other entity identifier. Such a field can be used as a categorical dimension or for counting entities, but summing or averaging it is usually not a meaningful KPI. The pipeline therefore records physical fields in `brief.columns` and keeps expressions such as `COUNT(field)` or `SUM(field)` as aggregate evidence. An aggregate expression is not inserted into the physical column list merely because it appears in the SQL projection. This prevents the normalized brief from teaching the model that every numerical-looking token is a valid measure.

4.4.3 KPI and aggregation extraction

KPI extraction begins with the aggregate expressions represented in the source query and visualization evidence. When one aggregate axis is unambiguous, it is retained as the primary KPI. When two aggregate axes are present, the y-side aggregate is used as the primary KPI while both aggregate expressions remain represented in the provenance and encoding. If the source does not contain an explicit aggregate, the selector searches for a numeric, non-identifier-like measure using the available axis and database evidence. The source expression and the project’s normalized KPI representation are stored separately so that a later reviewer can distinguish extraction from normalization.

COUNT is handled as an intent-sensitive case. A count of rows, entities, or a categorical field may be a meaningful measure when the query expresses entity-count intent. SUM, AVG, MIN, and MAX applied to a strong identifier are technically executable but can be analytically unsuitable. Such cases receive explicit reason codes, such as `identifier_as_measure`, `meaningless_identifier_aggregation`, or `invalid_identifier_aggregation`, and are not silently repaired into a different KPI. If the source y-field is retained for fidelity but no meaningful KPI can be established, the record can remain diagnostic or be rejected by a later quality gate.

The goal is not to impose one universal KPI policy on all visualization data. It is to prevent a source transformation from converting a questionable expression into an apparently reliable label. The pipeline thus preserves evidence of the original aggregation, records the project interpretation, and lets the quality layer decide whether the record is suitable as a positive dashboard-design example.

4.4.4 Filters, grouping, sorting, limits, and time grain

The parser extracts filters, SQL `GROUP BY` terms, sorting, limits, `HAVING` conditions, time functions, and VQL `BIN` expressions when these constructs can be represented without ambiguity. SQL grouping fields are retained as constraints or visual grouping evidence. A time function or a grouped datetime x-field is represented with a time-grain value when the source supports that interpretation. Sorting is stored with its field, expression, and direction. Limits and `HAVING` conditions remain separate so that a row limit is not confused with an aggregation condition.

Unsupported expressions fail closed. Unrepresentable `OR` conditions, subqueries, malformed binning, ambiguous nested aggregates, and multiple or otherwise unsupported sort expressions are rejected instead of being approximated. This policy is conservative by design. Silently dropping a filter changes the population being analyzed. Dropping a group field changes the meaning of a stacked or grouped chart. Moving a limit before an aggregation changes the scope of the result. Similarly, omitting a time grain can change a trend from a monthly view to an unordered collection of dates. A record is accepted only when the normalized representation preserves the relevant source constraint or clearly records the limitation.

Natural-language text is used for narrow consistency checks rather than as authority to rewrite the SQL or VQL. Simple keyword matching can create false positives: a word such as “total” may be an intensifier, and “number of” may not identify the intended aggregation. The implementation therefore uses source fields, query structure, and database evidence as the primary basis for field and constraint decisions. The natural-language query remains available as the source expression of the user goal, but it is not used to invent a physical column that is absent from the source.

4.4.5 Chart, encoding, and task mapping

Source chart labels are normalized through the versioned mapping in `src/config/data/nvbench_mapping.yaml`. The mapping converts Bar, Pie, Line, Scatter, Stacked Bar, Grouping Line, and Grouping Scatter into the project vocabulary. Grouping variants retain the base chart type and preserve the series or classification information in the encoding and provenance. Unsupported labels are rejected rather than silently converted to a plausible chart.

The analytical task type is derived separately from the chart and query evidence. The rule configuration maps bars primarily to comparison, pies to part-to-whole, lines to trend, scatter plots to correlation, and stacked bars to composition. Grouped variants retain the grouping evidence and may support comparison across series. Each inferred task is recorded as rule-derived with a rule version, confidence, and short evidence statement. It is not presented as an original nvBench annotation. This distinction follows visualization task taxonomies that separate the purpose of an analysis from the means used to perform it [Brehmer and Munzner \(2013\)](#).

The source chart and encoding are therefore not treated as sufficient proof that the record is a good dashboard-design positive. Source fidelity asks whether the transformed record still represents the source query. Chart suitability asks a separate question: whether the source-supported fields and result shape satisfy the project’s criteria for a usable dashboard-design example. This separation is consistent with work showing that graphical effectiveness depends on both the analytical task and the data distribution [Saket et al. \(2019\)](#); [Kim and Heer \(2018\)](#). Constraint-based visualization systems such as Draco provide a related methodological example by representing design knowledge as explicit constraints rather than relying on an unexplained chart label [Moritz et al. \(2019\)](#).

4.5 Iterative Validation and Quality Filtering

The construction rules were developed through progressive validation rather than fixed after the first successful JSON export. Early pilots showed that structural acceptance could coexist with missing grouping evidence, invalid or ambiguous aggregates, and chart/query conflicts. Later pilots added source-preservation checks, database profiling, identifier detection, chart-specific rules, and a quality tier. This sequence matters because it records how the project discovered and addressed failure modes before building the final 1,819-record source-grounded corpus.

4.5.1 What pilots v1–v3 revealed

Pilots v1 and v2 used the 25,762 local query-level candidates and selected 100 examples with a roughly balanced distribution across five normalized chart types. Their manifests reported zero technical rejections at

the initial parsing stage. That result was useful for checking the shape of the corpus, but it did not establish dashboard-design validity. The associated reports recorded unresolved grouping information, fallback axis typing, and near-duplicate relationships. Between v1 and v2, 19 common records changed, nine grouping fields were recovered, and the reported near-duplicate count decreased from 38 to 19. These changes affected the analytical interpretation of grouped records rather than only their formatting.

Pilot v3 introduced a stricter source-preservation representation. It accepted 20,986 candidates technically and rejected 4,776. The documented rejection categories were 3,944 records with unpreserved filters, 157 ambiguous nested aggregates, 260 categorical scatter axes, 236 aggregate-intent conflicts, 44 chart/query conflicts, 10 unpreserved sorts, and 125 missing group fields. The category counts sum to the reported 4,776 rejections. The selected 100-record pilot was created only after these checks, exact normalized-goal deduplication, source-group sampling, database caps, and near-duplicate checks.

The v2-to-v3 comparison shows why the additional validation was necessary. Aggregate expressions in the physical-column list fell from 86 to zero, valid stacked-bar group fields increased from zero to 20, and categorical-scatter warnings fell from eight to zero in the compared pilot records. Accepted filters increased from zero to 15, accepted sorts from zero to 43, accepted time grains from zero to 18, and grouped records from nine to 33. These changes indicate that the transformation began preserving source semantics that the first representation had either omitted or represented ambiguously. A valid JSON object was therefore treated as a necessary condition, not as the final quality criterion.

The pilots were methodological development stages rather than model experiments. Their counts describe source extraction and selection behaviour. They do not measure the performance of prompt-only generation, RAG, QLoRA, or the combined method. Keeping these roles separate avoids using construction diagnostics as if they were downstream experimental results. The failure-mode approach also follows the reasoning behind behavioural testing, where systematic checks are used to reveal errors that aggregate success rates can hide [Ribeiro et al. \(2020\)](#).

4.5.2 Source fidelity versus dashboard suitability

The quality pipeline separates three gates. Technical validity asks whether the record can be parsed, conforms to the `GoldItem` contract, contains the required fields, and uses valid controlled values. Source fidelity asks whether the normalized brief and recommendation preserve the analytical meaning of the original source query, including fields, aggregations, filters, grouping, sorting, limits, and temporal information. Dashboard-design suitability asks whether the source-faithful record is appropriate as a positive example for the specific recommendation task.

These gates answer different questions. A record can be technically valid but lose a source filter. It can preserve the source query but use an identifier as a KPI. It can contain a meaningful KPI and still fail a chart-specific requirement, for example by using an unordered categorical axis for a line chart or by using non-additive aggregation for a part-to-whole chart. The project does not correct such differences by silently changing the source. It records them as failures, warnings, or lower quality tiers so that the source record and its suitability remain distinguishable.

The distinction is also important for the interpretation of the final dataset. Tier A means that a record passed the project’s adopted suitability criteria. It does not mean that the record is universally optimal or that an expert has judged every design choice. The quality layer operationalizes the thesis task using explicit rules informed by visualization research, but it remains a project-specific admission procedure.

4.5.3 Identifier, KPI, and chart-suitability checks

Identifier detection combines database and field evidence. The implementation considers primary-key, foreign-key, and unique-index metadata, observed uniqueness and distinctness, numeric-looking entity-reference patterns, and name patterns such as `id`, `key`, `code`, or identity-qualified number fields. The configured strong-identifier conditions use a unique ratio of at least 0.98 and at least 20 distinct values, while weaker evidence can be marked ambiguous. The exact name patterns and thresholds are stored in `src/config/data/nvbench_quality_rules.yaml`; they are operational rules for this corpus, not definitions of identifiers for every database.

KPI checks use the identifier result together with the query aggregation and natural-language intent. A strong identifier used as a SUM, AVG, MIN, or MAX measure is not treated as a meaningful KPI. COUNT can remain valid when the query expresses a count of entities or rows. Aggregation conflicts are evaluated in an axis-aware way. A conflict between the encoded aggregate and the SQL or query intent blocks Tier A even when the database can execute the expression. This prevents the system from equating computational executability with analytical validity.

Chart suitability is checked with chart-specific conditions. Bar charts require meaningful measure evidence and are blocked when the principal measure is a strong identifier. Line charts require a meaningful measure

and an ordered x-axis, supported by temporal information or another source-backed ordering. Pie charts are limited to at most eight categories, disallow negative values and identifier-like categories, and use additive COUNT or SUM aggregation for Tier A. AVG, MIN, and MAX are retained as source evidence when present but are demoted under the `pie_non_additive_kpi` policy. Stacked bars require a bounded and unambiguous grouping field.

Scatter checks use the result profile in addition to base-table metadata. The configured minimum is ten distinct values for relevant axes, and the profile is capped at 1,000 rows. Identifier axes are rejected, and the two axes must provide independently numeric observations with sufficient variation. A large underlying table is not enough if the executed query produces only a few paired observations or collapses the variation through aggregation. These rules reflect the general need to consider task, data semantics, and encoding together [Cleveland and McGill \(1984\)](#); [Mackinlay \(1986\)](#); [Brehmer and Munzner \(2013\)](#), while the exact thresholds remain project-specific engineering decisions.

4.5.4 Tier A/B/C policy and pilots v4–v6

The quality layer assigns three tiers. Tier A is a high-confidence positive candidate that is technically valid, source-faithful, suitable under the project rules, free of mandatory failures, and above the configured quality threshold. Tier B is source-faithful or potentially useful material that is uncertain or unsuitable as a positive dashboard-design target but may remain useful for diagnostics. Tier C represents a severe contradiction or a transformation that cannot be relied upon. The tiers do not claim human annotation quality; they describe the project’s operational confidence in using a record for positive supervision.

The quality score has five components. Source fidelity contributes 30 points, KPI validity 20, chart suitability 25, constraint completeness 15, and database-profile support 10. Tier A requires a score of at least 90 and no mandatory failure. The score is therefore a weighted admission mechanism, not an interval scale with a universal interpretation. A score of 95 does not mean that one record is universally 5 points better than another, and a score of 90 does not certify expert-level dashboard quality.

Pilot v4 applied the first quality version to the 20,986-record technical pool. It reported 13,896 Tier A, 7,071 Tier B, and 19 Tier C records. The selected pilot contained 95 records because the desired scatter quota could not be reached under the Tier-A and deduplication constraints. No Tier-B records were inserted to make the chart counts appear balanced. Pilot v5 applied a revised rule set and reported 13,044 Tier A, 7,923 Tier B, and 19 Tier C records. Its requested balanced selection was also not possible because only ten high-confidence scatter examples survived the stricter gates.

Pilot v6 retained the quality-first policy and selected 100 records with 23 bar, 23 line, 22 pie, 22 stacked-bar, and 10 scatter examples. All selected records were Tier A, and the manifest records that no Tier-B fallback was used. The resulting imbalance was accepted because replacing scarce high-confidence scatter examples with lower-confidence records would have weakened the meaning of the positive dataset. This decision follows the principle that a controlled dataset should report a real coverage limitation instead of hiding it through artificial balance.

The later full quality-pool rebuild is a separate construction boundary. It contains 21,244 technically valid records under the final `nvbench_quality_v6` rules, with 12,147 Tier A, 9,064 Tier B, and 33 Tier C records. The earlier strict v3 report contains 20,986 technically accepted records and 4,776 rejected records. The repository does not provide a complete row-level reconciliation for the difference of 258 records. The final corpus therefore uses the later quality-pool rebuild as its source, while the 20,986-to-21,244 change is reported as an unreconciled rebuild boundary rather than as an invented sequence of individual additions and removals.

4.6 Selection, Deduplication, and Evaluation Isolation

4.6.1 Final source-grounded corpus

The final large-dataset selector consumes Tier-A records only. It does not promote Tier B or Tier C records simply to satisfy a target size. Eligibility requires a quality score of at least 90, no mandatory failure, a non-empty normalized goal, valid source-record identity, and survival of exact source-record, normalized-goal, fingerprint, group, database, leakage, and near-duplicate checks. The selector considers the available scatter records first because scatter is the scarce source chart family. The remaining allocation uses a deterministic target policy and redistributes deficits only when a chart pool is exhausted. These constraints define how many high-confidence source records can be retained; they do not relax the quality rules.

The resulting `dashboard_v3` source-grounded corpus contains 1,819 modeling records. Its chart distribution is 1,379 bar, 109 line, 242 pie, 13 scatter, and 76 stacked-bar records. The corresponding rule-derived task distribution is 1,379 comparison, 109 trend, 242 part-to-whole, 13 correlation, and 76 composition records. This is an imbalanced distribution, especially for scatter. It reflects the surviving Tier-A supply after source-group, database, deduplication, and leakage controls. The imbalance is retained as a documented property of

the evidence rather than corrected by inserting weaker examples.

The distribution has methodological consequences. Later model evaluation should report per-class or macro-averaged evidence in addition to overall values, because a large comparison/bar class can dominate an aggregate result. This recommendation is not a downstream result and does not assert that any model performs well or poorly on a particular class. It states how the class distribution should be considered when interpreting later results.

4.6.2 Duplicate and near-duplicate control

Deduplication operates at several levels. Exact item identifiers and source-record identifiers are checked first. The selector then compares source groups, normalized goals, brief fingerprints, exact record content, and near-duplicate similarity. A source group can contribute at most two selected records, and a second record is retained only when its wording is sufficiently different and its analytical semantic signature differs in fields such as KPI, aggregation, axis, grouping, filter, sorting, or time grain. This preserves limited within-source variation while reducing the risk that one source family dominates the corpus.

The near-duplicate gate uses character 3-gram Jaccard similarity with a threshold of 0.8. Candidate text at or above this threshold is rejected as a near duplicate under the project policy. The threshold is not presented as a universal definition of duplication. It is a reproducible operational criterion selected for this corpus. Its use is supported by the broader concern that duplicate training data can increase memorization and that overlap between training and evaluation material can lead to overly optimistic estimates [Lee et al. \(2022\)](#).

Exact row comparison alone would not be enough. Two records can have different identifiers and different serialized JSON while expressing nearly the same goal and constraint structure. Conversely, two records from the same source group can be distinct in wording but share enough analytical structure to create leakage if group membership is ignored. Combining exact checks, semantic signatures, source groups, and near-duplicate similarity makes the boundary more conservative while keeping the individual checks interpretable.

4.6.3 Group-disjoint splitting and held-out artifacts

The selected source-grounded records were split with seed 42 using the group-safe `nvbench_large_v2_split_v1_group_safe` procedure. Whole `source_group_id` values were assigned to one partition. The final counts are 1,281 training records, 264 validation records, and 274 held-out test records. The partitions contain 784, 167, and 167 unique source groups respectively. The observed record proportions are approximately 70.42%, 14.51%, and 15.06%, rather than exact nominal 70/15/15 proportions, because source-group integrity takes precedence over exact row fractions.

The 274-record test is in-domain and source-group-disjoint. It is not an external benchmark because it originates from the same registered nvBench corpus as the source-grounded training and validation records. It is nevertheless a meaningful held-out boundary for the project because its groups are not shared with the training or validation partitions and because it is preserved before the later generated augmentation. The test therefore measures generalization within the source domain and source construction process, not generalization to an unrelated external corpus.

A separate file contains 40 human-evaluation items selected from the held-out test. The file is an evaluation instrument, not a fourth modeling split. Its rows overlap with the test by design, remain outside training and validation, and were not used as context for enrichment, augmentation, or repair. The human-evaluation file should therefore not be added to the 3,819 modeling records as if it contained 40 new examples. The distinction between modeling records and evaluation items is maintained in the manifests and in the final split table in Section 4.10.

4.7 Constrained LLM Enrichment and the dashboard_v3 Freeze

4.7.1 Why enrichment was needed

The source-faithful transformation provides the analytical content required to connect a natural-language query with fields, aggregations, constraints, and a chart. It does not provide all of the presentation fields required by the thesis output contract. In particular, original nvBench does not authoritatively specify the intended user role, a structured context summary, a dashboard layout, styling and accessibility choices, interactions, or a full rationale for the design. Leaving these fields absent would prevent the record from matching the common recommendation interface. Filling them with generic templates and calling them source values would be worse because it would misrepresent the evidence.

The project therefore introduced a constrained LLM-enrichment stage for the source-grounded training and validation records. This stage is methodologically related to filtered model-generated instruction data, where a model proposes content and a separate process filters invalid or overly similar examples [Wang et al. \(2023\)](#). The project applies a stricter boundary than unconstrained instruction generation: the source-backed

analytical projection is immutable, and the model may write only six presentation-oriented fields. These fields are marked `llm_generated` and `not_gold=true` in the resulting lineage metadata.

4.7.2 Writable and immutable fields

The enrichment workflow can modify exactly `users`, `context_summary`, `layout`, `styling`, `interactions`, and `rationales`. These fields form the presentation layer of the recommendation. The model can describe the audience, summarize the analytical context, arrange the views, propose accessible styling, suggest interactions, and explain the design choices. It cannot change the analytical task by rewriting the goal or by replacing the source KPI with a more fluent alternative.

The immutable projection contains the item and source identifiers, source group, source record, original goal/query, KPI and aggregation evidence, physical columns and types, task and chart mapping, alternatives, encodings, filters, sorting, limits, grouping, time grain, source provenance, quality score and tier, and split membership. The implementation computes an immutable projection and a SHA-256 fingerprint before and after enrichment. A change to an immutable field raises an `immutable_source_field_changed` failure. This is a control against a common synthetic-data error: a generated explanation should not silently overwrite the facts that it is supposed to explain.

Test records and the 40 human-evaluation items were excluded from the enrichment input. The full run therefore enriched only the 1,545 train and validation records, consisting of 1,281 training and 264 validation records. The held-out analytical content and evaluation boundary were preserved independently of the generated presentation layer.

4.7.3 Model, automatic validation, and human pilot

The enrichment artifacts record the model as `deepseek-v4-flash-sovereign`, the generation temperature as 0.0, and the reasoning effort as requested or configured `xhigh`. The project reports the configuration that was supplied to the provider. It does not claim that the provider independently verified an internal reasoning level. This wording is important because a configuration value is evidence about the request, not proof of how a remote model internally reasoned.

The enrichment workflow used several checks. The payload had to satisfy the strict schema and contain all six required fields. The generated context could not introduce unsupported KPIs, chart types, business numbers, or fields. Interactions had to use approved types and refer to fields present in the record. Rationales had to mention the actual task, chart, and encoded fields without asserting an observed outcome that the source did not provide. The trusted application code, rather than the model, merged only the six permitted fields and recorded their lineage.

A ten-record technical sample passed with 10/10 accepted outputs. A 30-record human R1 pilot accepted 29 records and rejected one because the rationale disagreed with the task, chart, or encoding evidence. The full first pass processed 1,545 records and accepted 1,534 while rejecting 11. It reported a 100% schema-valid rate, zero immutable-field violations, 943 API calls, 617 cache hits, and 15 retries. These values describe the enrichment process and not a final human evaluation of dashboard quality.

The targeted retry and offline revalidation stage resolved ten of the eleven first-pass rejections without recalling accepted records. One remaining case required a targeted retry. The final result reconciled to all 1,545 expected train and validation records, with zero permanent rejections and zero recorded immutable-field violations. The pre-freeze audit also normalized fingerprint metadata for records where the metadata was absent; this was metadata normalization and did not change the source-backed or enriched values.

The 29/30 human result is a limited quality gate for the enrichment procedure. It exceeds the configured minimum of 27 accepted records, but it does not establish that the six fields are correct for every record or that they are expert gold. Human-evaluation guidance stresses the need to report the scope and limitations of human judgments separately from automatic checks [van der Lee et al. \(2019\)](#). The dataset therefore keeps the R1 result as evidence about a small pilot and retains the `llm_generated` and `not_gold` labels for the full enriched corpus.

4.7.4 Frozen v3 composition

The enriched source-grounded corpus was frozen as `dashboard_v3`. The frozen package contains 1,281 training records, 264 validation records, and 274 held-out test records. The 1,545 train and validation records contain the six LLM-generated presentation fields. The test remains outside the enrichment input and retains its source-grounded analytical lineage and held-out artifact values. The separate 40-item human-evaluation file is sampled from the test and is not trainable.

The v3 manifest records `nvbench_large_v2` as the source dataset version, `GoldItem` as the schema version, and `phase3-enrichment-v1` as the enrichment specification. It records source-grounded fields, deterministically derived fields, and LLM-generated enrichment fields separately. It also records the R1 pilot, validation

results, leakage checks, exact file-preservation checks, and the exclusion of raw API responses, secrets, provider caches, and temporary generations from the frozen package. This freeze created the stable predecessor that was later augmented in train and validation only.

4.8 Controlled dashboard_v4 Augmentation

4.8.1 Motivation and scope

The source-grounded v3 train and validation data provided a defensible analytical core but had narrow coverage for the full dashboard-design task. The 1,545 records covered five primary task/chart families: comparison/bar, part-to-whole/pie, trend/line, composition/stacked-bar, and correlation/scatter. The profile contained 316 records with explicit filters, 83 grouped records, and 275 records with temporal information. It contained no canonical multi-KPI records. This narrow coverage is a consequence of using an NL2VIS source for a dashboard-level task rather than evidence that the source corpus is representative of all dashboard work.

The purpose of v4 was to broaden the training and validation supervision to additional task families, chart types, grouped and temporal cases, filters, and multi-KPI layouts. The augmentation did not replace the source-grounded core and did not modify the held-out test. The project used a controlled generator with a versioned task/chart catalogue and deterministic sampling. This design allows the generated records to be useful for learning broader output behaviour while keeping their generated status visible.

The expansion is not interpreted as a correction of the natural distribution of real dashboards. It is an intervention on the training and validation material. The generated records can increase coverage of task families that are absent from the source-derived test, but they cannot turn the preserved test into a benchmark for those new families. That boundary is important for the interpretation of later experimental results.

4.8.2 Generation protocol and provenance

The generation run is recorded as `dashboard_v4-generation-v1`. It used `gpt-5.6-luna` in `codex_agent` mode, seed 42, batches of 40 records, and 50 batches. The generator read only the frozen v3 training and validation material as construction context. It did not read `test.jsonl` or the 40-item human-evaluation file while proposing candidates. No generated candidate could be assigned to the test or human-evaluation artifacts.

The accepted v4 records are marked `source=llm_generated`, carry generation-group metadata, and set `not_gold=true`. Unlike the preserved v3 analytical records, their goals, KPIs, columns, constraints, task/chart specification, and encodings are generated within the controlled scenario generator. These fields are then checked against the project’s controlled schema and generation rules, but validation does not transform them into source observations. The generated records are therefore best described as AI-generated supervision, not as additional nvBench examples.

The generator uses a catalogue of 20 operational domains and a set of controlled scenario features. It varies audiences, decision contexts, operating states, planning windows, time grains, themes, layout patterns, filters, grouping, sorting, limits, and the presence of a second KPI. The purpose of these controls is to create identifiable coverage variation rather than unconstrained text diversity. The catalogue and sampling code are versioned in the repository, while the accepted records preserve their generation index, scenario group, seed, and model metadata.

4.8.3 Acceptance, deduplication, and split restriction

The run attempted 2,915 candidates and accepted exactly 2,000. The remaining 915 candidates were rejected as near duplicates under the character 3-gram Jaccard threshold of 0.8. Candidate acceptance also required non-empty brief and recommendation fields, a sufficiently detailed constraint string, valid task/chart combinations, KPI references that resolved to the generated columns, task-appropriate encodings, valid filters and sorting, and complete layout, styling, interaction, and rationale fields. These gates are generation-specific checks; passing them does not establish human or expert gold status.

The 2,000 accepted records were assigned 1,651 to train and 349 to validation. No generated record was assigned to test. The generated records were compared against the preserved v3 train and validation material and against previously accepted generated records using identifiers, normalized goals, brief fingerprints, exact record hashes, scenario signatures, and near-duplicate similarity. This prevents the augmentation from simply repeating a source brief under a new identifier. The generator also applies a batch safety limit so that a target cannot be reached through uncontrolled rejection behaviour.

The accepted records broadened the structural coverage. The generation report records 1,215 records with explicit filters, 1,373 with grouping, 1,229 with temporal information, and 842 with multiple KPI mappings. At the record level, the generated task distribution is shown in Table 4.3. Each record is counted once using its primary mapping, so the counts sum to 2,000.

Table 4.3: Record-level distribution of primary task types in the 2,000 accepted generated records.

Primary task type	Generated records
correlation	300
comparison	300
trend	250
distribution	250
ranking	200
part_to_whole	200
composition	200
flow	150
deviation	150
Total	2,000

The generated primary chart distribution is shown in Table 4.4. It contains 14 chart types and also sums to 2,000 records. These counts are record-level primary chart counts, not mapping-level counts.

Table 4.4: Record-level distribution of primary chart types in the 2,000 accepted generated records.

Primary chart type	Generated records
table	351
bar	275
area	223
heatmap	163
scatter	137
line	134
box	127
histogram	123
grouped_bar	105
stacked_bar	93
donut	76
sankey	69
treemap	68
pie	56
Total	2,000

Some generated records contain two KPI mappings. The distribution report therefore also contains mapping-level counts that can exceed the record count. It reports 2,842 generated mapping instances, 3,635 train mapping instances, and 752 validation mapping instances. These values are not additional JSONL records. Keeping record-level and mapping-level units separate prevents a multi-KPI record from being counted twice in the dataset-size accounting while still allowing the project to describe the coverage of its individual mappings.

4.9 dashboard_v4_1 Semantic Repair

4.9.1 Why structural validity was insufficient

The first v4 generation pass passed its structural admission pipeline, but a presentation-layer audit found issues in all 2,000 generated records. The most common categories affected all 2,000 records: missing or unanchored context constraints, KPI/context mismatch, unsupported context fields, generic layouts, under-described layout blocks, missing styling semantics, interactions without a purpose, generic filler, and rationales that did not provide record-specific evidence. Generic user templates affected 1,819 records and generic palettes affected 1,906 records. Additional issues included sort/task mismatches in 779 records, unjustified status colors in 564 records, interactions referring to nonexistent columns in 250 records, and chart-principle mismatches in 876 records.

These findings show why a schema-valid object cannot be treated as semantically adequate. The object can have the right keys and valid enum values while its presentation fields remain generic or disconnected from the brief. In a dashboard setting, an interaction that points to an absent field is not useful merely because its JSON representation is valid. A rationale that mentions a chart principle without applying it to the record is also not evidence of a meaningful explanation. The audit therefore focused on whether each generated field was anchored to the record’s own brief and mapping.

4.9.2 Repair scope and protected fields

The repair workflow is versioned as `dashboard_v4_1-semantic-repair-v1`. It used `gpt-5.6-luna` in `codex_agent_context_aware` mode. Only the same six presentation fields were repairable: `users`,

`context_summary`, `layout`, `styling`, `interactions`, and `rationales`. The fields `goals`, `kpis`, `columns`, `constraints`, `task_type`, `chart_type`, and `encoding` were protected. In the generated lineage, these protected fields are generated or generated-and-validated content; protecting them means that the repair stage did not change them, not that they became source-grounded.

The repair report records 2,000 repaired records, zero records already valid before repair, and zero rejected or regenerated records. The `users` field changed in 1,819 records, while each of the other five presentation fields changed in all 2,000 records. The model-generated analytical specification was therefore kept fixed during presentation repair. This separation makes it possible to attribute the repair operation to the presentation layer rather than to an undocumented change in the task, KPI, chart, or encoding.

4.9.3 Post-repair validation and interpretation

The post-repair audit checks the fields against the record’s existing goal, KPI, columns, constraints, task, chart, and encoding. Context summaries must agree with the brief. Layout blocks must correspond to the KPI mappings and state a meaningful hierarchy or reading order. Styling must describe typography, contrast, accessibility, and a semantic color policy. Interactions must use approved types, have a purpose, and refer only to fields present in the record. Rationales must mention the actual task, chart, and KPI and must avoid unsupported outcome claims such as causal or observed-performance statements that are not present in the brief.

The frozen validation report records zero schema-invalid and zero semantic-invalid generated records after repair. The repair report and the manifest consequently describe the generated fields as semantically clean under the repository audit. This phrase has a narrow meaning. It means that the implemented predicates passed; it does not mean that independent visualization experts reviewed or certified all 2,000 generated recommendations. The generated records remain `not_gold=true`.

4.10 Final Dataset Composition and Mixed Lineage

4.10.1 Final split table

The operational package is named `dashboard_v4`, while the exact frozen manifest revision stored inside that package is `dashboard_v4_1`. The final modeling composition is shown in Table 4.5. The preserved v3 columns represent the source-grounded predecessor records, and the v4/v4.1 columns represent the added AI-generated train and validation records.

Table 4.5: Final split composition and lineage of the frozen modeling package.

Partition or artifact	Preserved nvBench-derived v3	AI-generated v4/v4.1	Total	Role
Train	1,281	1,651	2,932	Trainable
Validation	264	349	613	Validation/model selection only
Held-out test	274	0	274	Evaluation only
Modeling total	1,819	2,000	3,819	Modeling study
Human-evaluation item file	40 test-derived items	0	40	Separate evaluation instrument

The arithmetic is deliberate. The modeling total is $1,819 + 2,000 = 3,819$, and the final train and validation totals are $1,281 + 1,651 = 2,932$ and $264 + 349 = 613$. The 40 human-evaluation rows are sampled from the 274 held-out test records. They overlap with the test by design and must not be added as 40 new modeling records or treated as an independent fourth split.

4.10.2 Record-level and field-level lineage

The final dataset has mixed lineage by design. The 1,819 preserved records retain `source=nvbench` and the source query, database, visualization, and provenance information. The 1,545 train and validation records in this group contain six LLM-generated presentation fields from the v3 enrichment stage. Their goals, KPIs, columns, constraints, and analytical chart evidence remain source-grounded, while task inference and KPI selection remain deterministic derivations. The 274 test records remain in the source-grounded analytical lineage and were not processed by the enrichment run; their held-out recommendation fields were preserved as part of the evaluation artifact.

The 2,000 added records retain `source=llm_generated` and `not_gold=true`. Their goals, KPIs, columns, constraints, task/chart specification, and encodings were created by the controlled v4 generator and accepted by deterministic validation. Their six presentation fields were then changed or confirmed through the v4.1 semantic-repair workflow. The repair protected the generated analytical fields, but it did not change their evidence class. They remain AI-generated supervision rather than nvBench observations, human gold, or expert gold.

The field-level view is important for later analysis. A model can be evaluated on the same output schema across both record families, but the reference status of a field differs between them. The v3 analytical target has a source-backed origin, whereas the v4 analytical target is generated. The presentation fields are generated in the v3 train and validation records and generated-plus-repaired in the v4 records. A result that is strong on the generated portion should therefore be described as performance on the project's generated supervision, not as independent evidence of dashboard quality.

4.10.3 Consequences for interpretation

The preserved 274-record test measures in-domain, source-group-disjoint generalization to the five original source chart families. The test includes bar, pie, line, stacked-bar, and scatter records, with a strong concentration of bar/comparison examples and only two scatter records. It does not directly evaluate generated-only task families such as distribution, ranking, deviation, or flow, because no such records were added to the test. It also does not turn the generated v4 records into independent gold merely because they are held out from some training runs.

The broader v4 training and validation distribution can support experiments on additional task and chart families, but any conclusion about those families requires an evaluation set that contains them. The completed eight-brief human study provides complementary evidence for layout, styling, interaction usefulness, and rationale quality, but its small convenience sample does not validate all capabilities introduced by the augmentation. The preserved test therefore remains a controlled in-domain test of the source-grounded boundary.

4.11 Dataset Freeze, Versioning, and Reproducibility

The active configuration in `src/config/data/dashboard_v4.yaml` uses `dashboard_v4` as the operational dataset identity and points to `data/frozen/dashboard_v4/`. The manifest in that directory records `dashboard_v4_1` as the exact frozen revision, `dashboard_v4` as its parent package, and `dashboard_v4_1-semantic-repair-v1` as the repair version. This distinction prevents a path or configuration name from being confused with the precise materialized state of the data.

The frozen directory contains the train, validation, test, and human-evaluation artifacts together with the `GoldItem` schema, dataset card, manifest, hashes, validation report, duplicate report, leakage report, distribution report, semantic-audit reports, and repair report. The manifest records the v3 prefixes, generated counts, repair scope, protected fields, and the checks used to publish the package. The v4 freeze verified that the v3 train and validation prefixes were unchanged, that the test and human-evaluation files were byte-identical to their parent artifacts, and that generated records entered only train and validation.

The file-level hashes provide an additional reproducibility check. Direct SHA-256 verification performed on 2026-09-02 matched the hashes files for the v3 and v4 train, validation, test, and human-evaluation artifacts. The source archive is separately identified by its own digest in the nvBench source manifest. These hashes do not prove that a future model provider will produce the same output from the same prompt, but they do make the published bytes and the source package auditable after the fact.

The package is treated as write-once. A material change to a frozen JSONL, CSV, schema, manifest, or report should create a new version rather than silently modifying the existing release. The held-out artifacts are copied or verified byte-for-byte across the v3-to-v4 transition, and the generation and repair procedures operate

only on train and validation material. Raw API responses, credentials, caches, and temporary generations are excluded from the final package. This separation keeps reproducibility records useful without publishing sensitive or irrelevant execution material.

One metadata inconsistency is recorded in the project audit. The nested human-evaluation hash in the v4 manifest differs from the direct file-level hash and the value in the hashes files, while the actual v3 and v4 human-evaluation files are byte-identical. Because the disputed nested digest is not needed to explain the dataset composition, this chapter does not reproduce it as an authoritative value. The accompanying evidence audit records the conflict and gives priority to the directly recomputed file hash and the dedicated hashes files. Reporting the conflict explicitly is preferable to silently selecting one value.

The unpinned upstream `main` reference remains the main limitation of source reproducibility. The local archive digest identifies the exact bytes used in this project, but an independent researcher who downloads the branch at a later time may obtain a different archive. Reproduction should therefore use the registered archive or an independently verified copy of the same bytes whenever possible.

4.12 Limitations and Threats to Validity

4.12.1 Construct validity

The source-backed part of the dataset is derived from nvBench, which was designed for NL2VIS rather than full multi-view dashboard design [Luo et al. \(2021a,b\)](#). It provides useful evidence about natural-language analytical goals, relational data, aggregations, and chart specifications, but it does not provide authoritative personas, complete dashboard layouts, styling, interaction design, or full rationales. The six presentation fields in the v3 train and validation records are therefore LLM-generated annotations. They are not source gold or expert gold. The 2,000 v4 records are generated supervision at both the analytical and presentation levels, followed by a model-assisted repair of the six presentation fields.

The task and chart rules are also project-specific abstractions. The distinction between task and encoding is supported by visualization research, but the exact quality weights, Tier-A threshold, pie cardinality limit, scatter variation requirements, near-duplicate threshold, and related policies are operational decisions for this project [Cleveland and McGill \(1984\)](#); [Mackinlay \(1986\)](#); [Brehmer and Munzner \(2013\)](#). Passing the rules means that a record satisfies the adopted construction protocol. It does not prove that the record expresses the only valid design or that the selected chart is optimal for every audience.

Source metadata and heuristic fallbacks can misclassify semantic field roles. A field name may look like an identifier without being one, while a domain-specific identifier may not follow the configured patterns. Database profiles improve the evidence but cannot resolve every domain interpretation. The quality tiers should therefore be read as bounded operational confidence rather than as a universal semantic truth.

4.12.2 Internal validity

The project’s earlier synthetic-only workflow created a circularity risk because the same deterministic task-to-chart logic influenced both training and testing. The present pipeline reduces this risk by constructing the held-out test from source-group-disjoint nvBench records and by excluding test and human-evaluation items from v3 enrichment, v4 generation, and v4.1 repair. This is a meaningful improvement in the evaluation boundary, but it is not complete independence: the source-grounded train, validation, and test records still originate from the same upstream corpus and share its construction assumptions.

The v4.1 semantic repair is model-assisted. The post-repair audit reports zero remaining schema and semantic failures under the repository rules, but it does not constitute independent review by visualization experts. The R1 enrichment review covers only 30 records and accepted 29 of them. This enrichment gate is separate from the completed human evaluation reported in Chapters 6 and 7; no preference or agreement result is inferred from the 29/30 pilot.

The validated hybrid staging branch is another internal boundary. It combined 18 synthetic training records and three synthetic validation records with enriched nvBench material and passed its own checks, but it was not used in the final v3 freeze. The repository does not contain a dedicated final-decision report explaining why these 21 synthetic records were omitted. The final manifests clearly identify the selected nvBench-only v3 inputs, so the final scope is known; the rationale for omitting the staging records remains incompletely documented.

4.12.3 External validity

The held-out test is in-domain and imbalanced. It contains only the five original source chart families and includes very few scatter cases. The expanded v4 training and validation records cover nine task families and 14 chart types, but the generated-only task families are absent from the preserved test. Test performance therefore cannot be interpreted as direct evidence that a model generalizes equally well to distribution, ranking,

deviation, flow, or the other newly introduced chart/task combinations. Such claims require a matching evaluation set, human assessment, or an external benchmark.

The source domain is also constrained by the schemas and domains represented in nvBench and by the NL2SQL resources used in its construction, including Spider. Real dashboard requests may contain multi-table business logic, organization-specific definitions, missing data, or visual conventions not represented in this corpus. The final dataset should therefore be used to study the controlled comparison defined in this thesis, not as a representative sample of all dashboard design work.

4.12.4 Reproducibility validity

The source branch was downloaded as `main` without a pinned upstream commit. The local archive hash makes the bytes used in the project identifiable, but it does not guarantee that an independent future download of the branch will match. The 20,986-to-21,244 quality-pool change also lacks complete row-level reconciliation. Both limitations are documented rather than hidden behind a single final count.

The generated fields are reproducible at the level of stored files, manifests, hashes, prompts or specifications, and rule versions, but the semantic content came from language-model calls. Temperature 0.0 does not guarantee identical outputs across provider revisions or service changes. The v4.1 repair is similarly reproducible as a published artifact and rule-checked result, but its text and layout-like values were produced by a model rather than independently annotated by experts.

Finally, the 40-row human-evaluation item file is a sampling frame rather than a result. The deployed study used eight briefs and stored its assignments and ratings under the separate human-evaluation result directory. The 29/30 value belongs only to the earlier enrichment gate. Keeping these artifacts distinct prevents construction checks from being misreported as user evidence.

Chapter 5

System Design and Methods

This chapter describes the system and the experimental methods used to study structured dashboard-design recommendations. The chapter begins by defining the research object and the comparison logic of the four methods. It then formalizes the input and output of the task, describes the model and seed matrix, and explains the shared prompt and generation pipeline. The following sections present the prompt-only baseline, retrieval-augmented generation (RAG), QLoRA fine-tuning, and the combined fine-tuning-plus-RAG method. The chapter also discusses alternative parameter-efficient fine-tuning methods and explains why QLoRA is the main adaptation method in this thesis. It ends with the inference procedure, the evaluation protocol, and the measures used to preserve reproducibility and to limit over-interpretation of the results.

The design of this study separates decisions that are shared by all methods from decisions that define a particular method. The dashboard brief, the output schema, the test split, the generation settings, and the evaluation code are shared wherever the implementation allows it. Retrieval and parameter adaptation are introduced as separate interventions. This makes it possible to compare a prompt-only model with a model that receives design guidance, with a model that learns from the task data, and with a model that uses both sources of information. Chapter 4 described how the dataset was constructed and frozen. The present chapter describes how the frozen data and the implemented models are used; it does not present outcome values. Those values and their interpretation belong to the results and discussion chapters.

The implementation record and the repository configuration are the primary evidence for the project-specific details in this chapter. Scientific literature is used to justify the general method choices, including Transformer generation, low-rank adaptation, quantized fine-tuning, retrieval augmentation, structured outputs, and behavioral evaluation. Where the current implementation is incomplete or where a metric is only a diagnostic, this is stated explicitly. In particular, an internal synthetic chart label is not treated as independent evidence of visualization effectiveness, and a seed listed in a configuration file is not treated as proof that the corresponding run has already been completed.

5.1 Methodological Scope and Research Design

The system studied in this thesis maps a dashboard brief to a structured design recommendation. The brief describes the intended users, analytical goals, key performance indicators (KPIs), available data columns, and design constraints. The output is not a rendered dashboard and it is not a SQL query. It is an intermediate design object that can be inspected, validated, and used by a later rendering or application layer. The object contains a context summary, KPI-to-chart mappings, layout and styling decisions, interaction suggestions, and rationales. This scope is important because chart selection is only one part of a dashboard design. A method can select a plausible chart and still fail to provide a coherent layout, accessible styling, or a useful explanation.

The experimental design uses four method conditions. They differ in two binary factors: whether the prompt contains retrieved design guidance and whether the base model contains a task-specific adapter. Method A is the prompt-only reference condition. Method B adds retrieval but keeps the base model unchanged. Method C adds QLoRA adaptation but does not retrieve documents. Method D combines the adapter from Method C with the retrieval procedure from Method B. The names used by the repository are shown in Table 5.1.

Table 5.1: The four method conditions and the factors that distinguish them.

Method	Repository name	Retrieval context	Task-specific adapter	Main purpose
A	prompt_only	No	No	Reference condition for prompting alone
B	rag	Yes, TF-IDF over the frozen design-guideline corpus	No	Estimate the effect of retrieved design guidance
C	ft	No	Yes, QLoRA adapter trained on the task data	Estimate the effect of task-specific adaptation
D	ft_rag	Yes, TF-IDF over the same corpus	Yes, the compatible C adapter	Test the combination of adaptation and retrieval

The comparison is designed as a two-by-two intervention study for a fixed base model and seed. The B–A comparison changes retrieval while keeping the model parameters unchanged. The C–A comparison changes the parameters through QLoRA while keeping the prompt closed-book. The D–C comparison asks whether retrieval still changes the output after task adaptation. The D–B comparison asks whether adaptation changes the output when the same retrieval context is available. The direct D–A comparison is also useful as an overall system comparison, but it does not isolate one intervention because both retrieval and adaptation change at the same time. This distinction prevents a combined system from being described as evidence for one component alone.

The unit of analysis is one dashboard brief and one generated recommendation for a specified model, method, seed, and input variant. The same item identifier is used to join a prediction with its reference and with any robustness variant. A model run is therefore not treated as an independent observation when its predictions are later compared item by item with another method. Paired comparisons are used where the evaluation protocol supports them. This is appropriate because the methods receive the same test items, while the model or context condition changes.

The implementation follows a staged experimental design. The primary dataset is `dashboard_v4` with the frozen revision `dashboard_v4_1`, and the available seed values are 42, 43, and 44. The executed model matrix comprises three Qwen3 profiles of increasing capacity — Qwen3 1.7B, Qwen3 8B, and Qwen3.8 27B — together with OLMo-2-0425-1B-Instruct as a second model family at a scale comparable to the smallest Qwen3 profile. The execution plan uses seed 42 first and reserves repeated seeds for the two smaller profiles, where the stability of the method contrast can be examined within the available budget. A condition that is executed only with seed 42 is reported as a single-seed condition. It is not combined with a multi-seed mean and is not used as evidence of seed stability. The small Qwen2.5-0.5B model is a local smoke-test profile and is not part of the reported comparison. Chapter 6 records the executed coverage cell by cell, including the cells that are missing and the runs that were rejected.

This staged policy is a resource decision, not a claim that larger models are deterministic or that one seed is sufficient in general. Training and generation for several models, four methods, and several seeds require substantial memory and time. QLoRA reduces the cost of adaptation, but it does not remove the cost of model loading, generation, validation, and repeated evaluation. Reporting the seed coverage together with the result is therefore part of the method. It makes clear which conclusions describe a repeated small-model stability check and which conclusions describe a single executed run.

5.2 Formal Task and Output Contract

The input to every method is represented as a dashboard brief

$$x = (u, G, K, F, c),$$

where u denotes the intended users, G is a list of analytical goals, K is a list of KPIs, F is the set of available data columns and their types, and c denotes optional constraints. The project also stores an `item_id`, but this identifier is used for provenance and joining records rather than as semantic input to the recommendation. A column can be represented as a pair

$$F = \{(n_i, d_i)\}_{i=1}^p,$$

where n_i is a field name and d_i is its declared data type. The brief may also contain additional fields, but the shared prompt exposes the users, goals, KPIs, columns, and constraints as the common interface for all methods.

The output is a structured object

$$y = (s, m, l, t, i, r),$$

where s is `context_summary`, m is `kpi_chart_mapping`, l is layout, t is styling, i is interactions, and r is rationales. A mapping entry connects a KPI to an analytical task and a chart type. It also contains optional alternatives and an encoding object. The schema therefore separates the semantic task from the visual representation. This separation is consistent with the view of visualization design as a mapping between data, task, and graphical encoding [Mackinlay \(1986\)](#); [Munzner \(2014\)](#). It also makes it possible to evaluate a chart token without losing the context in which the token was recommended.

Table 5.2: Required top-level fields of the structured recommendation.

Output field	Type in the project contract	Role in the recommendation
context_summary	Object	Summarises the design situation and the main goal
kpi_chart_mapping	Array of objects	Connects each KPI to a task, chart type, alternatives, and encodings
layout	Object	Describes placement, grouping, order, or emphasis of dashboard elements
styling	Object	Describes visual styling and accessibility-related choices
interactions	Array	Describes possible filters, drill-downs, or other interactions
rationales	Array of objects	Gives a claim and the design principle supporting it

The project defines controlled vocabularies for the two most important categorical fields. The available task types are trend, comparison, composition, distribution, correlation, ranking, deviation, part_to_whole, and flow. The available chart types include line, bar, stacked_bar, grouped_bar, area, pie, donut, scatter, heatmap, histogram, box, kpi_card, table, gauge, sankey, treemap, and map. These values are a project interface and not a universal taxonomy of all visualization tasks or chart forms. Their purpose is to provide a stable vocabulary for the dataset, the prompt, the parser, and the metrics.

The recommendation is generated as a token sequence whose final text should be one JSON object. For a model with parameters θ , the autoregressive generation process can be written as

$$p_{\theta}(y \mid x) = \prod_{q=1}^T p_{\theta}(y_q \mid y_{<q}, x),$$

where y_q is the next output token and $y_{<q}$ is the generated prefix. This is the standard decoder formulation used by Transformer language models [Vaswani et al. \(2017\)](#). In the prompt-only and RAG conditions, the model parameters remain those of the selected base model. In the fine-tuned conditions, the base model is combined with the learned adapter described in Section 5.7.

For the supervised adaptation condition, the target is the serialized reference object. If $D_{\text{train}} = \{(x_j, y_j)\}_{j=1}^n$ is the frozen training set and $y_j = (y_{j,1}, \dots, y_{j,T_j})$ is the target token sequence, the training objective is

$$\mathcal{L}(\theta) = - \sum_{j=1}^n \sum_{q=1}^{T_j} \log p_{\theta}(y_{j,q} \mid y_{j,<q}, x_j).$$

The target is therefore a complete structured recommendation rather than only a chart label. The loss does not prove that every target design is the only correct design. It only defines how the selected model is adapted to reproduce the frozen reference representation. This distinction becomes important in the evaluation: exact agreement with the synthetic reference is an internal diagnostic, while design effectiveness requires independent evidence.

The schema is implemented in `src/core/schemas.py` with Pydantic models. The regular runtime models allow additional keys so that a slightly extended model output can be retained for inspection. The strict generation metric uses the separate strict response schema and validates the raw extracted JSON object before lenient parser repairs are applied. This creates two clearly separated purposes. The permissive model helps preserve information during analysis, while the strict metric measures whether the model produced the required contract without relying on a correction made after generation. JSON Schema provides a general formal language for describing JSON structure and validation constraints; the project uses a project-specific Pydantic contract and a generated schema artifact for the same practical purpose [JSON Schema \(2022\)](#).

5.3 Model Matrix and Experimental Factors

5.3.1 Primary model profiles

The study uses four base models. Three of them are Qwen3-family profiles that span an order of magnitude in capacity, which makes it possible to ask whether a method effect changes with model size inside one family [Yang et al. \(2025\)](#); [Qwen Team \(2026a\)](#). The fourth is an OLMo 2 instruction-tuned checkpoint of comparable scale to the smallest Qwen3 profile, which makes it possible to ask whether an effect observed in Qwen3 reappears in an independently developed family [OLMo Team et al. \(2025\)](#). Model identifiers, context

lengths, preferred data types, revisions, and access requirements are read from the model configuration files and from the run manifests rather than inferred from a result file. The repository also contains a Qwen2.5-0.5B smoke-test profile, which is used only to verify that the pipeline runs end to end and is not part of the reported comparison.

Table 5.3: Base-model profiles used in the study.

Model key	Hugging Face identifier	Nominal size Pinned revision	Sequence length in this study Runtime profile	Role
qwen3_1_7b	Qwen/Qwen3-1.7B	1.7B not pinned	4096 bfloat16 preferred; thinking disabled	Smallest scale; three-seed stability condition
qwen3_8b	Qwen/Qwen3-8B	8B not pinned	4096 bfloat16 preferred; thinking disabled	Intermediate scale
qwen3_8_27b	Qwen/Qwen3.8-27B	27B 1d4bf0f2ff60...	4096 bfloat16 preferred; thinking disabled	Largest scale
olmo2_1_49b	allenai/OLMo-2-0425-1B-Instruct	1.49B 48d788eca847...	4096 bfloat16 preferred	Cross-family check at the small scale
qwen2_5_0_5b	Qwen/Qwen2.5-0.5B-Instruct	0.5B not pinned	2048 float16 on GPU; float32 CPU fallback	Smoke test only

The `qwen3_8_27b` profile is used through its text-only causal-language-model path. Its published model card describes a hybrid attention stack and a native context window far larger than the 4,096-token window configured here [Qwen Team \(2026b\)](#); the project deliberately holds the effective sequence length constant across profiles so that the context budget is not an uncontrolled factor in the comparison.

The OLMo profile is not an arbitrary fourth model. It is selected because it provides a second model family whose construction is documented in unusual detail: the OLMo 2 release publishes training data, code, recipes and intermediate checkpoints alongside the weights, so the ways in which it differs from Qwen3 can be described rather than assumed [OLMo Team et al. \(2025\)](#). Its instruction-tuned variant is post-trained with supervised fine-tuning on an OLMo-specific variant of the Tülu 3 mixture, followed by direct preference optimisation and reinforcement learning with verifiable rewards [Allen Institute for AI \(2025\)](#); [Lambert et al. \(2025\)](#). Its parameter count, measured when the model is loaded for adapter training, is 1.50 billion against 1.74 billion for the smallest Qwen3 profile, so the cross-family comparison is made at a comparable scale rather than across a capacity gap.

Because architecture, tokenizer, pretraining corpus, post-training recipe and native context window all differ simultaneously between the two families, the OLMo profile supports a model-family comparison and nothing narrower. The published configurations record that the OLMo checkpoint uses multi-head attention with 16 query and 16 key-value heads while the small Qwen3 profile uses grouped-query attention with 16 query and 8 key-value heads; this is a documented difference and not an explanation of any observed behaviour. A result from this comparison can support a statement about whether a pattern generalises across families. It cannot isolate any single architectural component as the cause. The OLMo profile is compared conceptually with the smallest Qwen3 profile only, and is not treated as size-matched to the 8B or 27B profiles.

All models are used as decoder-only language models through the Hugging Face model interface, and the model-specific chat template is applied before generation. For the Qwen3 profiles the configuration disables the optional thinking mode, because the target is a compact JSON object and the comparison concerns the requested design output rather than unobserved reasoning traces. The same output contract is used for the OLMo profile, while its tokenizer and chat template remain model-specific.

5.3.2 Methods, seeds, and controlled factors

For each selected model and seed, the method factor is A, B, C, or D. The dataset identity, prompt contract, output parser, evaluation code, and generation settings are held fixed unless a method explicitly adds retrieval or an adapter. The main generation settings are `max_new_tokens=512`, `temperature=0.1`, `top_p=0.9`, `do_sample=true`, and `repetition_penalty=1.15`. The low temperature reduces unnecessary variation in structured output, but sampling remains enabled; the setting is therefore not deterministic generation.

The available seeds in the matrix are 42, 43, and 44. The staged execution uses 42 as the first seed. Repeated seeds are used to check whether the smaller Qwen3-1.7B condition gives stable conclusions across training and generation. Larger models can be run at seed 42 to extend the capacity comparison under the available budget. The resulting evidence is labelled accordingly. A mean across seeds is calculated only when all required seed runs for the stated comparison exist. Missing runs are not silently replaced by a value from another seed.

The seed is an experimental factor, not a row-level label that increases the number of independent dashboard briefs. For a fixed test item, the predictions from different seeds remain repeated observations of the same item. This is why seed variation is reported as spread or as a stability diagnostic, while method comparisons use the same items whenever possible. The project records the seed in every `GenerationResult` and in training metadata. Exact bitwise reproduction is not promised merely because the same integer is recorded: sampling, hardware kernels, and CUDA execution can introduce variation. The seed is therefore necessary provenance, but it is not by itself a guarantee of identical output.

5.4 Shared Prompt and Structured Generation Pipeline

5.4.1 Single prompt source

All four methods use the prompt builder in `src/core/prompts.py`. The system message defines the model as a dashboard design consultant and requires one valid JSON object without Markdown fences or commentary outside the object. The user message renders the current brief and explicitly names the six required top-level fields. It also specifies the types of the fields, the required mapping entries, the allowed task and chart vocabularies, and the rule that KPI and encoding names must come from the current brief. This explicit contract gives the prompt-only baseline the same structural information that the fine-tuned methods receive.

The prompt does not ask the model to invent data values or to generate a rendered image. It asks for a recommendation based on the users, goals, KPIs, columns, and constraints. The prompt also distinguishes the rationale from the design decision. A rationale must contain a claim and a principle, so a response can

state both what it recommends and why the recommendation fits the brief. This separation is useful for later grounding analysis, although a generated rationale is not automatically a faithful explanation.

The formatter in `src/data_pipeline/formatter.py` imports the same prompt builder for supervised training. It serializes the reference recommendation as indented JSON and appends the tokenizer end-of-sequence token when one is available. Using one prompt construction path for training and inference reduces an avoidable difference between the input seen during adaptation and the input seen during evaluation. The output still depends on model-specific tokenization and chat templates, so the implementation records the model profile and tokenizer provenance for each run.

5.4.2 Pipeline stages

The common execution path is shown in Table 5.4. A method-specific difference appears only at the retrieval and adapter stages. The separation between preparation, generation, parsing, and scoring is deliberate. It allows a prompt-length failure or parser failure to be recorded as a failure of the relevant stage instead of being hidden inside one aggregate score.

Table 5.4: Shared execution path; only stages 3 and the adapter load differ between methods.

Stage	Operation	Main artifact or check
1. Load	Read the resolved model, method, dataset, seed, and evaluation configuration	Configuration and dataset identity
2. Prepare	Build the system and user messages from the dashboard brief	Shared prompt text
3. Retrieve	For B and D, retrieve up to three positive-scoring guideline passages	Retrieved passage IDs, scores, and context
4. Fit	Count prompt tokens and reserve the output budget	Input-token budget and truncation flag
5. Generate	Apply the model chat template and generate up to 512 new tokens	Raw model text
6. Parse	Extract a JSON object, normalize recoverable aliases, and validate the result	Parsed object and parse error
7. Evaluate	Reparse consistently and compute the configured metrics	Per-item records and aggregate reports

For a model with maximum sequence length L and an output limit M , the available input budget is

$$B_{\text{input}} = L - M.$$

For the final model profiles, the nominal values are $4096 - 512 = 3584$ input tokens. For the smoke profile, the corresponding budget is $2048 - 512 = 1536$ tokens. The model wrapper checks this budget before generation and raises an error instead of silently dropping part of the brief. In the RAG conditions, the retrieved passages are fitted to the same budget. If the full passages do not fit, the implementation clips each passage to an equal token limit and records `rag_context_truncated=true`. If even the passage headings do not fit, the item is treated as a preparation failure.

The standard A–D configurations use sequential generation with batch size one. This choice makes the item-level execution order and failure attribution simple. The code supports larger batches only when batching is explicitly acknowledged in the configuration. A batched generation call cannot provide a separate model duration for every item, so the implementation amortizes the fused duration and records the corresponding semantics. Batch size one is therefore the default for the main methods.

5.4.3 Parsing and strict validation

The parser in `src/inference/postprocess.py` first searches for a JSON object. It can handle a fenced JSON object, a brace-delimited object surrounded by short prose, or a complete JSON response. It then applies limited normalization before Pydantic parsing. Examples include mapping comparison to comparison, mapping column chart to bar, and removing alternatives that are not valid chart tokens. Object-like fields returned as strings can be wrapped so that the content remains available for inspection.

These repairs are useful for diagnosing near-miss outputs, but they must not be confused with a strict success. The schema-compliance metric extracts the raw JSON object again and validates the raw values against the strict response contract. An output that becomes valid only after an alias repair is therefore not counted as strict raw-schema validity. The raw text, the parsed representation, and the parse error are stored together in `GenerationResult`. This permits later audits without asking the model to generate the item again.

The repository also contains an optional constrained decoder based on an output schema. Constrained decoding can reduce invalid structures, and structured-output benchmarks show that the behavior of constrained

generation depends on both the constraint method and the model [Geng et al. \(2025\)](#). The constrained decoder is not enabled in the authoritative A–D final configurations. It is treated as a separate engineering or ablation path so that an improvement from hard decoding is not incorrectly attributed to RAG or QLoRA.

5.5 Method A — Prompt-Only Baseline

Method A is the reference condition for the study. It loads the selected base model and tokenizer, applies the shared system and user messages, generates the JSON response under the common generation settings, and passes the raw text to the common parser. It does not load a task-specific adapter and it does not add a retrieval context. The method therefore measures how much structured dashboard recommendation the base instruction-tuned model can produce from the explicit prompt alone.

The baseline is not intended to represent an unprompted language model. It is a controlled prompt-only condition. The model receives the same output contract, field names, and allowed vocabularies as Methods B–D. Without this information, a comparison would mix the effect of the method with the effect of giving one condition a clearer task definition. The baseline also does not use the target recommendation at inference time. It has access to the brief only.

Method A provides two comparison points. First, it shows whether retrieval adds useful information beyond the prompt. Second, it shows whether task-specific adaptation improves the structured output beyond the capability of the base model. Because the model is still stochastic at temperature=0.1 with sampling enabled, repeated seeds or repeated executions must be labelled when they are used. A single successful JSON response does not establish general reliability.

5.6 Method B — Retrieval-Augmented Generation

Method B adds a non-parametric information source to the prompt-only condition. Retrieval-augmented generation combines the language model’s parametric knowledge with passages selected from an external or local knowledge source [Lewis et al. \(2020\)](#). In this thesis, the retrieved information consists of dashboard and visualization design guidance. The method tests whether an explicit, auditable design context helps the model produce recommendations that are better aligned with the project principles or that provide more supported rationales.

5.6.1 Frozen knowledge base

The authoritative knowledge base contains three Markdown documents: `accessibility_guidelines.md`, `chart_selection_guidelines.md`, and `dashboard_design_guidelines.md`. The builder splits the documents at Markdown headings, keeps a heading with its body text, removes chunks below the configured minimum word count, assigns deterministic identifiers, and writes the resulting records to `data/knowledge_base/chunks.jsonl`. The frozen manifest reports three source documents and 41 chunks. It records the source hashes, byte sizes, the chunk-file hash, and the builder settings. The knowledge-base version recorded in the repository manifest is 46b8575d98d37312; the version recorded by every executed retrieval run is 3d1409d8347fc165. The two identities describe the same content and differ only in line terminators, as Section 6.3.1 documents; the value recorded by the runs is authoritative for the reported results.

Heading-based chunking was selected because the corpus is small and the headings already provide meaningful topical boundaries. The resulting chunks can be inspected by a researcher and cited in a retrieval trace. A smaller chunk may improve lexical matching for a short rule, while a larger chunk may preserve the context needed to interpret that rule. The project accepts this trade-off and records the exact chunk file so that a later change in chunking is a new knowledge-base version rather than an invisible modification.

The knowledge base is project guidance, not an independent human gold set for chart effectiveness. Its role is to provide design principles to the model and to support a retrieval trace. It is not used to claim that a retrieved sentence proves a chart is effective. This distinction is necessary because a rule document can be incomplete, simplified, or written for the project rather than validated as a complete theory of visualization.

5.6.2 Query and retrieval algorithm

For each brief, the retriever builds a query from the users, the joined goals, and the joined KPIs. It intentionally omits the target recommendation and the reference chart mapping. This exclusion prevents the retrieval step from receiving information that would not be available at inference time. The current retriever is TF-IDF with English stop-word removal. It vectorizes the 41 chunks and the query, computes cosine similarity, orders the chunks by score, and returns at most the top three chunks with a positive score.

For a term w in document d , TF-IDF can be understood as weighting a term by its frequency in the document and reducing the weight of terms that occur in many documents. The retriever then compares the query vector q with a chunk vector v_d using

$$\cos(q, v_d) = \frac{q \cdot v_d}{\|q\| \|v_d\|}.$$

The use of sparse TF-IDF is a deliberate scope choice. Term weighting by inverse document frequency is a long-established retrieval principle: a term that occurs in few documents carries more information about which document is relevant than a term that occurs in many [Spärck Jones \(1972\)](#). For a corpus of 41 short, topically distinct guideline chunks, this simple weighting is sufficient to separate an accessibility rule from a chart-selection rule, and it has four properties that matter for an experiment rather than for a production system. It is deterministic, so the same brief retrieves the same passages in every run. Its scores can be inspected directly, so a retrieval trace can be explained. It runs on the CPU and adds no model to the experiment. And it is model-independent, so the same retrieval mechanism can be applied unchanged across two model families without introducing a component that was itself trained on one of them.

Two better-known alternatives were not used. BM25 refines the same sparse principle with document-length normalisation and term-frequency saturation [Robertson and Zaragoza \(2009\)](#); those refinements matter most for long and heterogeneous documents, which this corpus does not contain. Dense retrieval encodes queries and passages with a learned encoder and matches them in embedding space, which can retrieve a relevant passage that shares no vocabulary with the query [Karpukhin et al. \(2020\)](#); a dense profile using BGE embeddings exists in the repository as an optional configuration. Neither was used for the reported conditions, and neither is presented as an evaluated method. The thesis does not compare retrievers, and the claim made here is that a simple sparse retriever is defensible for this corpus and this experimental purpose — not that it is the best retriever for the task.

The parameter `top_k=3` limits the amount of external context. A larger value could provide more coverage, but it would also consume input tokens and increase the chance that unrelated rules distract the model. A smaller value could omit a relevant rule. The selected value is kept fixed across B and D so that the comparison between the two RAG conditions changes the adapter, not the retrieval depth.

5.6.3 Prompt integration and grounding records

When positive matches exist, the retrieved passages are appended to the system prompt between the delimiters — Relevant Design Guidelines — and — End of Guidelines —. Each passage includes its identifier, source, heading, text, and retrieval score. The user message remains the same as in Method A. If no passage has a positive score, the method falls back to the base system prompt rather than adding an empty context block.

The retrieved context is guidance and not a hard constraint. The model can ignore it, misunderstand it, or generate a recommendation that is not supported by it. The pipeline therefore stores the retrieved passages in every RAG GenerationResult. It also calculates a claim-based grounding diagnostic for the generated rationales. The default mode is a lexical-overlap proxy. An optional semantic mode uses a sentence-transformer encoder when explicitly enabled. The value is labelled with its mode because word overlap is not a faithfulness judge. It can show that a claim shares words with a passage, but it cannot prove that the passage entails the claim.

The RAG method also records whether the context had to be truncated to fit the input budget. This field is important for interpretation. A model that receives three full passages is not under the same context condition as a model that receives only short excerpts. The context-truncation flag allows the results chapter to report this difference instead of treating all RAG calls as identical.

5.7 Method C — QLoRA Fine-Tuning

Method C adapts the selected base model to the structured dashboard task with Quantized Low-Rank Adaptation (QLoRA). LoRA freezes the original model weights and learns a low-rank update for selected linear transformations [Hu et al. \(2022\)](#). QLoRA combines this idea with 4-bit quantization of the frozen base model and trains the adapter while the quantized base remains fixed [Detrmers et al. \(2023\)](#). In the project, Method C is the main task-adaptation condition and Method D reuses its compatible adapter.

5.7.1 Why QLoRA is the primary adaptation method

The first reason for choosing QLoRA is the resource profile of the study. The model matrix spans roughly 1.5B to 27B parameters. Full-parameter fine-tuning would require updating and storing the full parameter set and its optimizer state for every selected model and seed. QLoRA keeps the base weights frozen and limits the learned update to a small adapter while reducing the memory needed to load the base model. This makes a multi-condition master-thesis experiment more feasible. The reason is practical and methodological: a method that cannot be executed consistently across the planned profiles cannot provide a useful comparison.

The second reason is experimental control. The adapter is a separate artifact that can be stored with its base-model identity, seed, dataset version, training configuration hash, and training metadata. Method D can

then use the adapter produced by Method C for the same model, dataset, and seed. The difference between C and D is consequently the addition of retrieval at inference time, rather than a newly trained and potentially different adapter. This provenance relation is harder to maintain when every combined condition is fine-tuned independently.

The third reason is alignment with the research question. The thesis asks how prompting, retrieval, and task-specific adaptation affect structured dashboard recommendations. It does not aim to establish a universal ranking of all fine-tuning algorithms. Selecting one well-defined PEFT method for the primary matrix keeps the adaptation factor interpretable. Alternative methods are discussed in Section 5.9 and can be treated as separate ablations, but they are not mixed into the main A–D comparison.

These reasons do not imply that QLoRA is always better than other adaptation methods. QLoRA can introduce quantization error, depends on hardware and software support, and may behave differently across model families. The scientific claim is narrower: QLoRA is an appropriate and reproducible primary method for this resource-constrained structured-generation study, provided that its configuration, adapter provenance, and limitations are reported.

5.7.2 Training data and target formatting

The training examples are taken from the frozen `dashboard_v4_1` package described in Chapter 4. The package contains 2,932 training records, 613 validation records, and 274 test records. The 40-item human-evaluation selection is held apart from training and is not used as an additional training split. The test and human-evaluation records remain byte-identical to their parent records according to the manifest checks. The fact that the training package contains generated enrichment fields is carried into the interpretation of the fine-tuned model: adaptation teaches the model to reproduce the frozen structured representation, but it does not turn AI-generated enrichment into independent expert gold.

For each training record, the formatter creates the system message and user message from the brief and appends the full recommendation as pretty-printed JSON. The target includes the context summary, KPI-to-chart mapping, layout, styling, interactions, and rationales. The primary configuration uses full-sequence loss with `completion_only_loss=false`. This setting is retained to preserve compatibility with the existing C and D training path. A response-only prompt-completion mode exists in the formatter and trainer, but enabling it creates a separately identified training condition and is not silently substituted into the primary run.

Using the same prompt builder in training and inference is important for this task because small wording changes can alter the model’s expected output structure. It also means that the prompt-only and fine-tuned conditions see the same field names and allowed values. The difference between them is the learned adapter, not a hidden change in the task definition. Instruction tuning research shows the value of aligning language-model behavior with explicit instructions; the present training procedure applies that principle to a project-specific structured output contract [Wei et al. \(2022\)](#).

5.7.3 QLoRA configuration

The authoritative training configuration is `src/config/training/qlora_default.yaml`. Table 5.5 records the parameters that define the primary adaptation condition. The values are included because the rank, target modules, quantization mode, optimizer, and schedule can all affect the result. Reporting only the word “QLoRA” would not be sufficient to reproduce the experiment.

Table 5.5: Primary QLoRA configuration as requested by `src/config/training/qlora_default.yaml`.

Configuration group	Parameter	Primary value
Adapter	Rank r	16
Adapter	LoRA scaling α	32
Adapter	Dropout	0.05
Adapter	Bias	none
Adapter	Target modules	all-linear
Quantization	Weight loading	4-bit
Quantization	Quantization type	NF4
Quantization	Double quantization	Enabled
Quantization	Compute dtype request	float16
SFT	Epochs	3
SFT	Per-device train batch size	2
SFT	Per-device evaluation batch size	1
SFT	Gradient accumulation	4
SFT	Learning rate	2.0×10^{-4}
SFT	Scheduler	Cosine
SFT	Optimizer	<code>adamw_torch</code>
SFT	Warm-up ratio	0.1
SFT	Weight decay	0.01

Configuration group	Parameter	Primary value
SFT	Gradient checkpointing	Enabled
SFT	Validation	Every epoch
SFT	Best checkpoint criterion	Lowest eval_loss
SFT	Packing	Disabled

The adapter update can be written conceptually as

$$W' = W + \Delta W, \quad \Delta W = BA,$$

where W is a frozen base weight matrix, B and A are trainable low-rank matrices, and the rank r is much smaller than the dimensions of W . The actual implementation uses PEFT to attach the adapter to the supported linear layers. Quantization affects how the frozen base is loaded and computed; it does not mean that the adapter itself is treated as a substitute for the complete base model.

The YAML requests NF4 4-bit loading and double quantization, while the training precision helper resolves the effective hardware-safe precision from precision: auto. The effective precision and the selected device are written to training metadata. This distinction matters because a configuration request and the effective runtime setting are not always identical across different accelerators. A run is therefore interpreted from its saved metadata rather than from the YAML file alone.

The same principle applies to three further parameters that the executed runs resolved differently from the defaults in Table 5.5: the per-device batch size and gradient-accumulation count, the maximum training sequence length, and the numerical precision. All eight trained adapters share the same effective optimisation batch of eight sequences, but they reach it through different combinations depending on the memory of the assigned device, they trained at either 1,024 or 4,096 tokens, and two of them trained in fp16 because the assigned device does not support bfloat16. Section 6.4 records the effective value for every adapter.

One consequence of the shared rank deserves explicit statement, because it is easy to state incorrectly. A fixed LoRA rank does not produce a fixed number of trainable parameters. The number and width of the adapted linear projections depend on the architecture, so rank 16 with all-linear targeting yields a different adapter size for every profile in the matrix. The actual counts recorded by the trainer are reported in Section 6.4; the share of the model that is trainable falls as the base model grows.

5.7.4 Validation and adapter provenance

The trainer loads the validation file when the evaluation strategy is not no. For the primary configuration, evaluation and checkpoint saving occur at epoch boundaries, and the trainer records the best validation loss and best checkpoint. The validation split is used for training control and checkpoint selection; the held-out test split remains reserved for evaluation. This separation prevents a test score from being used to select the adapter.

The trainer also checks for non-finite training metrics and trainable weights before continuing or saving an adapter. On completion, it stores the adapter weights, tokenizer, configuration, and training_metadata.json. The metadata includes the base model identifier, model key and revision, model configuration hash, training configuration hash, dataset version, training-file hash, seed, effective precision, parameter counts, optimizer and schedule, training duration, and validation history.

Method D can reuse an adapter only when the adapter resolver and validator confirm the expected identity. The checks cover the base model, model key, model revision where available, model configuration hash, seed, dataset version, and training configuration. If a mismatch is detected, the run fails rather than silently attributing an adapter from another model or seed to Method D. This is a central provenance safeguard because the D–C comparison depends on the two methods sharing the same learned adapter.

5.8 Method D — QLoRA Fine-Tuning with Retrieval

Method D combines the task adaptation of Method C with the retrieval process of Method B. It does not train a second independent adapter for the RAG condition in the primary matrix. Instead, it resolves the adapter produced by the corresponding C run and loads it into the same base-model profile. It then builds the RAG query from the brief, retrieves up to three passages from the same frozen knowledge base, fits the passages to the input budget, and adds them to the system prompt before generation.

The dependency is defined in src/config/method/ft_rag.yaml and in the final matrix through adapter_from: C. The resolver maps the dependency to the same model, dataset, and seed. Thus, C seed 43, when executed, is the adapter source for D seed 43. The method would be invalid if D used a C adapter from another seed or from another model size, because the comparison would then contain an uncontrolled training difference.

Method D tests a specific interaction between the two interventions. Retrieval may supply design principles that are absent from the learned task representation, but it may also add irrelevant or conflicting text. Fine-tuning may make the model better at converting the brief into the required JSON structure, but it may also make the model less responsive to new context. The method is therefore not based on the assumption that combining two interventions must improve the result. Its purpose is to measure the combined behavior and to examine whether the retrieved rationales and chart decisions remain compatible with the adapted output style.

The interpretation of D requires special care. A difference between D and A reflects both the adapter and the retrieved context. A difference between D and C is the cleaner test of adding retrieval after adaptation. A difference between D and B is the cleaner test of adding adaptation in the presence of retrieval. The results chapter must keep these comparisons separate rather than presenting one combined score as proof that both components independently caused the change.

5.9 Alternative Fine-Tuning Methods and Selection Rationale

QLoRA is the primary method, but it is not the only possible way to adapt a language model. The repository contains or documents several alternatives. They are useful for understanding the design space and for possible ablations, but they are not included in the primary A–D matrix. Their status and the reason for excluding them from the main comparison are given in Table 5.6.

Table 5.6: Alternative adaptation methods and why each is not a primary A–D condition.

Method	Scientific idea	Project status	Reason it is not a primary A–D condition
Plain LoRA	Learns low-rank updates while loading the base model without 4-bit quantization Hu et al. (2022)	Functional optional configuration in <code>lora_default.yaml</code>	Changes the quantization factor as well as the adaptation method and uses more memory
AdaLoRA	Allocates a parameter budget adaptively according to the estimated importance of updates Zhang et al. (2023)	Not part of the authoritative final matrix	Adds dynamic rank allocation and a second budget policy that would require a separate matched experiment
DoRA	Decomposes weight magnitude and direction and applies a low-rank update to the directional component Liu et al. (2024)	Supported by the trainer and an optional <code>dora.yaml</code> configuration	Tests a different adapter parameterization and should be reported as an ablation, not pooled with QLoRA
RSLoRA	Uses rank-stabilized scaling to reduce sensitivity to the adapter rank Kalajdzievski (2023)	Supported by the trainer and an optional <code>rsloa.yaml</code> configuration	Uses a different rank and scaling rule, so it is not a like-for-like continuation of the primary adapter
GaLore	Projects full-parameter gradients into a low-rank subspace while retaining full-parameter training Zhao et al. (2024)	Optional trainer and configuration	Produces a full-model training condition with different memory, optimizer, and artifact semantics
Full fine-tuning	Updates the complete model parameter set	Excluded from the final matrix	Exceeds the intended resource and provenance scope of the primary experiment

Plain LoRA is the closest alternative to QLoRA because it uses the same low-rank idea without quantizing the frozen base. It could answer whether quantization changes the adaptation result, but that is a separate question from whether adaptation helps dashboard recommendation. Including it in the main A–D matrix would add a new factor and make the interpretation less direct. It is therefore better treated as a matched quantization ablation with the same data, seed, prompt, evaluation, and training budget wherever such an ablation is actually executed.

AdaLoRA changes the allocation of the parameter budget during training. This can be useful when different layers or updates have different importance, but it introduces additional choices about the budget schedule and importance estimates. DoRA changes the parameterization by separating weight magnitude and direction, while RSLoRA changes the scaling behavior of the low-rank update. Both are scientifically relevant alternatives, and the project includes optional configurations for them. Their outputs should not be combined with QLoRA outputs as if they were repetitions of the same method.

GaLore follows a different resource strategy. It performs full-parameter learning while using a low-rank gradient projection to reduce optimizer memory. This can be valuable when full-model adaptation is a research goal, but it produces a full model rather than the same small adapter artifact used by C and D. The resulting

training cost, checkpoint behavior, and provenance checks are different. It is therefore outside the primary controlled comparison.

The selection of QLoRA is consequently based on scope, resource compatibility, and experimental control. It is not based on a claim that QLoRA dominates DoRA, AdaLoRA, RSLORA, GaLore, or plain LoRA for every task. A future algorithmic comparison would need to match the base model, data, prompt, seed, number of update steps, evaluation split, and output processing before drawing a conclusion about algorithm quality. The current thesis keeps those alternatives separate so that its main question remains answerable.

5.10 Inference and Output Processing

5.10.1 Loading and preparing a method

The inference runner reads the frozen test file and the selected method configuration. Method A and Method B load the base model. Method C loads the base model and its own adapter. Method D validates and loads the adapter produced by Method C before setting up the retriever. For C and D, the method configuration requests 4-bit inference loading with NF4 and double quantization. A and B use the base-model runtime profile. This difference is part of the actual implementation and is considered when latency and memory are interpreted.

For each item, the method constructs the system prompt and user prompt, retrieves context if required, and calls the model's prompt preparation function. The preparation function counts input tokens and returns the input budget. It does not silently truncate the dashboard brief. In a RAG condition, context fitting occurs before final tokenization, and the resulting system prompt is recorded through the retrieval fields and truncation flag.

The generation call uses the shared settings from the method YAML files: a maximum of 512 new tokens, temperature 0.1, top-p 0.9, sampling enabled, and a repetition penalty of 1.15. The maximum is a safety boundary for the structured response; it is not a statement that every response requires 512 tokens. If the model reaches the boundary, the raw output and any parse failure are retained so that truncation is visible in the error analysis.

5.10.2 Parsing and re-parsing

The raw response is passed to `parse_json_safe`. The function extracts a JSON object, normalizes a limited set of spelling and alias variants, and attempts to construct the runtime DesignOutput object. The `GenerationResult` stores the raw text, the parsed result or null, and a parse-error code such as `no_json_found` or `schema_error`. For RAG methods it also stores the retrieved documents and their scores.

Before evaluation, cached predictions can be reparsed with the current parser. This permits an improvement to the parser to be applied consistently to existing raw outputs. Strict schema metrics do not use the lenient parsed object as their sole evidence; they re-extract the raw JSON object and apply the strict validation path. The two paths therefore answer different questions: the lenient path asks whether a usable structured object can be recovered, while the strict path asks whether the model emitted a valid object without repair.

5.10.3 Failure handling and artifacts

The runner joins predictions and references by `item_id`, writes predictions as JSONL, records item-level errors, and writes automatic and layered metrics. A failed item is not silently removed from the denominator. If a run becomes incomplete, the run status and error artifact make that fact visible. The output layout includes the experiment identifier, model, method, seed, configuration hash, and relevant result files.

The cache identity includes a short SHA-256 prefix of the resolved configuration. This prevents a prediction generated under one method setting from being reused without notice after the setting changes. The hash is not a replacement for storing the full resolved configuration; it is an index used together with the configuration and run metadata.

Latency is measured by the method wrapper around the generation call after item preparation. Retrieval and prompt preparation occur before this timer in the shared implementation. Consequently, the recorded latency is primarily model-generation latency and should not be described as complete end-to-end user latency unless retrieval and preparation are measured separately. This boundary is important for a fair interpretation of the RAG condition.

5.11 Evaluation Protocol

The evaluation is layered because no single automatic metric can establish that a generated dashboard recommendation is useful. A valid JSON object is necessary for downstream use, but it is not sufficient for a good visualization. A chart token that matches a synthetic reference is also not sufficient evidence that users

would perform a task effectively with that chart. The protocol therefore separates technical format checks, internal structured diagnostics, robustness, retrieval and grounding diagnostics, independent chart-effectiveness evidence, realism, and human usefulness.

5.11.1 Technical and structured-output metrics

The implemented schema metrics report JSON extraction, required-key presence, strict raw-schema validity, encoding-object validity, completeness, and field coverage. Strict schema validity checks the raw extracted object against the strict contract. Empty objects, empty arrays, missing fields, invalid enum values, and invalid mapping structures are not treated as complete success. Parse failures count in the denominator, so a method cannot improve its schema rate by omitting difficult items.

The chart-type metrics compare the primary predicted chart with the primary chart in the frozen synthetic reference. Top-1 counts a parse failure or missing chart as incorrect, so it does not reward a method for omitting the items it finds difficult.

The synthetic chart and macro-F1 scores are labelled internal-circular in the reporting layer. The reason is that the chart labels and some structural targets originate from the project data-generation and transformation pipeline. Agreement with those labels is useful for checking whether the implementation is working and whether methods learn the frozen representation. It is not independent evidence of human-effective chart choice. This limitation follows the distinction between a formal output contract and the broader effectiveness of a visualization [Mackinlay \(1986\)](#); [Munzner \(2014\)](#).

5.11.2 Robustness and behavioral checks

The runner supports a paraphrased test variant and a missing-information variant. Paraphrase consistency measures whether the output remains stable after the wording changes while the intended task remains similar. Paraphrase accuracy compares the variant with the synthetic reference, but it inherits the internal-circular limitation of that reference. Consistency and correctness are therefore reported separately. A model can be consistently wrong.

The missing-information condition checks whether the model asks for clarification or produces a structured response when important information is absent. The current clarification measure uses a regular-expression heuristic over the generated text. It is an implementation diagnostic, not a complete measure of whether the model recognized the ambiguity. A high missing-information schema rate is not automatically good: a model that confidently produces a complete-looking recommendation from an under-specified brief may be less trustworthy than a model that requests the missing information.

These perturbation tests follow the broader principle that accuracy on one fixed test form is not enough to assess model behavior. Behavioral testing can reveal failures under controlled changes to the input [Ribeiro et al. \(2020\)](#). In the present study, the perturbations are limited to the variants implemented in the repository and should not be generalized to all possible paraphrases or incomplete briefs.

5.11.3 Retrieval and grounding

For RAG methods, the pipeline records the ranked chunk identifiers, scores, and text. Retrieval relevance metrics such as Recall@3, MRR@3, and nDCG@3 are available only when explicit human or expert qrels are configured. The default evaluation configuration does not contain such qrels, so the reporting layer marks retrieval relevance as unavailable rather than inferring relevance from lexical overlap.

The grounding metric checks the rationale claims against the retrieved passages. In semantic mode, a claim is considered supported when its maximum cosine similarity to a retrieved passage reaches the configured threshold. In the default lexical-proxy mode, support is based on content-word overlap. Both modes have limitations. Similarity is not entailment, and a retrieved passage can be relevant even when the generated explanation contains an unsupported conclusion. The mode, claim coverage, number of scored claims, and number of items with retrieved documents are therefore reported together.

The RAG metrics answer different questions. Retrieval relevance asks whether the selected chunks match an independent relevance judgement. Grounding asks whether generated rationale text resembles the retrieved context. Neither metric directly measures whether the final chart and layout are effective. The results chapter must keep these quantities separate from human usefulness.

5.11.4 Independent effectiveness and human evaluation

The evaluation protocol defines an independent chart-effectiveness layer based on human-effectiveness evidence from Saket et al. [Saket et al. \(2019\)](#) and Kim and Heer [Kim and Heer \(2018\)](#). The intended score is set-valued membership: a predicted primary chart is correct when it belongs to the human-effective set for the relevant task and data shape. Items without coverage in the independent table are excluded from that accuracy denominator and reported as uncovered. This is a stronger validity anchor than comparing only with the

project’s synthetic single-label rule, but the scorer is documented as designed and not yet implemented in the current evaluation code.

The usefulness layer uses human ratings on six dimensions: chart appropriateness, layout quality, styling and accessibility, interaction design, rationale quality, and overall usefulness. Each dimension uses a 1–5 rubric and permits “Cannot judge”, which is treated as missing. The deployed study presents anonymised Qwen3.8-27B seed-42 outputs for the four methods and stores assignments, submitted pages, and rubric responses separately from the automatic metrics.

Human evaluation should report the task, instructions, sampling, number of raters, aggregation rule, and agreement measure explicitly [van der Lee et al. \(2019\)](#). The final analysis treats the six dimensions separately, weights each of the eight briefs equally when summarising a method, reports per-dimension missingness, and calculates ordinal Krippendorff’s alpha. Because assignments form an incomplete block and the sample is small and shared across outputs, method contrasts are descriptive; an LLM-judge score is not substituted for human evidence.

5.11.5 Statistical comparison

The evaluation code and protocol define paired comparisons because the same items are evaluated under several methods. Binary outcomes such as strict schema validity or covered chart membership can be compared with an omnibus Cochran’s Q test and exact pairwise McNemar tests with Holm correction. Continuous or ordinal paired measures such as completeness and human rubric scores can be analysed with Friedman and Wilcoxon procedures where their assumptions and sample sizes permit. Bootstrap confidence intervals are reported for rates and paired differences when the required per-item vector is available.

These procedures are part of the analysis plan, not evidence that a statistical test has already been completed for every model and method. A confidence interval cannot repair a biased gold set, and a significant difference on an internal-circular metric cannot establish a better dashboard design. The statistical result must therefore be reported together with the evidence layer, the sample size, the seed coverage, and the status of the metric implementation.

5.12 Reproducibility and Provenance

Reproducibility is implemented through configuration, data hashes, model and adapter metadata, raw-output retention, and explicit run identities. The environment is represented by pyproject.toml and poetry.lock. The resolved configuration is written with the run, and a SHA-256 configuration hash is used in the cache identity and result metadata. The goal is not only to allow a later rerun; it is also to make it possible to determine which data, method, model, and settings produced a prediction.

5.12.1 Dataset and knowledge-base anchors

The dataset and knowledge-base identifiers used by the final methods are shown in Table 5.7. The full manifest contains additional source and report hashes. The exact values are included here for the files that directly define the test and retrieval inputs.

Table 5.7: Dataset and knowledge-base anchors for the executed runs.

Artifact	Version or file	SHA-256 or identifier
Dataset	dashboard_v4_1 manifest	291f57a50b0f7fa7c1c515031468d0b9458d9a11fb353f3cdf00c66168a3729e
Dataset test split	test.jsonl	e2df055d0a75c25f53a88cb830a5b6d66411fa179413f352f45ca2f6873829d5
Human-evaluation selection	human_eval_test_items_40.csv	2c336ee26398e8e487991df7e1c6e31385787c555228962d77e811f9a920cb8a
Dataset schema	schema.json	50e264be71f5d052f489e9a2753c4d878ec6b1ea17b8b38e3b19cf8cd0129183
Knowledge base, as executed	kb_version	3d1409d8347fc165
Knowledge-base chunks, as executed	chunks.jsonl	19fbbfa4ad333dec7298f53c98b25bf5a8bf5059236e8fa4885febede0e54159
Knowledge base, repository manifest	kb_version	46b8575d98d37312
Knowledge-base chunks, repository copy	chunks.jsonl	cad3bb0c1606fab3062d3235740d7701e1fb8ac46702af8b09f615f66bfc3afb

The dataset manifest identifies 2,932 training records, 613 validation records, 274 test records, and 40 human-evaluation items. It also records that the generated enrichment fields are not gold, that protected structural fields were kept unchanged, and that duplicate and leakage checks passed at freeze time. Chapter

4 provides the full transformation and provenance discussion. Repeating the anchors here connects the model methods to the exact frozen inputs without redefining the dataset construction chapter.

The knowledge-base manifest records the three source hashes and the deterministic builder settings. A change to a guideline document, a heading boundary, a minimum chunk size, or the chunk identifier function changes the knowledge-base artifact. Such a change must receive a new version and must not be described as the same RAG condition.

5.12.2 Model and adapter provenance

The model configuration records the Hugging Face identifier, nominal size, family, maximum sequence length, preferred dtype, chat-template options, token requirement, and model revision field. The current revision fields are null. This means that the identifier is known but an immutable upstream commit is not pinned in the final configuration. Exact model revisions should be recorded for the final published runs if the upstream repository supports it. Until then, model identity is less reproducible than the frozen dataset and knowledge base.

Every adapter stores its own base model identifier, model key, model revision field, model configuration hash, training configuration hash, dataset version, seed, training-file hash, and validation history. Method D validates these fields before loading the adapter. This prevents a technically loadable but scientifically incompatible adapter from being used in a final comparison.

The run manifest also records Python, PyTorch, CUDA, device, and package information for executed runs. The project uses a precision helper because the safe effective dtype can depend on the available hardware. The saved effective precision is therefore more authoritative than a generic statement such as “the model was trained in mixed precision.”

5.12.3 Reproducibility boundaries

The project deliberately keeps raw model text. This is necessary because parser versions can change, and because a parsed object alone cannot show whether a response contained extra prose, invalid enum values, or a truncated JSON object. The results can be reparsed from the raw text, while strict metrics can still use the raw extracted object.

Seed recording, configuration hashes, and fixed input files improve reproducibility but do not guarantee exact output equality. The inference entry point records the seed in the result object, but stochastic generation and hardware-specific operations can still differ. The training seed is passed to the trainer, while the project seed helper avoids a CUDA seeding call that caused a Windows DLL load-order problem. This implementation choice is documented because it affects the strength of any claim about bitwise reproducibility.

The final methods are therefore reproducible in the sense of documented inputs, code paths, configuration, and provenance checks, subject to the unpinned upstream model revisions and hardware-dependent numerical behavior. The results chapter must state these boundaries whenever it reports repeated runs or compares models from different families.

5.13 Methodological Limitations and Threats to Validity

5.13.1 Construct validity

The first limitation concerns what the automatic metrics actually measure. JSON parsing, required-key coverage, strict schema validity, and completeness are meaningful for a system that must produce a machine-readable recommendation. They do not measure whether a dashboard is visually clear, whether a user can complete a task quickly, or whether the layout supports a real decision. The structured output contract is an operational definition of the system interface, not a complete definition of dashboard quality.

The chart-type comparison has a related limitation. The frozen synthetic reference provides one target representation, but visualization tasks can admit several valid charts. The project therefore plans an independent set-valued effectiveness layer and a human usefulness layer. Until these layers are implemented and rated, automatic chart agreement must be described as an internal diagnostic or as format-level evidence. This boundary must be stated directly in the reported tables.

5.13.2 Dataset and supervision validity

The dataset is based mainly on nvBench-derived analytical examples and project transformations. Chapter 4 records which fields originate from the source and which enrichment fields were generated during the project. The generated fields are useful for constructing a complete training target, but they are not human expert annotations. Fine-tuning on them can teach the model the conventions and errors of the transformation pipeline. It cannot establish that the generated layout, styling, interaction, or rationale is the best design for a user.

The test split is protected from the later semantic repair and is byte-identical to the parent according to the frozen manifest. This protects the split from the enrichment operation, but it does not remove the broader source limitation. The coverage is still concentrated in the source task and domain distribution, and complex multi-KPI dashboards may be underrepresented. Results should therefore be generalized to this structured recommendation setting, not to all business dashboards.

5.13.3 Experimental confounds

Model-family comparisons include more than parameter count. The Qwen3 profiles and the OLMo 2 profile differ in tokenizer, pretraining data, post-training recipe, architecture, native context window, and model-specific chat template. A result difference between the two families cannot be attributed to size or to any one of these properties alone. The thesis treats model family as an experimental factor and avoids language such as “the larger model is better because it is larger” unless the evidence supports that narrower comparison.

Seed coverage is also not symmetric by design. Repeated seeds are concentrated on the small-model stability condition, while larger profiles may be run first with seed 42. This gives the study a feasible staged execution but limits claims about variance for larger models. A single-seed result can show the behavior of that executed condition; it cannot show the expected distribution across seeds.

The primary QLoRA configuration also introduces a quantization choice, a specific adapter rank, a target-module policy, a schedule, and a full-sequence loss. The result is evidence about this QLoRA configuration, not about every possible QLoRA setting. Similarly, the A–D comparison does not establish that QLoRA is the best adaptation algorithm. Alternative methods must be evaluated in a separately matched experiment.

5.13.4 Retrieval and grounding limitations

The RAG corpus contains only three project guideline documents and 41 chunks. It may not cover all visualization principles, domain terminology, or user constraints. TF-IDF is transparent and suitable for this small corpus, but lexical similarity can miss a useful paraphrase and can retrieve a passage because of shared words without understanding the design task. The fixed `top_k=3` policy also creates a trade-off between coverage and context length.

The default grounding score is a lexical proxy and the optional semantic score is a similarity test. Neither one establishes logical entailment or faithful causal use of the retrieved passage. Retrieval relevance is unavailable without independent qrels. A RAG result can therefore be discussed as showing retrieved-context behavior and a grounding diagnostic, but not as proof that the model followed a design rule.

Method D depends on Method C. If an adapter is damaged, incompatible, or trained with a different dataset, D cannot be interpreted as a clean combined condition. The adapter validator reduces this risk, but it does not prove that the adapter learned only the intended signal. It also means that failures in the C training pipeline can propagate to D.

5.13.5 Measurement and implementation limitations

The parser can repair some spelling and format errors. This is useful for a recoverable-output analysis but can make a lenient metric look better than the raw model output. The strict metric and raw-text retention reduce this risk, but the results must state which path was used. Similarly, the missing-information clarification rate is based on a regular-expression heuristic and should not be interpreted as a complete test of uncertainty recognition.

The recorded latency begins after item preparation in the shared method wrapper. RAG retrieval and context fitting are therefore not necessarily included in the same way as model generation. Memory use also differs between the base and 4-bit adapter conditions. These measurements are useful for the implemented pipeline, but they are not a complete deployment benchmark.

Two final configurations lack immutable upstream model revisions. The repository preserves the code, dataset, and resolved run configuration, but the corresponding upstream default branch can change. Reproducibility of those model binaries is therefore weaker than reproducibility of the hashed dataset and knowledge base.

5.13.6 External validity and current evidence status

The system produces textual recommendations for a structured brief and does not render a dashboard or measure real task performance. The human study therefore informs perceived quality of the recommendations, but not adoption, decision quality, time-on-task, or practical business value. Those outcomes require a rendered-dashboard or field study.

The evidence layers are therefore kept distinct: format and robustness checks are implemented; chart agreement uses a project-constructed reference; semantic grounding is a lexical proxy; independent human-effectiveness scoring is specified but not implemented; and the completed human ratings are exploratory

evidence about perceived recommendation quality. Chapter 7 reports these layers separately and labels incomplete seed, coverage, and model support rather than filling gaps with assumptions.

Chapter 6

Experimental Design and Evaluation

Chapter 5 described the output contract, four methods, training procedure, and provenance machinery. This chapter records the experiment as executed: model–method–seed coverage, effective hyperparameters and hardware, automatic metrics and their limits, perturbation conditions, statistical analysis, and the completed exploratory human evaluation.

Two conventions apply throughout the chapter and the two that follow. First, a value is reported only if it can be read from a generated artifact in the repository; configuration files are treated as a statement of intent and run manifests as a statement of fact, and where the two disagree the manifest wins. Second, a metric is always reported together with the evidence class it belongs to. Agreement with the frozen reference recommendation is agreement with the project’s own construction, not independent evidence about visualization effectiveness, and it is labelled as such every time it appears.

6.1 Executed experiment matrix

The experiment is a three-factor design: model (4 levels), method (4 levels: A, B, C, D), and seed (up to 3 levels: 42, 43, 44). A complete crossing would give 48 runs. Thirty-two runs were executed and passed the completeness check. Table 6.1 records the executed coverage.

Table 6.1: Executed run coverage. Each cell lists the seeds for which a complete run exists.

Model profile	Hugging Face identifier	Nominal size A (prompt-only)	B (RAG)	C (QLoRA)	D (QLoRA + RAG)
qwen3_1_7b	Qwen/Qwen3-1.7B	1.7B 42, 43, 44	42, 43, 44	42, 43, 44	42, 43, 44
qwen3_8b	Qwen/Qwen3-8B	8B 42	42	42	42
qwen3_8_27b	Qwen/Qwen3.8-27B	27B 42	42	42	42
olmo2_1_49b	allenai/OLMo-2-0425-1B-Ins truct	1.49B 42, 43, 44	42, 43, 44	42, 43, 44	42, 43, 44

A run counts as complete only if it satisfies six conditions, all of which were verified programmatically: `predictions.jsonl`, `predictions_paraphrased.jsonl` and `predictions_missing_info.jsonl` each contain exactly 274 records; every line is valid JSON; item identifiers are unique within each file; and an automatic metrics file exists. All thirty-two runs listed in Table 6.1 pass all six conditions with no file-level defects.

Every model has a complete A–D matrix at the primary seed. The four-way method contrast is therefore available for all four models, which is the coverage the research questions require. What the matrix does not have is three-seed replication at the two larger scales.

6.1.1 Missing cells

No runs exist for `qwen3_8b` seeds 43 and 44 or for `qwen3_8_27b` seeds 43 and 44. These were not executed. Repeating the full A–D matrix for the two larger profiles at three seeds would have required roughly three times the GPU time already spent on them, which the available allocation did not permit within the schedule of the thesis. The consequence is stated directly rather than argued away: the two smaller profiles support statements about seed-to-seed variability, and the two larger profiles do not. A single-seed run is one realisation of a stochastic process; it is reported as such and is never combined with a multi-seed mean or used to estimate a standard deviation.

One earlier defect is worth recording because it shaped the schedule. A first attempt at the prompt-only and RAG conditions for `qwen3_8b` produced files with duplicated item identifiers carrying different generated responses, because two processes were started against the same output paths and wrote into the same files. Metrics over such a file mix two executions of the same condition, so those exports were set aside in a separate `incomplete` directory and both conditions were inferred again from a single process. The runs reported in this thesis are the clean re-executions; the defective exports contribute nothing to any number. The detection mechanism — checking that item identifiers are unique within each prediction file — is now part of the completeness check quoted above and is applied to every run.

6.1.2 Priorities behind the allocation

The compute budget was spent according to a fixed order of priorities. The first priority was that every model that appears in the study should have a complete A–D matrix for at least the primary seed, so that the four-way method contrast is available for every model. The second was that the method contrast should be replicated across three seeds for at least one model, so that the stability of the contrast itself can be examined. The third was that the replication should be available in two different model families rather than only in Qwen3, so that a family-specific artifact can be distinguished from a general pattern. The fourth was that the scale sweep within Qwen3 should reach a model an order of magnitude larger than the smallest profile. Table 6.1 is the result of applying these priorities in order until the allocation was exhausted: the first three are satisfied in full, and the fourth is satisfied for absolute performance but not for seed variability. Across the thirty-two runs, roughly 162 hours of GPU wall-clock time were spent on inference and roughly 23 hours on the eight adapter trainings, giving about 186 GPU-hours in total for the reported experiment.

6.2 Models

Four base models were used. Two are small enough for three-seed replication, and two extend the capacity range within the Qwen3 family. Table 6.2 records the architectural facts that are relevant for interpreting the comparison; all values are read from the published model configurations.

Table 6.2: Base models. Attention notation is heads (query) / heads (key-value).

Profile	Identifier	Total parameters (measured at load)	Layers	Hidden size Attention	Vocabulary	Native context Family
qwen3_1_7b	Qwen/Qwen3-1.7B	1,738,007,552	28	2,048 16 / 8 (grouped-query)	151,936	40,960 Qwen3 Yang et al. (2025)
qwen3_8b	Qwen/Qwen3-8B	8,234,382,336	36	4,096 32 / 8 (grouped-query)	151,936	40,960 Qwen3 Yang et al. (2025)
qwen3_8_27b	Qwen/Qwen3.8-27B	27,012,726,272	64	5,120 hybrid: gated linear attention interleaved with gated full attention	248,320	262,144 Qwen3.8 Qwen Team (2026a,b)
olmo2_1_49b	allenai/OLMo-2-0 425-1B-Instruct	1,496,975,360	16	2,048 16 / 16 (multi-head)	100,352	4,096 OLMo 2 OLMo Team et al. (2025); Allen Institute for AI (2025)

The parameter counts in Table 6.2 are not taken from marketing names; they are the totals recorded by the trainer when the model was loaded for adapter training, which is why the OLMo profile is described as roughly 1.49B rather than 1B.

6.2.1 Why a second model family, and why this one

Three Qwen3 profiles alone would answer a question about capacity inside one family. They cannot distinguish a general property of the task from a property of how Qwen3 models are built and post-trained. A second family is therefore part of the design rather than an extra data point.

OLMo 2 was selected for four reasons that can be verified from primary sources. It is developed independently of Qwen by a different organisation, so its tokenizer, pretraining corpus, architecture and post-training recipe differ simultaneously from the Qwen3 profiles [OLMo Team et al. \(2025\)](#). Its development is unusually transparent: the training data, code, intermediate checkpoints and recipes are released alongside the weights, which makes the differences from Qwen3 documentable rather than a matter of speculation [OLMo Team et al. \(2025\)](#). The selected instruction-tuned checkpoint has a parameter count close to the smallest Qwen3 profile — 1.50B against 1.74B — so the cross-family comparison is made at a comparable scale rather than across a capacity gap. And the post-training pipeline of the checkpoint is documented: supervised fine-tuning on an OLMo-specific variant of the Tulu 3 mixture, followed by direct preference optimisation and reinforcement learning with verifiable rewards [Allen Institute for AI \(2025\)](#); [Lambert et al. \(2025\)](#).

The interpretive limit of this design must be stated at the same time. Because architecture, tokenizer, pretraining corpus and post-training recipe all change together between the families, a difference between the OLMo profile and the Qwen3 profiles is a *model-family* difference. It cannot be attributed to any single component. In particular, the fact that the OLMo checkpoint uses multi-head attention while the small Qwen3 profile uses grouped-query attention is a recorded architectural difference and not an explanation of any observed behaviour. The cross-family comparison can support a statement of the form “the pattern observed in Qwen3 does or does not reappear in an independently developed family at a comparable scale”. It cannot support a statement of the form “the pattern is caused by attention type”.

OLMo is compared conceptually with `qwen3_1_7b`. It is not size-matched to the 8B or 27B profiles, and no conclusion in this thesis treats it as such.

6.3 Data and inputs

All runs consume the frozen `dashboard_v4` package described in Chapter 4, whose exact frozen revision is `dashboard_v4_1`. Table 6.3 lists the input artifacts and the digests recorded by the runs.

Table 6.3: Input artifacts, with SHA-256 digests as recorded in the run manifests.

Artifact	Records	SHA-256
data/frozen/dashboard_v4/train.jsonl	2,932	c618488d92016e6c67925c2cbf021fb860a9e37ad6322107d535413f0bca84b1
data/frozen/dashboard_v4/val.jsonl	613	78847c3fe42a0a7221ead9b8970cfa17e51b4d9e79e103e4c4f15961e6462356
data/frozen/dashboard_v4/test.jsonl	274	e2df055d0a75c25f53a88cb830a5b6d66411fa179413f352f45ca2f6873829d5
data/eval/robustness_v4/test_phrased.jsonl	274	46de83228fa8aef24c0aca459502345b62ade634afa475ee6a472e5135cf4ac
data/eval/robustness_v4/test_misinfo.jsonl	274	f0c4e695d9c71f369247ca76bd707f2223d85880b59e3e5eb079d6fc3946f088
data/frozen/dashboard_v4/human_eval_test_items_40.csv	40	2c336ee26398e8e487991df7e1c6e31385787c555228962d77e811f9a920cb8a

Every one of the thirty-two runs records the same train, validation and test digests, so all reported comparisons are made on identical inputs. The train, validation and test digests were additionally recomputed from the local files during the preparation of this chapter and matched the values in the frozen `hashes.json`.

6.3.1 Knowledge base

Conditions B and D retrieve from a frozen corpus of three Markdown guideline documents — `accessibility_guidelines.md`, `chart_selection_guidelines.md` and `dashboard_design_guidelines.md` — split at Markdown headings into 41 chunks with a minimum chunk length of five words and deterministic chunk identifiers. All sixteen executed RAG runs record the same knowledge-base identity: version `3d1409d8347fc165`, chunk file digest `19fbbfa4ad333dec7298f53c98b25bf5a8bf5059236e8fa4885febede0e54159`.

The repository manifest for the same corpus records version `46b8575d98d37312` and chunk digest `cad3bb0c1606fab3062d3235740d7701e1fb8ac46702af8b09f615f66bfc3afb`. The two digests differ, and the cause

was determined rather than assumed: converting the line endings of the local chunk file from CRLF to LF reproduces the digest recorded by the runs exactly. The working copy on the Windows development machine carries CRLF line endings, while the copy transferred to the compute cluster carried LF. The retrieved *content* is therefore identical; only the byte encoding of the line terminator differs. The version recorded by the runs is the authoritative identity for the reported results, and the difference is noted here because a reader who recomputes the hash on a Windows checkout will obtain the other value.

A separate distinction applies to what the corpus is. The knowledge base is project-authored guidance whose statements are traceable to the visualization literature summarised in Chapter 2. A file digest establishes that the same bytes were retrieved in every run; it establishes nothing about the scientific authority of the guidance itself. Retrieval of a passage is therefore never treated as evidence that the recommendation it supports is correct.

6.3.2 Perturbation conditions

Two perturbation sets were derived from the held-out test split by a deterministic, dependency-free transformation with no language model, no network access and no randomness, and both preserve the original `item_id` so that every perturbed prediction pairs with its original.

The **paraphrase** condition rewrites only the free-text fields of the brief — **users**, **constraints**, and the string entries of **goals** and **kpis** — using a fixed table of meaning-preserving synonym substitutions applied to framing and verb words (for example *show* ↔ *display*, *track* ↔ *monitor*, *compare* → *contrast*, *goal* → *objective*, *metric* → *measure*). Domain nouns such as KPI expressions and column names are left untouched, so the analytical content of the brief is unchanged and a system that recommends a different primary chart has changed its answer for a reason that is not in the data.

The **missing-information** condition produces an under-specified brief by removing the constraints field entirely and dropping the last KPI and the last data column when more than one is present. Here the correct behaviour is not obvious by construction, and this is the point of the condition: a system may reasonably ask for the missing information instead of producing a confident complete recommendation.

Both conditions are narrow. They test one paraphrase family and one information-removal family. Results from them support statements about robustness *under these perturbations* and nothing broader. The general principle — that accuracy on one fixed test form is insufficient to characterise behaviour, and that controlled input changes reveal failures a single score hides — is the standard argument for behavioural testing of language systems [Ribeiro et al. \(2020\)](#).

6.4 Executed training configuration

Chapter 5 documents the training configuration file. The values that the runs actually used differ from that file in three places, and the manifests rather than the YAML are reported here. Table 6.4 records the effective configuration of each of the eight trained adapters.

Table 6.4: Effective QLoRA training configuration per adapter, read from `training_metadata.json`. All adapters use rank 16, scaling $\alpha = 32$, three epochs, learning rate 2×10^{-4} , a cosine schedule with 10 % warm-up, the `adamw_torch` optimiser, and 4-bit NF4 quantization of the frozen base with double quantization.

Model	Seeds	Training-config hash	Batch $\times$ accumulation (effective)	Max sequence length Precision	Trainable parameters	Share of total Training GPU	Hours per adapter
olmo2_1_49b	42, 43, 44	94bd7af87e2d	2×4 (8)	4,096 bf16	12,058,624	0.81 % H100 NVL	0.54
qwen3_1_7b	42	823472d2903a	1×8 (8)	1,024 bf16	17,432,576	1.00 % A30	1.00
qwen3_1_7b	43, 44	94bd7af87e2d	2×4 (8)	1,024 fp16	17,432,576	1.00 % Tesla T4	3.35
qwen3_8b	42	94bd7af87e2d	2×4 (8)	4,096 bf16	43,646,976	0.53 % GB10	4.87
qwen3_8_27b	42	8998747cdc16	1×8 (8)	1,024 bf16	116,727,808	0.43 % GB10	8.82

Four observations follow from this table and are carried into the interpretation of the results.

The intended common recipe held for the parameters that define the adaptation itself. Rank, scaling factor, dropout, target-module policy, learning rate, schedule, optimiser, epoch count and quantization scheme are identical across all eight adapters. The effective optimisation batch is also identical at eight sequences, although it is reached by different combinations of per-device batch size and gradient accumulation depending on the memory available on the assigned device.

The trainable parameter count is not constant. The same rank produces 12.1 M trainable parameters on the OLMo profile, 17.4 M on Qwen3-1.7B, 43.6 M on Qwen3-8B and 116.7 M on Qwen3.8-27B, because the number and width of the adapted linear projections depend on the architecture. The share of total parameters that is trainable consequently *falls* as the model grows, from 1.00 % to 0.43 %. A statement such as “the same adapter capacity was used for every model” would be false and is not made anywhere in this thesis.

The maximum training sequence length is not constant. Two profiles trained at 4,096 tokens and two at 1,024. A shorter training window truncates longer training targets, and the structured recommendation is a long target. This is a real difference between conditions that share a training-config hash label, and it is treated as a potential confound in Chapter 8 rather than as a detail.

The training precision is hardware-determined. The precision helper resolves `precision: auto` against the weakest available device, so the two Qwen3-1.7B adapters for seeds 43 and 44 were trained in fp16 on a Tesla T4, which does not support bfloat16, while every other adapter was trained in bf16. Seed 42 for the same model also used a different per-device batch size and a different code state. The three Qwen3-1.7B seeds are therefore *not* three draws from one identical procedure, and Chapter 7 reports them individually rather than as a pooled mean.

6.5 Inference configuration and execution environment

Every one of the thirty-two runs used identical generation settings, recorded in each manifest: at most 512 new tokens, temperature 0.1, top-p 0.9, sampling enabled, and a repetition penalty of 1.15. Generation is sequential with batch size one. Conditions C and D additionally load the base model in 4-bit NF4 with double quantization; conditions A and B use the base runtime profile. Conditions B and D retrieve at most three positively scoring chunks (`top_k = 3`).

The 512-token output limit deserves emphasis here because it becomes important in Chapter 7. It was chosen to bound key-value cache growth during 27B inference while leaving room for a structured response. It is a hard ceiling: a response that has not closed its outermost JSON object by token 512 is cut off, and the stored raw text ends mid-structure. The pipeline records the raw text in every case, so truncation is recoverable after the fact, but the metric that reads the parsed object cannot score a truncated response.

The runs were executed on heterogeneous hardware across several compute environments: NVIDIA GB10, H100 NVL, A30, L4 and Tesla T4 devices, with Python 3.11.7 or 3.12.x, PyTorch 2.6.0+cu124 or 2.13.0+cu130. This heterogeneity has one direct methodological consequence: **latency is not comparable across conditions that ran on different devices**. Section 6.6.10 restricts the interpretation of latency accordingly.

6.5.1 Code-state comparability

Each run manifest records the git commit of the code that produced it. These commits are not uniform across the matrix, and the differences are material enough that the analysis records them per comparison rather than assuming a single protocol. Three levels are distinguished:

- **homogeneous** — every run in the comparison recorded the same git commit, and the trained conditions share one training-config hash;
- **mixed code state** — the runs recorded more than one known commit;
- **unknown code state** — at least one run did not record its commit.

The label is computed at two granularities, because they answer different questions. The **block** label covers all four methods entering the omnibus test for one model and one seed. The **pair** label covers only the two runs actually being contrasted, and is the one that matters when reading an individual contrast: a block can be mixed because a third condition came from a different commit while the two runs in a given contrast did not.

Of the eight model-by-seed blocks, none is homogeneous: three are mixed and five contain at least one run whose commit was not recorded. At the level of individual contrasts the picture is better. Of the forty distinct contrasts, **seven are homogeneous**, fourteen are mixed and nineteen contain an unrecorded commit. The homogeneous seven are the B — A contrasts for `qwen3_8b` and `qwen3_8_27b` at seed 42 and for `olmo2_1_49b` at seed 42, and the D — C contrasts for `qwen3_8b`, `qwen3_8_27b` and `olmo2_1_49b` at seed 42 and for `olmo2_1_49b` at seed 43. Both contrasts that carry the main retrieval finding of this study — B — A at 8B and at 27B — are therefore code-state homogeneous.

Contrasts drawn from non-homogeneous pairs are reported as descriptive: the method changed, but so did something else that the manifests cannot rule out. This remains the largest threat to the internal validity of those contrasts, and it is a consequence of scheduling the runs across four compute environments over several weeks. It is discussed again in Chapter 8. Every contrast in Chapter 7 carries its pair-level label.

6.5.2 Adapter provenance in condition D

Condition D must load the adapter produced by condition C for the same model, dataset, training configuration and seed. Six of the eight D runs record the identifier of their source C run explicitly — for example `dashboard_v4_olmo2_1_49b_C_seed_43` for OLMo seed 43 — together with a digest of the adapter manifest. The remaining two D runs (`qwen3_1_7b` seed 42 and `qwen3_8_27b` seed 42) were produced by an earlier version of the runner that did not write the source-run field; for these the evidence is the recorded adapter path, which in both cases points at the condition-C adapter directory for the same model, method and seed. All eight D runs record a training-config hash identical to that of the corresponding C run. The pairing is therefore verified for all eight, by manifest field for six and by path plus configuration hash for two.

6.6 Metrics

The evaluation is layered because no single automatic number can establish that a dashboard recommendation is good. Each metric below is defined, motivated, and bounded. For every metric the chapter states whether it is *independent evidence* about the task or an *internal diagnostic* of the pipeline and its reference construction.

6.6.1 JSON parse rate

What it measures. The percentage of the 274 test responses from which a JSON object could be extracted from the raw model text. The extractor accepts a fenced JSON block, a brace-delimited object surrounded by short prose, or a bare object.

Why it matters. The whole point of a structured recommendation is that a downstream system can read it. A response that cannot be decoded at all is unusable regardless of the design it describes.

How to read it. A high value means the model reliably emits something machine-readable. It says nothing about whether the fields are correct, complete, or sensible. A parse failure has two very different causes — the model produced prose instead of JSON, or the model produced JSON that the token budget cut off — and Chapter 7 separates them.

Evidence class. Internal diagnostic of output form. Independent of the reference labels.

6.6.2 Runtime-contract parse rate

What it measures. The percentage of responses whose extracted object also validated into the runtime `DesignOutput` model after a limited set of alias repairs (for example `comparision` → `comparison`, `column chart` → `bar`).

Why it matters. This is the level at which the downstream metrics operate. If a response does not reach it, the pipeline has no typed object from which to read a chart type, and the item is scored as incorrect by the chart metric.

How to read it. The gap between the JSON parse rate and this rate is exactly the set of responses that were decodable but did not match the expected types. In this study the dominant cause is an `encoding` field emitted as a string where the contract requires an object.

Evidence class. Internal diagnostic of contract conformance. This is a property of the model output *relative to this project's contract*, not a general capability measure.

6.6.3 Strict schema validity

What it measures. The percentage of responses whose *raw* extracted object validates against the strict response contract, with no alias repair, no extra keys anywhere, non-empty strings for encoding channels, and an aggregate drawn from a closed set.

Why it was included. It was intended as the honest counterpart to the lenient parse rate: a measure of whether the model emitted a valid record without the pipeline fixing it afterwards.

Why it must be reported with a ceiling statement. During the preparation of this chapter the strict contract was applied to the frozen reference recommendations themselves. **None of the 274 reference records satisfies it:** the strict encoding model forbids extra keys, while every reference encoding carries the additional source-grounded keys `x_aggregate`, `y_aggregate`, `grouped`, `classify`, `group_field`, `filters`, `sort`, `limit`, `having`, `time_grain`, `visual_grouping`, `source_x` and `source_y`. A system that reproduces the reference format perfectly therefore scores 0 % on this metric by construction. The check is reproducible via `experiments/scripts/analyze_final_results.py`, which writes `strict_schema_ceiling.json`.

How to read it. Strict schema validity in this study measures *distance from a narrow contract*, and that contract is narrower than the training target. A high value indicates output closer to the minimal contract, which for fine-tuned systems means output further from the reference. It is reported for completeness and is explicitly **not** used as a quality ranking in Chapter 7.

Evidence class. Internal diagnostic with a known ceiling artifact.

6.6.4 Completeness and field coverage

What it measures. Completeness is the mean fraction of the six required top-level fields that are present *and non-empty*; an empty list, empty object or empty string does not count. Field coverage reports the presence rate of each of the six fields separately.

Why it matters. A record can decode successfully and still omit the layout, the interactions or the rationales. Completeness detects that failure mode without requiring type-level validity.

How to read it. High completeness means the response has the expected shape. It says nothing about whether the content of any field is appropriate. A complete recommendation can be a wrong recommendation.

Evidence class. Internal diagnostic of output form.

6.6.5 Chart agreement (top-1), reported at three levels

What it measures. Whether the primary recommended chart type matches the primary chart type in the frozen reference recommendation for the same item.

Why it matters. Chart selection is the single decision in the output that the source corpus can support with evidence, and it is the decision most directly connected to the visualization literature.

Why three levels. The metric as implemented in the pipeline reads the chart from the parsed runtime object. That couples two distinct failures: a wrong chart, and a record that never reached the runtime contract. Because the raw text of every response is stored, the same comparison can be repeated under weaker preconditions, and the difference between the levels isolates the cause. The three levels are:

1. **Pipeline top-1** — the pre-specified metric. Requires a valid runtime object. A parse failure counts as incorrect.
2. **Format-tolerant chart agreement** — reads `kpi_chart_mapping[0].chart_type` directly from the raw extracted JSON object, ignoring whether the record satisfies the runtime contract. Still requires a complete, decodable object.
3. **Text-level chart agreement** — reads the first the first `chart_type` field value occurrence in the raw text. Survives a response that the token budget cut off before the object could be closed.

The difference between levels 1 and 2 is the cost of the runtime contract; the difference between levels 2 and 3 is the cost of the generation-length budget. All three are computed by the same script and normalise the chart token identically. Levels 2 and 3 are **post-hoc diagnostics** introduced after inspecting the level-1 results, and they are labelled as such wherever they appear. Level 1 remains the headline metric.

How to read it. Agreement is agreement with the project’s frozen reference. Because a visualization task frequently admits more than one defensible chart, a mismatch is not automatically an error and a match is not automatically a good design [Saket et al. \(2019\)](#); [Kim and Heer \(2018\)](#); [Mackinlay \(1986\)](#).

Evidence class. Internal diagnostic relative to the project’s own reference construction. Not independent evidence about visualization effectiveness.

6.6.6 Macro-F1 over chart classes and the confusion matrix

What it measures. The unweighted mean of the per-class F1 scores over the chart-type vocabulary, plus the full predicted-versus-reference confusion matrix.

Why it matters. The held-out split is strongly imbalanced towards bar charts. A system that predicts `bar` for everything would score well on plain agreement. Macro-F1 weights every chart class equally and exposes that behaviour.

How to read it. A large gap between top-1 agreement and macro-F1 indicates that performance is concentrated in the frequent classes. Classes with very small support — the split contains only two scatter records — produce unstable per-class F1 values, so per-class numbers are read as descriptive.

Evidence class. Internal diagnostic, same reference as top-1.

6.6.7 Robustness under the two perturbations

Four quantities are computed by pairing each perturbed prediction with its original by `item_id`.

Paraphrase consistency is the percentage of items whose primary predicted chart is unchanged after the paraphrase. It measures stability of the answer, not its correctness — a system can be perfectly consistent and consistently wrong.

Paraphrase accuracy is chart agreement computed on the paraphrased briefs, over the items present in both runs. **Paraphrase accuracy delta** is the difference between paraphrased and original accuracy on the same items; a negative value means accuracy was lost under a change that should not have mattered.

Missing-information clarification rate is the percentage of under-specified responses that ask for clarification or signal uncertainty. It is computed by a regular-expression detector over the generated text, matching phrases such as *clarify*, *more information*, *ambiguous*, *insufficient*, *cannot determine*, *not enough*, *please specify* and *underspecified*. This is a lexical heuristic, not a semantic judge: it will miss an implicit request and can fire on an unrelated use of a matched word. It is reported as an implementation diagnostic.

Missing-information schema rate is the percentage that instead produce a complete valid record despite the missing information. It is reported for contrast, and a high value is not automatically a good outcome: confidently completing an under-specified brief may be less trustworthy than asking for what is missing.

All four apply only to the two implemented perturbation families and are never generalised beyond them.

6.6.8 Retrieval grounding

What it measures. For conditions B and D, the percentage of generated rationale claims that are supported by at least one retrieved passage. Support is decided in the default lexical-proxy mode by content-word overlap; an optional semantic mode using a sentence encoder exists but was not enabled for the reported runs. The metric is reported together with the number of scored claims, the claim coverage rate (the share of items that contributed any scorable claim), and the number of items that received retrieved documents.

How to read it. Lexical overlap is not entailment. A high value means the rationale text reuses vocabulary from the retrieved passage; it does not establish that the passage justifies the claim, and it does not establish that the chart the rationale defends is appropriate. Claim coverage must be read first: a grounding rate computed over a handful of claims from a handful of items describes those claims only.

Evidence class. Weak proxy diagnostic.

6.6.9 Retrieval relevance

Recall@3, MRR@3 and nDCG@3 require independent relevance judgements linking briefs to guideline chunks. No such judgements exist in the project, the evaluation configuration leaves the qrels path empty, and the reporting layer marks the metric unavailable rather than inferring relevance from the retriever's own scores. Retrieval relevance is therefore **not measured**.

6.6.10 Latency

What it measures. Wall-clock duration of the generation call for one item, averaged over the run, together with the median and the 95th percentile.

Interpretive limits. Two restrictions apply. The timer wraps the generation call after item preparation, so retrieval and prompt construction are not included in the same way as generation; the value is generation latency, not end-to-end user latency. More importantly, the runs were executed on five different GPU types, so a latency difference between two conditions that ran on different devices is partly a hardware difference. Latency is therefore reported as a descriptive cost indicator for the executed configuration and is not used to compare methods.

6.6.11 Metrics that are specified but not executed

Three layers of the protocol are defined and remain unmeasured. They are listed here so that their absence is visible in the setup rather than discovered in the results.

The **independent chart-effectiveness layer** would score a predicted chart as correct if it belongs to the set of charts shown to be effective for the relevant task and data shape in published human-subject studies Saket et al. (2019); Kim and Heer (2018), reporting covered accuracy and coverage rate separately so that uncovered items are visible. This would be a genuinely independent criterion rather than agreement with the project's own reference. The gold table exists in the repository; the scorer is not implemented, and every per-item record accordingly carries `l1_covered=\not_applicable`.

The **realism layer**, which would compare the distribution of generated dashboard structures against a corpus of real-world dashboards, requires a corpus that was not acquired. It is not measured.

The **human-rating layer** is reported in Section 6.8 and analysed descriptively in Chapter 7.

6.7 Statistical analysis plan

The design is paired: every method sees the same 274 held-out items for a given model and seed, and the per-item outcome records are stored, so comparisons are made item by item rather than between run-level averages. The tests were chosen from the structure of the outcome and the design, not from what happened to be available in the code.

Binary outcomes. Chart agreement, JSON extraction, runtime-contract validity, strict schema validity and encoding-object validity are dichotomous per item. For a block with three or more methods, Cochran’s Q provides the omnibus test for matched binary outcomes across k conditions [Cochran \(1950\)](#). Pairwise comparisons then use the exact McNemar test, which conditions on the discordant pairs — items where exactly one of the two methods succeeded — and evaluates them against a binomial null [McNemar \(1947\)](#). The exact binomial form is used rather than the chi-square approximation because several contrasts have few discordant pairs.

Multiple comparisons. Within each block of pairwise tests, p-values are adjusted with the Holm step-down procedure, which controls the family-wise error rate without the conservatism of a single Bonferroni threshold [Holm \(1979\)](#). A contrast is called significant when its Holm-adjusted p-value falls below 0.05. The correction family is one block: one model, one seed, one outcome.

Effect size and uncertainty. A p-value alone does not describe how large a difference is. Every contrast is therefore also reported as the mean paired difference in percentage points with a 95 % percentile bootstrap confidence interval obtained by resampling items — not runs — 10,000 times with pairing preserved [Efron \(1979\)](#). Per-condition rates are reported with their own bootstrap intervals. Statistical significance is never described as practical importance; a difference of one or two percentage points on 274 items can be significant and still be irrelevant for a design tool.

Ordinal and continuous outcomes. Completeness is analysed with the paired procedures supported by the item-level design. Friedman and Wilcoxon procedures were considered for the human rubric [Friedman \(1937\)](#); [Wilcoxon \(1945\)](#), but were not applied: the final convenience sample forms an incomplete block with shared raters and only eight briefs. Human method differences are therefore reported descriptively, with agreement and missingness, rather than as independent-observation significance tests.

Seeds. Seeds are repeated realisations of the same 274 items and are not additional observations. Where three seeds exist, the per-seed values are shown individually and summarised descriptively by mean and standard deviation. No confidence interval is computed across three seeds, and no variance estimate of any kind is computed for a single-seed condition. The reporting layer enforces this: it labels seed summaries with “not reported for fewer than 5 independent seeds” rather than emitting a spurious interval. That random seeds can produce non-trivial variation in fine-tuned language models — at the level of aggregate metrics and at the level of individual predictions — is documented in the literature and is a reason to report the spread rather than a single run [Bui et al. \(2025\)](#).

Comparability gating. Every reported test carries the code-state label from Section 6.5.1. A significant result from a `mixed_code_state` or `unknown_code_state` block establishes that the two runs differ; it does not establish that the method is the only reason they differ.

What the statistics cannot do. A confidence interval cannot repair a reference set, and a significant difference on an internal diagnostic cannot establish a better dashboard design. Every test in Chapter 7 is reported together with the evidence class of the outcome it was applied to.

6.8 Human evaluation

Automatic metrics in this study measure output form and agreement with a constructed reference. They cannot establish whether a layout is sensible, whether styling is accessible, whether an interaction is useful, or whether a rationale is convincing. These dimensions require human judgement, and reporting practice requires the task, instructions, sampling, assignment, aggregation, and agreement analysis to be explicit [van der Lee et al. \(2019\)](#).

The study is anchored to the manifest in `experiments/results/human_eval/dashboard_v4/qwen3_8_27b/seed_42_open_v1/` and the frozen eight-brief package in `seed_42_final_30min/`. Prediction digests identify 32 method-specific outputs from Qwen3.8-27B at seed 42. Each participant receives eight anonymised recommendation pages; method identity, the reference, and automatic scores are hidden. Recruitment is an open convenience sample, and assignments are made without viewing scores. The stopping target was three usable pages per output. It was an operational coverage rule rather than a power calculation.

The bilingual rubric `dashboard-plain-bilingual-v2` asks six separate questions on 1–5 scales: chart appropriateness, layout quality, styling and accessibility, interaction design, rationale quality, and overall usefulness. A “Cannot judge” response is stored as missing rather than as a neutral score. The 30-active-minute cap is a burden safeguard; submitted pages remain available, while unfinished pages are neither submitted nor imputed. The current rubric has not undergone an independent equivalence study and must not be pooled with the earlier fixed-rater rubric.

The analysis is exploratory and descriptive. It reports recruitment and completion, page and per-dimension coverage, score distributions, and method summaries. For each method and dimension, ratings are averaged within brief and then across the eight briefs so that unequal page counts do not give a brief additional weight. Ordinal Krippendorff’s alpha is calculated separately for each dimension as a description of agreement

Krippendorff (2018). A reliability coefficient is neither a significance test nor evidence that the rubric itself is valid.

The final non-test export contains 15 consented participant records, ten complete sessions, one additional submitted page from a partial session, and 81 submitted pages in total. Four registered participants submitted no rating. The 32 outputs received two to four pages each; thus the target of three pages was not met for every output. Across six dimensions, 11 of 486 possible judgements are missing. German and English account for 41 and 40 submitted pages respectively. These final counts, together with the input hashes and analysis outputs, are frozen with the study artifacts used in Chapter 7.

6.9 Reproducibility of the reported analysis

Every number in Chapter 7 is produced by code in the repository from artifacts in the repository. The aggregation of run-level metrics is produced by `experiments/scripts/aggregate_results.py` and the figures by `experiments/scripts/make_figures.py`. The paired statistics, the provenance table, the three-level chart re-scoring, the failure-mode counts and the strict-schema ceiling check are produced by `experiments/scripts/analyze_final_results.py`, which reads only `manifest.json`, `metrics_auto.json`, `eval_per_item.jsonl` and `predictions.jsonl` from the thirty-two completed runs plus the frozen test file, and writes its outputs to `experiments/results/final/dashboard_v4/statistics/`. None of these scripts requires a GPU, loads a model, or contacts a network service, so the entire analysis can be re-executed from the stored artifacts:

```
pythonexperiments/scripts/analyze_final_results.py--datasetdashboard_v4
```

The reproducibility of the analysis is not the same as the reproducibility of the runs. The runs depend on upstream model repositories, and the model revision field is recorded for two of the four profiles (`allenai/OLMo-2-0425-1B-Instruct` at revision `48d788eca847d4d7548f375ad03d3c9312f6139e` and `Qwen/Qwen3.8-27B` at revision `1d4bf0f2ff6012fd82039f2fa52739d0dd7c60c0`) and left null for the two other Qwen3 profiles. For those two, the identifier is known but no immutable upstream commit is pinned. Sampling is enabled at temperature 0.1, and hardware kernels differ across the five device types used, so recording a seed documents the intended configuration without guaranteeing bitwise identical regeneration. These boundaries are stated wherever a reproduction claim is made.

Chapter 7

Results

This chapter reports what the thirty-two completed runs produced. It follows the three research questions from the exposé: chart-reference agreement for RQ1; structural validity, completeness, and the unavailable human-rating layer for RQ2; and grounding plus perturbation robustness for RQ3. Model scale, model family, seed variability, failure modes, and computational cost are supporting analyses rather than separate research questions.

Every number in this chapter comes from a generated artifact. Run-level metrics come from the `metrics_auto.json` file of each run; per-item outcomes, paired statistics and the three-level chart re-scoring come from `experiments/results/final/dashboard_v4/statistics/`, produced by `experiments/scripts/analyze_final_results.py`. Three reading rules apply throughout and are not repeated at every table. Chart agreement is agreement with the project’s frozen reference recommendation and is an internal diagnostic, not independent evidence about visualization effectiveness. Seeds are repeated realisations of the same 274 held-out items and are shown individually rather than pooled. And every paired contrast carries the pair-level code-state label defined in Section 6.5.1; a significant result from a pair that is not code-state homogeneous shows that two runs differ, not that the method is the only reason they differ.

The chapter reports observations. Their interpretation, the failure analysis behind them, and the limits on what they license are the subject of Chapter 8.

7.1 Structured-output reliability (RQ2: structure)

Table 7.1 reports the structured-output quantities for every condition. The columns move from the weakest requirement to the strongest: whether a JSON object could be recovered at all, whether that object satisfied the runtime contract, whether every emitted mapping carried an object-valued encoding, and whether the raw object satisfied the strict response contract. Completeness is the mean fraction of the six required top-level fields that are present and non-empty.

The JSON-recovery column was recomputed from the stored raw text for all thirty-two runs so that runs written by different code vintages are comparable; the recomputed values match the `json_parse_rate` recorded in every one of the thirty-two `metrics_auto.json` files exactly.

Table 7.1: Structured-output reliability, in percent unless stated otherwise. $n = 274$ held-out items per cell.

Model	Method	Seed	JSON object recovered	Runtime contract satisfied	Encoding object valid	Strict schema valid	Completeness (0-1)
Qwen3-1.7B	A	42	99.64	0.36	0.36	0.36	0.996
Qwen3-1.7B	A	43	100.00	0.00	0.00	0.00	1.000
Qwen3-1.7B	A	44	100.00	0.36	0.36	0.36	1.000
Qwen3-1.7B	B	42	100.00	1.09	1.09	1.09	1.000
Qwen3-1.7B	B	43	99.64	1.46	1.46	1.46	0.996
Qwen3-1.7B	B	44	100.00	1.09	1.09	1.09	1.000
Qwen3-1.7B	C	42	97.45	1.46	1.46	1.46	0.974
Qwen3-1.7B	C	43	83.94	83.94	83.94	0.00	0.839
Qwen3-1.7B	C	44	87.96	87.96	87.96	0.00	0.880
Qwen3-1.7B	D	42	95.99	0.00	0.00	0.00	0.958
Qwen3-1.7B	D	43	98.54	98.54	98.54	0.00	0.985
Qwen3-1.7B	D	44	100.00	100.00	100.00	0.00	1.000
Qwen3-8B	A	42	100.00	100.00	100.00	0.00	1.000
Qwen3-8B	B	42	100.00	100.00	100.00	0.00	1.000
Qwen3-8B	C	42	86.13	86.13	86.13	0.00	0.861
Qwen3-8B	D	42	74.09	74.09	74.09	0.00	0.741
Qwen3.8-27B	A	42	100.00	100.00	100.00	46.72	1.000
Qwen3.8-27B	B	42	100.00	100.00	100.00	84.67	0.999
Qwen3.8-27B	C	42	100.00	100.00	100.00	0.00	1.000
Qwen3.8-27B	D	42	100.00	100.00	100.00	0.00	1.000
OLMo-2-1B-Instruct	A	42	30.29	26.28	12.04	0.00	0.248
OLMo-2-1B-Instruct	A	43	32.85	27.74	10.22	0.00	0.265
OLMo-2-1B-Instruct	A	44	32.12	27.01	12.04	0.00	0.257
OLMo-2-1B-Instruct	B	42	46.35	38.32	9.85	0.00	0.368
OLMo-2-1B-Instruct	B	43	49.64	42.70	9.85	0.00	0.391
OLMo-2-1B-Instruct	B	44	47.08	38.69	9.12	0.00	0.381
OLMo-2-1B-Instruct	C	42	2.92	2.92	2.92	0.00	0.029
OLMo-2-1B-Instruct	C	43	2.92	2.92	2.92	0.00	0.029
OLMo-2-1B-Instruct	C	44	2.92	2.92	2.92	0.00	0.029
OLMo-2-1B-Instruct	D	42	1.09	1.09	1.09	0.00	0.011
OLMo-2-1B-Instruct	D	43	2.55	2.55	2.55	0.00	0.026
OLMo-2-1B-Instruct	D	44	3.28	3.28	3.28	0.00	0.033

Five observations follow directly from Table 7.1.

The two largest models satisfy the output contract without adaptation. Qwen3-8B and Qwen3.8-27B both recover a JSON object and satisfy the runtime contract on 274 of 274 items in the prompt-only and RAG conditions. Neither needs fine-tuning to produce a machine-readable record.

Fine-tuning does not preserve that property at 8B. Qwen3-8B falls from 100 % contract satisfaction under A and B to 86.13 % under C and 74.09 % under D. The drop is significant on the paired test ($C - A = -13.87$ pp, Holm-adjusted $p = 2.9 \times 10^{-11}$; $D - C = -12.04$ pp, $p = 2.1 \times 10^{-8}$, and this second contrast comes from a code-state homogeneous pair). Section 7.5 shows that the lost items are truncated, not malformed.

Recovering a JSON object and satisfying the contract are very different things at 1.7B. In the prompt-only and RAG conditions, Qwen3-1.7B produces a recoverable JSON object on essentially every item (99.6–100 %) but satisfies the runtime contract on at most 1.5 %. The gap is a single recurring type error: the model emits `encoding` as a string — the name of one column — where the contract requires an object with named channels. The same failure appears in the seed-42 fine-tuned runs, which were produced by an earlier code and training state; the seed-43 and seed-44 adapters trained under the current state removed it, raising contract satisfaction from 1.5 % to 84–99 %.

Strict schema validity behaves inversely to reference fidelity, as predicted in Section 6.6.3. Every fine-tuned condition scores 0.00 %, as do both Qwen3-8B base-model conditions, while the two 27B base-model conditions score 46.72 % and 84.67 %. This is not a quality ordering. The strict contract forbids extra keys in the encoding object, and the frozen reference encodings carry thirteen additional source-grounded keys. Applying the strict contract to the 274 reference recommendations themselves yields **0 of 274 passing**, with the violations concentrated entirely on those extra keys. A model that has learned to reproduce the reference format therefore *must* score 0 %, and a model that emits a minimal encoding scores well. Strict schema validity is reported here for completeness and is excluded from every comparative statement in this thesis.

Retrieval helps output recoverability only where it is missing. For OLMo, retrieval raises JSON recovery by 15–17 pp across all three seeds. For Qwen3-8B and Qwen3.8-27B, where recovery is already complete, it changes nothing: every $B - A$ contrast on that outcome has zero discordant items.

Table 7.2 gives the paired tests for JSON recovery in the blocks where a difference exists.

Table 7.2: Paired contrasts on JSON recovery, restricted to contrasts with at least one discordant item. Difference in percentage points with a 95 % bootstrap interval; p is the Holm-corrected exact McNemar value within the block; b+c is the number of discordant items; the last column is the code state of the two runs being contrasted.

Model	Seed Contrast	Δ (pp) 95 % CI	b+c	p (Holm) Pair code state
OLMo-2-1B	42 B – A	+16.06 [+7.66, +24.45]	148	7.5×10^{-4} homogeneous
OLMo-2-1B	43 B – A	+16.79 [+8.76, +24.82]	128	1.2×10^{-4} unknown
OLMo-2-1B	44 B – A	+14.96 [+6.57, +22.99]	135	1.1×10^{-3} unknown
OLMo-2-1B	42 C – A	−27.37 [−33.21, −21.53]	87	2.1×10^{-17} mixed
OLMo-2-1B	43 C – A	−29.93 [−35.77, −24.09]	90	1.3×10^{-20} unknown
OLMo-2-1B	44 C – A	−29.20 [−35.04, −23.36]	90	3.0×10^{-19} unknown
Qwen3-1.7B	43 C – A	−16.06 [−20.44, −11.68]	44	6.8×10^{-13} unknown
Qwen3-1.7B	44 C – A	−12.04 [−16.06, −8.39]	33	1.4×10^{-9} unknown
Qwen3-1.7B	43 D – C	+14.60 [+10.22, +18.98]	44	4.5×10^{-10} mixed
Qwen3-1.7B	44 D – C	+12.04 [+8.39, +16.06]	33	1.4×10^{-9} mixed
Qwen3-8B	42 C – A	−13.87 [−18.25, −9.85]	38	2.9×10^{-11} mixed
Qwen3-8B	42 D – C	−12.04 [−16.42, −8.03]	37	2.1×10^{-8} homogeneous
Qwen3-8B	42 D – B	−25.91 [−31.02, −20.80]	71	5.1×10^{-21} mixed

Adding fine-tuning lowers JSON recovery in every model where it has room to fall: by 27–30 pp for OLMo, by 12–16 pp for Qwen3-1.7B seeds 43 and 44, and by 13.87 pp for Qwen3-8B. At 27B nothing changes, because every condition already succeeds on every item.

7.2 Chart agreement at three levels of strictness (RQ1)

Section 6.6.5 defined three levels at which the primary chart decision can be compared with the reference. Table 7.3 reports all three side by side, together with the number of responses whose outermost JSON object was never closed.

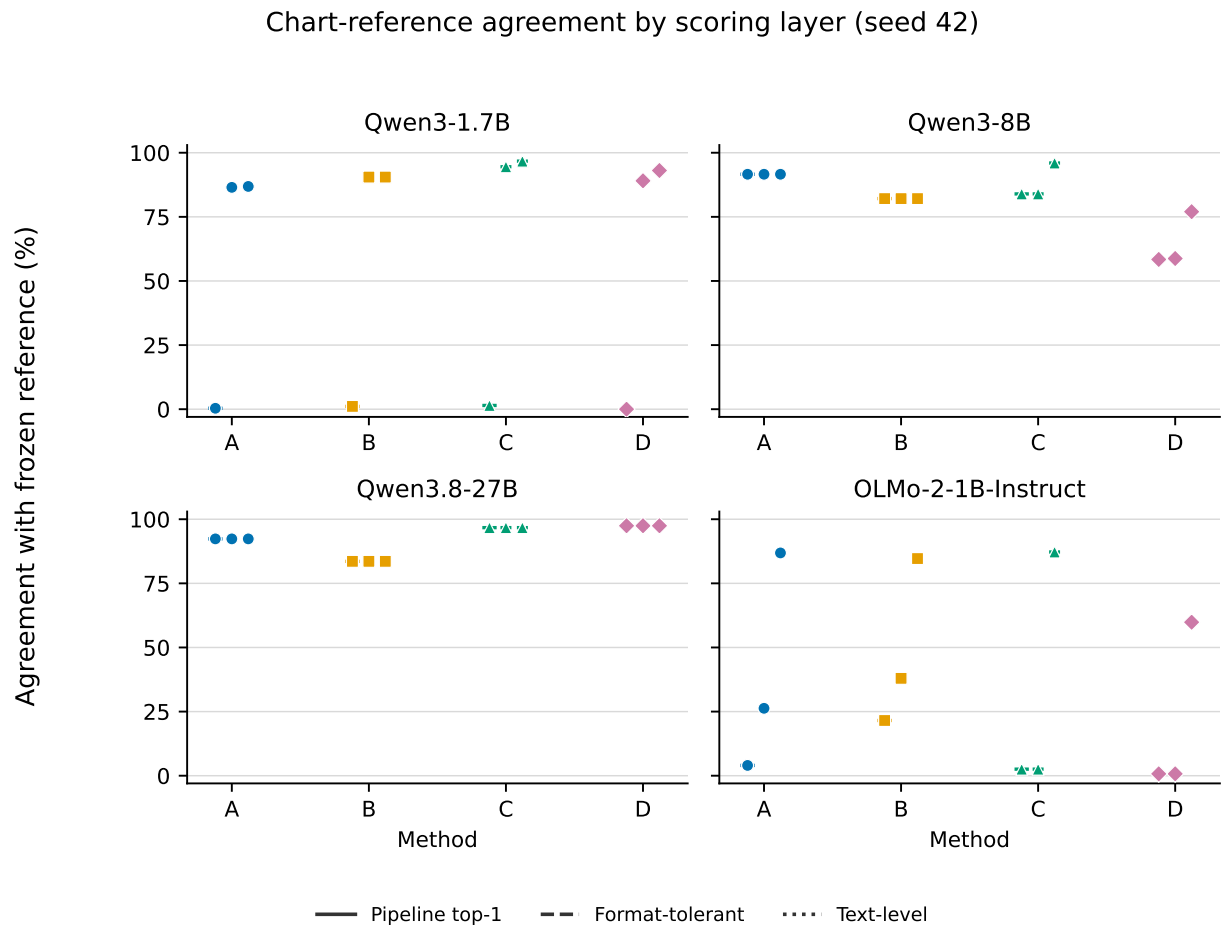


Figure 7.1: Agreement with the frozen chart reference at three scoring layers for seed 42. The layers separate end-to-end pipeline success from format-tolerant and text-level recovery. Values are internal reference-agreement diagnostics, not independent evidence of visualization effectiveness. Methods are A (prompt-only), B (RAG), C (QLoRA), and D (QLoRA+RAG). Source: `tables/format_tolerant_chart_accuracy.csv`.

Table 7.3: Primary-chart agreement in percent at three levels of strictness, and truncated responses out of 274. Levels 2 and 3 are post-hoc diagnostics computed from the stored raw text; level 1 is the pre-specified metric.

Model	Method	Seed	Truncated	(1) Pipeline top-1	(2) Format-tolerant	(3) Text-level	(3) – (1)
Qwen3-1.7B	A	42	0	0.36	86.50	86.86	+86.50
Qwen3-1.7B	A	43	0	0.00	87.23	87.23	+87.23
Qwen3-1.7B	A	44	0	0.36	86.86	86.86	+86.50
Qwen3-1.7B	B	42	0	1.09	90.51	90.51	+89.42
Qwen3-1.7B	B	43	0	1.09	89.78	90.15	+89.06
Qwen3-1.7B	B	44	0	1.09	89.05	89.05	+87.96
Qwen3-1.7B	C	42	1	1.46	94.53	96.72	+95.26
Qwen3-1.7B	C	43	38	80.66	81.02	96.72	+16.06
Qwen3-1.7B	C	44	29	85.40	85.40	97.45	+12.05
Qwen3-1.7B	D	42	1	0.00	89.05	93.07	+93.07
Qwen3-1.7B	D	43	0	89.42	89.42	90.88	+1.46
Qwen3-1.7B	D	44	0	89.05	89.05	89.05	0.00
Qwen3-8B	A	42	0	91.61	91.61	91.61	0.00
Qwen3-8B	B	42	0	82.12	82.12	82.12	0.00
Qwen3-8B	C	42	36	83.94	83.94	95.99	+12.05
Qwen3-8B	D	42	69	58.39	58.76	77.01	+18.62
Qwen3.8-27B	A	42	0	92.34	92.34	92.34	0.00
Qwen3.8-27B	B	42	0	83.58	83.58	83.58	0.00
Qwen3.8-27B	C	42	0	96.72	96.72	96.72	0.00
Qwen3.8-27B	D	42	0	97.45	97.45	97.45	0.00
OLMo-2-1B	A	42	272	4.01	26.28	86.86	+82.85
OLMo-2-1B	A	43	272	6.93	28.83	87.59	+80.66
OLMo-2-1B	A	44	273	5.11	30.29	86.13	+81.02
OLMo-2-1B	B	42	274	21.53	37.96	84.67	+63.14
OLMo-2-1B	B	43	274	24.09	41.61	83.58	+59.49
OLMo-2-1B	B	44	272	23.36	40.51	85.77	+62.41
OLMo-2-1B	C	42	256	2.55	2.55	87.23	+84.68
OLMo-2-1B	C	43	254	2.92	2.92	93.43	+90.51
OLMo-2-1B	C	44	258	2.92	2.92	94.89	+91.97
OLMo-2-1B	D	42	264	0.73	0.73	59.85	+59.12
OLMo-2-1B	D	43	262	2.19	2.19	56.93	+54.74
OLMo-2-1B	D	44	258	2.92	2.92	66.06	+63.14

The final column is the central result of this chapter. It is zero for every 27B condition and for the two Qwen3-8B base-model conditions: there the pipeline metric and the raw text agree completely, because every response is a complete, contract-valid object. Everywhere else it is large, and it is large for two distinct reasons that the three levels separate.

For Qwen3-1.7B in conditions A and B, the responses are complete and decodable — zero truncations — but fail the runtime contract because of the string-valued `encoding` field. Level 2 already recovers the chart decision: 86.5–90.5 % agreement against a pipeline value below 1.5 %. Here the gap is entirely the cost of the runtime contract.

For OLMo in every condition, and for the fine-tuned Qwen3-8B and Qwen3-1.7B conditions with non-zero truncation counts, the second gap appears. OLMo truncates on 254–274 of 274 items in every condition: the model produces a well-formed and often correct prefix and then runs out of the 512-token budget before the outermost object can be closed. Level 3 recovers 84–95 % chart agreement from those prefixes in the A, B and C conditions.

Table 7.4 gives the omnibus test across the four methods for the two chart levels.

Table 7.4: Cochran’s Q across methods A–D on matched items ($n = 274$, $df = 3$). Rates are per-condition agreement in percent.

Model	Seed Outcome	A	B	C	D	Q	p
Qwen3-1.7B	42 pipeline top-1	0.36	1.09	1.46	0.00	5.00	0.172
Qwen3-1.7B	43 pipeline top-1	0.00	1.09	80.66	89.42	657.80	3.0×10^{-142}
Qwen3-1.7B	44 pipeline top-1	0.36	1.09	85.40	89.05	671.96	2.5×10^{-145}
Qwen3-1.7B	42 text-level	86.86	90.51	96.72	93.07	34.84	1.3×10^{-7}
Qwen3-1.7B	43 text-level	87.23	90.15	96.72	90.88	28.86	2.4×10^{-6}
Qwen3-1.7B	44 text-level	86.86	89.05	97.45	89.05	33.79	2.2×10^{-7}
Qwen3-8B	42 pipeline top-1	91.61	82.12	83.94	58.39	133.12	1.2×10^{-28}
Qwen3-8B	42 text-level	91.61	82.12	95.99	77.01	75.16	3.4×10^{-16}
Qwen3.8-27B	42 pipeline top-1	92.34	83.58	96.72	97.45	84.46	3.4×10^{-18}
Qwen3.8-27B	42 text-level	92.34	83.58	96.72	97.45	84.46	3.4×10^{-18}
OLMo-2-1B	42 pipeline top-1	4.01	21.53	2.55	0.73	111.72	4.7×10^{-24}
OLMo-2-1B	43 pipeline top-1	6.93	24.09	2.92	2.19	110.51	8.5×10^{-24}
OLMo-2-1B	44 pipeline top-1	5.11	23.36	2.92	2.92	104.46	1.7×10^{-22}
OLMo-2-1B	42 text-level	86.86	84.67	87.23	59.85	110.05	1.1×10^{-23}
OLMo-2-1B	43 text-level	87.59	83.58	93.43	56.93	158.37	4.1×10^{-34}
OLMo-2-1B	44 text-level	86.13	85.77	94.89	66.06	114.72	1.1×10^{-24}

The omnibus test rejects the hypothesis of equal method performance in every block except Qwen3-1.7B seed 42 on the pipeline metric, where all four conditions fail almost every item and there is nothing to distinguish.

7.2.1 The pipeline contrasts

Table 7.5 reports the four planned contrasts on the pre-specified metric.

Table 7.5: Paired contrasts on pipeline top-1 agreement. Δ in percentage points; the bracketed interval is the 95 % bootstrap interval on the paired difference; p is Holm-corrected within the block. Bold marks significance at Holm-adjusted $\alpha = 0.05$; “ns” marks a non-significant contrast; † marks a pair whose two runs share one recorded code state.

Model	Seed	B – A	C – A	D – C	D – B
Qwen3-1.7B	42	+0.73 (ns)	+1.09 (ns)	–1.46 (ns)	–1.09 (ns)
Qwen3-1.7B	43	+1.09 (ns)	+80.66 [+75.91, +85.04]	+8.76 [+3.65, +13.87]	+88.32 [+84.31, +91.97]
Qwen3-1.7B	44	+0.73 (ns)	+85.04 [+80.66, +89.05]	+3.65 (ns)	+87.96 [+83.94, +91.61]
Qwen3-8B	42	–9.49 † [–13.50, –5.84]	–7.66 [–12.41, –2.92]	–25.55 † [–31.02, –20.07]	–23.72 [–29.56, –17.88]
Qwen3.8-27B	42	–8.76 † [–12.41, –5.11]	+4.38 [+2.19, +6.93]	+0.73 (ns) †	+13.87 [+9.85, +17.88]
OLMo-2-1B	42	+17.52 † [+12.04, +22.99]	–1.46 (ns)	–1.82 (ns) †	–20.80 [–25.55, –15.69]
OLMo-2-1B	43	+17.15 [+11.68, +22.63]	–4.01 (ns)	–0.73 (ns) †	–21.90 [–27.37, –16.79]
OLMo-2-1B	44	+18.25 [+12.77, +23.72]	–2.19 (ns)	0.00 (ns)	–20.44 [–25.91, –15.33]

7.2.2 The same contrasts at the text level

Table 7.6 repeats the contrasts on level 3, which removes both the runtime-contract requirement and the truncation penalty. This is the closest available approximation to comparing the chart decision alone.

Table 7.6: Paired contrasts on text-level chart agreement. Same conventions as Table 7.5.

Model	Seed	B – A	C – A	D – C	D – B
Qwen3-1.7B	42	+3.65 [+1.09, +6.57]	+9.85 [+6.57, +13.50]	–3.65 [–6.20, –1.46]	+2.55 (ns)
Qwen3-1.7B	43	+2.92 (ns)	+9.49 [+6.20, +13.14]	–5.84 [–8.76, –2.92]	+0.73 (ns)
Qwen3-1.7B	44	+2.19 (ns)	+10.58 [+7.30, +14.60]	–8.39 [–11.68, –5.11]	0.00 (ns)
Qwen3-8B	42	–9.49 [†] [–13.50, –5.84]	+4.38 [+1.09, +7.66]	–18.98 [†] [–24.09, –14.23]	–5.11 [–9.49, –0.73]
Qwen3.8-27B	42	–8.76 [†] [–12.41, –5.11]	+4.38 [+2.19, +6.93]	+0.73 (ns) [†]	+13.87 [+9.85, +17.88]
OLMo-2-1B	42	–2.19 (ns) [†]	+0.36 (ns)	–27.37 [†] [–33.94, –20.44]	–24.82 [–31.02, –18.61]
OLMo-2-1B	43	–4.01 [–7.66, –0.73]	+5.84 [+1.82, +9.85]	–36.50 [†] [–42.70, –29.93]	–26.64 [–33.21, –20.07]
OLMo-2-1B	44	–0.36 (ns)	+8.76 [+5.11, +12.77]	–28.83 [–34.67, –22.99]	–19.71 [–25.91, –13.50]

Comparing Table 7.5 with Table 7.6 changes the size and in several cases the sign of the estimated effects:

- **C – A.** On the pipeline metric this contrast ranges from -7.66 pp (Qwen3-8B) to $+85.04$ pp (Qwen3-1.7B seed 44). At the text level it collapses to a narrow, consistent band of $+0.36$ to $+10.58$ pp, positive in all eight blocks and significant in seven. Fine-tuning improves the chart decision by roughly 4–11 pp; the remaining 70–80 pp of the pipeline effect at 1.7B is output-format compliance, and the negative pipeline value at 8B is truncation.
- **B – A.** Retrieval alone is small and positive at 1.7B ($+2.19$ to $+3.65$ pp), close to zero at OLMo (-4.01 to $+0.36$ pp), and **significantly negative at both larger Qwen3 scales**: -9.49 pp at 8B and -8.76 pp at 27B. Both of those contrasts come from code-state homogeneous pairs, and both are identical at all three measurement levels because those conditions have no format or truncation losses. This is the cleanest single effect in the study.
- **D – C.** This contrast is negative in eleven of the twelve blocks where it can be computed at the text level, and significantly negative in eight. Adding retrieval on top of a fine-tuned adapter lowers chart agreement: by 3.7–8.4 pp for Qwen3-1.7B, by 18.98 pp for Qwen3-8B and by 27.4–36.5 pp for OLMo. The only exception is Qwen3.8-27B, where $D - C$ is $+0.73$ pp and not significant. The 8B and the two OLMo seed-42/43 contrasts are code-state homogeneous.
- **The 27B block is unchanged** between the two tables, because that model has no format or truncation losses to recover.

7.2.3 Why retrieval lowers agreement at 8B and 27B: one traceable mechanism, replicated

The two significantly negative $B - A$ contrasts have the same cause, and it is directly inspectable.

At Qwen3.8-27B, $B - A$ is -8.76 pp, corresponding to 24 additional disagreements out of 274. Under prompt-only the model substitutes **donut** for a reference **pie** on 5 items; under retrieval it does so on **28 of the 36 pie items**. The net change of $+23$ pie→donut substitutions accounts for essentially the whole difference.

At Qwen3-8B, $B - A$ is -9.49 pp, corresponding to 26 additional disagreements. Under prompt-only the model produces **no** pie→donut substitutions at all; under retrieval it produces **28**. Here the substitution more than accounts for the entire drop; the remaining error families (**bar** → **histogram**, **stacked_bar** → **bar**) are almost unchanged between the two conditions. The same substitution appears again in the 8B fine-tuning-plus-retrieval condition, at 29 items.

The cause is a single sentence in the retrieved corpus. The chart-selection guideline document contains the passage “*Prefer donut over pie: Donut charts are easier to read because the center can show a total value.*” Retrieval surfaced this guidance, both models followed it, and the reference-agreement metric scored every instance as an error.

This is a measurement result rather than a quality result. The metric records that retrieval deviated from the reference; it cannot decide whether a donut or pie is better for those items. The exploratory human study rates retrieval-only highly overall but does not isolate these substitutions with enough agreement for an item-level verdict. What the automatic result establishes across two model scales is that a negative $B - A$ is not equivalent to “retrieval made the recommendations worse”.

7.2.4 Macro-F1 and the class distribution

The held-out split is heavily imbalanced. The reference primary chart is **bar** for 208 of 274 items (75.9 %), **pie** for 36, **line** for 17, **stacked_bar** for 11 and **scatter** for 2; the corresponding task types are **comparison**, **part_to_whole**, **trend**, **composition** and **correlation**. Macro-F1 therefore separates broad competence from majority-class behaviour.

Table 7.7: Macro-F1 over chart classes, by condition.

Model	A	B	C	D
Qwen3-1.7B, seed 42	0.111	0.022	0.064	0.000
Qwen3-1.7B, seed 43	0.000	0.113	0.391	0.613
Qwen3-1.7B, seed 44	0.018	0.022	0.627	0.531
Qwen3-8B, seed 42	0.558	0.462	0.530	0.310
Qwen3.8-27B, seed 42	0.445	0.430	0.762	0.789
OLMo-2-1B, seed 42	0.188	0.101	0.018	0.011
OLMo-2-1B, seed 43	0.079	0.238	0.020	0.017
OLMo-2-1B, seed 44	0.018	0.176	0.020	0.027

Macro-F1 is computed from the parsed runtime object and therefore inherits the same format penalty as

pipeline top-1; the values for conditions that fail the contract are not informative about the chart decision. The informative comparisons are the two larger Qwen3 profiles, where the base-model conditions parse perfectly.

At 27B, macro-F1 is 0.445 for prompt-only and 0.762 for fine-tuned, against top-1 values of 92.34 % and 96.72 %. The gap between a top-1 of 92 % and a macro-F1 of 0.44 shows how much of the prompt-only performance rests on the dominant **bar** class. The per-class breakdown for the 27B fine-tuned condition confirms this: F1 is 1.00 for **line** (support 17), 0.98 for **bar** (208), 0.97 for **pie** (36), 1.00 for **scatter** (2) and 0.63 for **stacked_bar** (11), where recall on **stacked_bar** is only 0.45 because six of eleven stacked-bar references are answered with a plain **bar**.

At 8B the picture differs. Prompt-only reaches macro-F1 0.558 with per-class F1 of 1.00 for **line**, 0.95 for **bar**, 0.93 for **pie**, 1.00 for **scatter** and 0.59 for **stacked_bar**. Fine-tuning *lowers* macro-F1 to 0.530 even though it raises text-level agreement by 4.38 pp, because the adapter’s losses are concentrated in the smaller classes: per-class F1 falls to 0.67 for **line** and to 0.15 for **stacked_bar** while **bar** and **pie** hold at 0.93 and 0.96. Fine-tuning at 8B therefore moves the model toward the majority class, which the aggregate agreement metric rewards and the class-balanced metric penalises.

7.2.5 Alternatives

The number of items for which a condition offers more than one distinct recommendation separates the base-model conditions from the fine-tuned ones completely. Qwen3-8B offers alternatives on all 274 items under both A and B; Qwen3.8-27B on 266 (A) and 267 (B); Qwen3-1.7B on 0–4 items and OLMo on 8–68 items depending on condition and seed. **Every fine-tuned condition of every model offers alternatives on exactly zero items.**

The adapters learned to emit an empty **alternatives** list because that is what the frozen training targets predominantly contain. This is a faithful reproduction of the supervision, and it is a behavioural change that the aggregate agreement metrics do not capture: a system that proposes one chart and a system that proposes a ranked set of defensible charts are different products, and fine-tuning on this dataset converts the second into the first.

7.3 Supporting analysis: model scale and model family

Within the Qwen3 family the base models improve steadily with capacity, but the *method effects* do not scale in the same direction, and one of them reverses.

Prompt-only text-level agreement rises from 86.9–87.2 % at 1.7B to 91.61 % at 8B and 92.34 % at 27B. Output-contract satisfaction rises from under 1.5 % at 1.7B to 100 % at both larger scales. The base model therefore acquires the ability to satisfy the output contract somewhere between 1.7B and 8B, and after that point fine-tuning has nothing to add on that axis — at 8B it actively costs 13.87 pp of contract satisfaction, because the adapter teaches a longer target that the 512-token budget cannot always finish.

The marginal value of fine-tuning for the chart decision falls with capacity but stays positive: +9.5 to +10.6 pp at 1.7B, +4.38 pp at 8B, +4.38 pp at 27B. The marginal value of retrieval changes sign: +2.2 to +3.7 pp at 1.7B, −9.49 pp at 8B, −8.76 pp at 27B. Combining the two is neutral at 27B (+0.73 pp, ns), clearly harmful at 8B (−18.98 pp) and mildly harmful at 1.7B (−3.7 to −8.4 pp).

The two larger Qwen3 profiles agree closely on every base-model quantity — 91.61 % against 92.34 % prompt-only agreement, identical B − A direction and magnitude within one percentage point, the same donut mechanism — and diverge on what fine-tuning does. At 27B the adapter improves both agreement (96.72 %) and macro-F1 (0.762) without any loss of output validity. At 8B the same recipe improves text-level agreement (95.99 %) but loses 13.87 pp of contract satisfaction to truncation and lowers macro-F1. The difference between the two is not capacity alone: the 8B adapter trained with a 4,096-token window and the 27B adapter with 1,024 (Table 6.4), and the two models have very different context architectures.

The cross-family comparison at the small scale gives a partly different picture. OLMo-2-1B-Instruct reaches text-level chart agreement of 86.1–87.6 % under prompt-only — indistinguishable in magnitude from Qwen3-1.7B’s 86.9–87.2 % — so the two families are comparable on the chart decision itself. They are not comparable on producing a usable record. OLMo recovers a complete JSON object on 30–33 % of items against Qwen3-1.7B’s 99.6–100 %, and truncates on 272 of 274 items against zero. Fine-tuning, which improves Qwen3-1.7B’s contract compliance, makes OLMo’s recoverability worse (2.9 %), because the adapter teaches it to emit the long reference format that it cannot finish within 512 tokens. Retrieval after fine-tuning is worse still, and it is the only place in the whole experiment where a method degrades the chart decision itself by a large margin (D − C of −27 to −37 pp at the text level).

The two families therefore agree on one pattern and disagree on another. Both show that the base models can largely identify the reference chart from the prompt alone, and both show that adding retrieval on top of an adapter does not help. They disagree on whether fine-tuning is a net gain: for Qwen3 it improves the

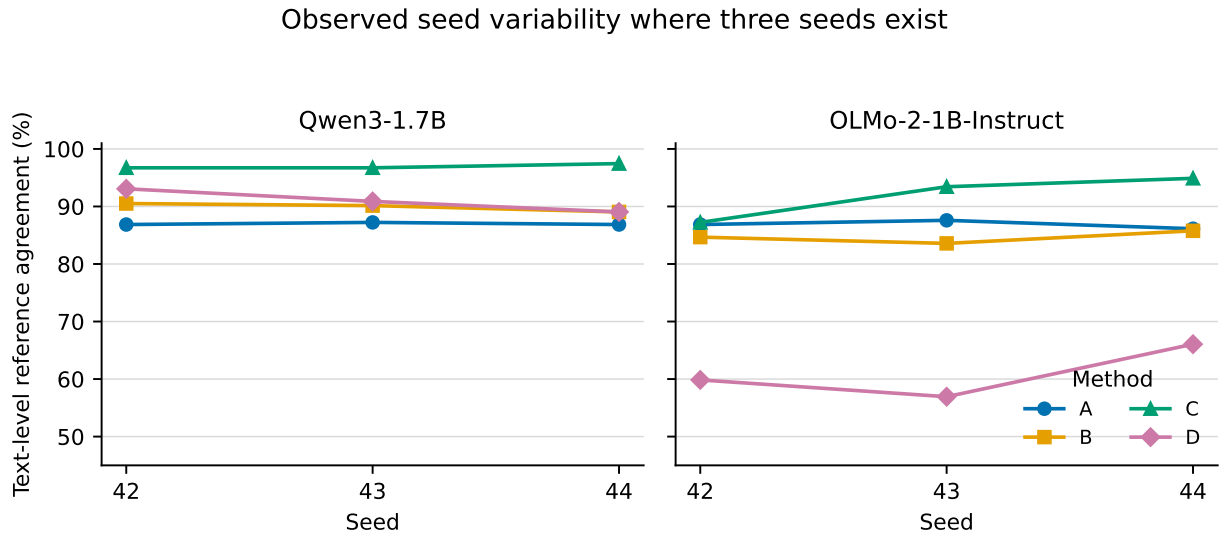


Figure 7.2: Observed text-level chart-reference agreement across seeds 42–44 for the two profiles with three-seed coverage. Lines aid reading across seeds; they are not confidence intervals. The 8B and 27B profiles are omitted because only seed 42 exists. Source: `tables/format_tolerant_chart_accuracy.csv`.

chart decision at every observed scale; for OLMo under this common recipe and this output budget, the chart decision improves at the text level but the usable output collapses.

Because the two families differ simultaneously in architecture, tokenizer, pretraining corpus, post-training recipe and native context length (4,096 for the OLMo checkpoint against 40,960 for the small Qwen3 profile), none of these differences can be isolated as the cause. The comparison supports the statement that the Qwen3 pattern does not automatically transfer to an independently developed family at a comparable parameter scale. It supports no causal attribution to any single component.

7.4 Stability across seeds and under perturbation (RQ3)

7.4.1 Seed variability

Three-seed coverage exists only for Qwen3-1.7B and OLMo-2-1B. Table 7.8 shows the spread on text-level chart agreement, which is the outcome least contaminated by the format and truncation artifacts.

Table 7.8: Text-level chart agreement across seeds, in percent. Values are shown individually; sd is the sample standard deviation over the three seeds and is descriptive only.

Model	Method	Seed 42	Seed 43	Seed 44	Mean	sd
Qwen3-1.7B	A	86.86	87.23	86.86	86.98	0.21
Qwen3-1.7B	B	90.51	90.15	89.05	89.90	0.76
Qwen3-1.7B	C	96.72	96.72	97.45	96.96	0.42
Qwen3-1.7B	D	93.07	90.88	89.05	91.00	2.01
OLMo-2-1B	A	86.86	87.59	86.13	86.86	0.73
OLMo-2-1B	B	84.67	83.58	85.77	84.67	1.10
OLMo-2-1B	C	87.23	93.43	94.89	91.85	4.07
OLMo-2-1B	D	59.85	56.93	66.06	60.95	4.66

The seed spread is small for the untrained conditions — standard deviations of 0.2–1.1 pp for A and B in both families — which is expected, since only generation sampling varies. It is larger for the trained conditions, where the training trajectory also varies: 0.4 pp for Qwen3-1.7B C but 4.1 pp for OLMo C and 4.7 pp for OLMo D. Reporting three seeds is therefore not a formality; the trained conditions in the weaker family vary by several percentage points between otherwise identical runs, which is consistent with published observations that random seeds produce measurable variation in fine-tuned language models at both aggregate and per-item level [Bui et al. \(2025\)](#).

Two cautions apply to Table 7.8. First, the three Qwen3-1.7B seeds are not three draws from one procedure: seed 42 was trained under a different code state, a different per-device batch size and bf16 precision, while seeds 43 and 44 were trained under the current state on a Tesla T4 in fp16 (Table 6.4). Its lower contract

compliance in Table 7.1 reflects that difference rather than seed noise. Second, no standard deviation is reported for Qwen3-8B or Qwen3.8-27B, because a single run cannot estimate one. The scale comparison in Section 7.3 rests on three-seed evidence at the small scale and single-run evidence at the two larger scales, and that asymmetry is a property of the evidence, not of the models.

7.4.2 Paraphrase robustness

Table 7.9 reports the two paraphrase quantities. Consistency is the share of items whose primary predicted chart is unchanged after the meaning-preserving rewrite; accuracy is pipeline agreement on the paraphrased briefs, and the delta compares it with pipeline agreement on the same items unperturbed.

Table 7.9: Behaviour under the paraphrase perturbation, in percent. Multiple values are seeds 42 / 43 / 44.

Model	Method	Consistency	Paraphrase accuracy	Δ vs original
Qwen3-1.7B	A	0.00 / 0.00 / 0.36	0.36 / 0.36 / 0.73	≈ 0
Qwen3-1.7B	B	1.09 / 0.36 / 1.09	1.09 / 0.73 / 1.46	≈ 0
Qwen3-1.7B	C	1.46 / 81.39 / 86.86	3.65 / 81.75 / 84.31	+2.19 / +1.09 / -1.09
Qwen3-1.7B	D	0.00 / 97.45 / 98.54	0.36 / 89.78 / 87.59	+0.36 / +0.36 / -1.46
Qwen3-8B	A	100.00	91.61	0.00
Qwen3-8B	B	100.00	82.12	0.00
Qwen3-8B	C	83.58	83.94	0.00
Qwen3-8B	D	69.34	58.39	0.00
Qwen3.8-27B	A	98.54	91.24	-1.10
Qwen3.8-27B	B	96.72	83.94	+0.36
Qwen3.8-27B	C	100.00	96.72	0.00
Qwen3.8-27B	D	100.00	97.45	0.00
OLMo-2-1B	A	1.82 / 2.19 / 2.92	6.57 / 4.38 / 6.20	small positive
OLMo-2-1B	B	9.12 / 10.95 / 9.49	22.99 / 25.55 / 23.36	small negative
OLMo-2-1B	C	2.55 / 2.92 / 2.92	2.55 / 3.28 / 3.28	≈ 0
OLMo-2-1B	D	1.09 / 2.19 / 2.55	1.46 / 2.19 / 2.19	≈ 0

Both quantities are computed from the parsed runtime object, so a condition that fails the output contract shows near-zero consistency simply because neither prediction has a readable chart. The consistency values are informative only where the condition also satisfies the contract. Where that holds, the picture is clear and favourable: Qwen3-8B is perfectly consistent under both base-model conditions (100.00 %), Qwen3.8-27B is 96.7–100 % consistent in all four conditions, and the current-state Qwen3-1.7B fine-tuned conditions are 81.4–98.5 % consistent. The weakest consistency among usable conditions is Qwen3-8B under D, at 69.34 %.

Where the paraphrase does change the answer, the change is close to accuracy-neutral: no condition loses more than 1.5 pp of agreement, and eight of the sixteen conditions lose exactly nothing. Within the tested synonym family, the systems that produce usable output are stable, and the instability that does exist is not systematically in the direction of worse answers. This statement does not extend to paraphrase families that the perturbation set does not contain — sentence restructuring, register changes, translations, or rewrites that touch the domain nouns.

7.4.3 Behaviour on under-specified briefs

The clarification rate — the share of responses to an under-specified brief that ask for the missing information or signal uncertainty — is at or near zero everywhere. It is 0.0 % in every fine-tuned condition of every model (C and D, all seeds, all four models). Among the base-model conditions it reaches 3.65 % at most (Qwen3-1.7B A, seed 42) and is 0.73 % and 0.36 % for the two Qwen3-8B conditions, 1.09 % for both 27B conditions and 1.09–3.28 % for OLMo.

The complementary quantity shows what the systems do instead. Qwen3-8B emits a complete valid record for 100 % of the under-specified briefs under both base-model conditions, and the 27B model for 99.27 %. The fine-tuned conditions of both larger models also stay high, at 78.1–100 %.

The desirable behaviour on a brief with a removed constraint and a removed KPI is not necessarily a confident complete recommendation. The measured behaviour is that the systems almost never ask. Fine-tuning removes even the small tendency that the base models show, which is expected given that no training target in the frozen dataset is a clarification request: the adapters were never shown that this is a permitted response. Two limits apply to reading these numbers. The detector is a regular-expression heuristic over the generated text and will miss an implicit request for information. And the perturbation removes the constraints field, the last KPI and the last column, which does not always make a brief genuinely unanswerable. The finding is therefore reported as: under this perturbation and this detector, no condition demonstrates a reliable clarification behaviour.

7.5 Failure modes

Table 7.10 aggregates the per-item failure records. “Parse failures” counts items whose raw text did not yield a runtime object; “no primary chart” counts items for which the pipeline had no readable chart to score, which the top-1 metric records as incorrect.

Table 7.10: Failure counts per condition, out of 274 items.

Model	Method	Seeds	Parse failures	Dominant parse error	Items with no primary chart
Qwen3-1.7B	A	42 / 43 / 44	273 / 274 / 273	schema_error: encoding is a string	273 / 274 / 273
Qwen3-1.7B	B	42 / 43 / 44	271 / 270 / 271	schema_error: encoding is a string	271 / 270 / 271
Qwen3-1.7B	C	42 / 43 / 44	270 / 44 / 33	seed 42 schema_error; seeds 43–44	270 / 45 / 33
Qwen3-1.7B	D	42 / 43 / 44	274 / 4 / 0	no_json_found seed 42 schema_error; seed 43	274 / 4 / 0
Qwen3-8B	A	42	0	—	0
Qwen3-8B	B	42	0	—	0
Qwen3-8B	C	42	38	no_json_found	38
Qwen3-8B	D	42	71	no_json_found	72
Qwen3.8-27B	A–D	42	0	—	0
OLMo-2-1B	A	42 / 43 / 44	202 / 198 / 200	no_json_found	262 / 255 / 258
OLMo-2-1B	B	42 / 43 / 44	169 / 157 / 168	no_json_found	201 / 191 / 199
OLMo-2-1B	C	42 / 43 / 44	266 / 266 / 266	no_json_found	266
OLMo-2-1B	D	42 / 43 / 44	271 / 267 / 265	no_json_found	271 / 267 / 265

Two failure modes dominate, and they are distinguishable.

Type error in the encoding field. For Qwen3-1.7B under A and B, and under C and D at seed 42, the model returns a complete, well-formed JSON object in which `encoding` is a bare string — typically the name of the x-axis column, for example `"encoding": \"payment_method_code"` — rather than an object with named channels. The response is otherwise correct, including the chart type. The runtime contract rejects it, so the item is scored as a chart error. Zero of these responses are truncated. No other model in the study shows this failure.

Truncation at the output budget. For OLMo in all conditions, and for the fine-tuned Qwen3-8B and Qwen3-1.7B conditions, the failure is `no_json_found` and the cause is that the response never closed. Of the 191 OLMo prompt-only responses at seed 42 that failed to parse, 189 do not end with a closing brace. The stored prefixes are well-formed and often correct: an OLMo seed-43 fine-tuned response begins with a valid `context_summary`, a correct `chart_type`, and a fully populated encoding, and stops in the middle of the `having` array. The behaviour is consistent across the affected conditions: the model emits the long reference format faithfully and runs out of budget.

The Qwen3-8B row isolates this mechanism cleanly, because it is the one model where the same base produces zero truncations without an adapter and a substantial number with one. Fine-tuning raises truncations from 0 to 36 of 274, and adding retrieval on top raises them to 69. Retrieval consumes input budget and appears to lengthen the response; both effects push more items past the ceiling.

The mean raw response length is consistent with this account. OLMo prompt-only responses average about 2,150 characters and 27B prompt-only responses about 2,300, both against a hard ceiling of 512 generated tokens.

7.6 Retrieval grounding

Table 7.11 reports the grounding diagnostic for the sixteen RAG conditions. Claim coverage must be read first: it is the share of items that contributed at least one scorable rationale claim, and where it is low the supported-claim rate describes only a handful of claims.

Table 7.11: Lexical-proxy grounding for conditions B and D. All conditions retrieved documents for all 274 items.

Model	Method	Seed	Claim coverage (%)	Scored claims	Supported claims (%)
Qwen3-8B	B	42	100.0	832	63.33
Qwen3.8-27B	B	42	100.0	695	61.08
Qwen3.8-27B	D	42	100.0	330	48.33
Qwen3-1.7B	D	43	98.54	310	31.48
Qwen3-1.7B	D	44	100.0	304	31.20
Qwen3-8B	D	42	74.09	206	57.64
Qwen3-1.7B	B	42	not recorded	9	77.78
Qwen3-1.7B	B	43	1.46	13	64.58
Qwen3-1.7B	B	44	1.09	9	77.78
Qwen3-1.7B	D	42	0.0	0	not available
OLMo-2-1B	B	42	18.61	91	39.05
OLMo-2-1B	B	43	20.44	101	46.58
OLMo-2-1B	B	44	21.53	116	49.72
OLMo-2-1B	D	42	1.09	3	100.00
OLMo-2-1B	D	43	2.55	7	28.57
OLMo-2-1B	D	44	3.28	9	44.44

Only six of the sixteen conditions have claim coverage above 70 %, and only those six support a reading of the supported-claim rate. Among them, the two base-model conditions of the larger Qwen3 profiles are the highest, at 63.33 % (8B) and 61.08 % (27B) over 832 and 695 claims respectively. The fine-tuned-plus-retrieval conditions are lower: 57.64 % at 8B, 48.33 % at 27B and 31.2–31.5 % at 1.7B. The remaining ten conditions have coverage between 0 % and 21.5 %, because their rationales could not be extracted from unparseable or truncated output; the OLMo D seed-42 value of 100 % is computed over three claims and carries no weight.

Two limits bound every value in this table. Support is decided by content-word overlap, and lexical overlap is not entailment: a rationale can reuse the vocabulary of a retrieved passage without being justified by it, and it can be justified by a passage it does not lexically resemble. Retrieval relevance — whether the retriever selected the *right* passages — was not measured at all, because the evaluation needs independent relevance judgements and none exist. The grounding numbers therefore describe how much retrieved vocabulary reappears in the rationales and nothing stronger.

7.7 Computational cost

Table 7.12 reports mean generation latency per item. The comparison across conditions is confounded by hardware and is reported as a descriptive cost record for the executed configuration only.

Table 7.12: Mean generation latency per item in seconds, with the inference device.

Model	A	B	C	D
Qwen3-1.7B, seed 42	7.1 (A30)	6.6 (A30)	9.4 (A30)	9.7 (A30)
Qwen3-1.7B, seeds 43/44	7.1 (A30)	6.6-6.7 (A30)	38.7-39.0 (L4)	57.2-59.1 (T4)
Qwen3-8B, seed 42	33.0 (GB10)	34.4 (GB10)	22.5 (GB10)	23.1 (GB10)
Qwen3.8-27B, seed 42	30.9 (H100 NVL)	29.4 (H100 NVL)	76.2 (GB10)	81.3 (GB10)
OLMo-2-1B, seed 42	10.1 (GB10)	10.0 (GB10)	13.6 (H100 NVL)	13.8 (H100 NVL)

Where the device is held constant, three regularities appear. Retrieval is close to free at generation time: B is within 1.4 s of A in every same-device pair. Loading a 4-bit quantized base model with an adapter is more expensive than running the base model in its normal precision at 1.7B, where C is 2.3 s slower than A on the same A30. And D is consistently a little slower than C, by 0.3–5.1 s.

The Qwen3-8B row is the only complete four-condition comparison on a single device, and it runs against that pattern: the fine-tuned conditions are about 11 s *faster* per item than the base-model conditions. This is consistent with the response-length evidence in Section 7.2.5 — the base-model conditions emit alternatives on all 274 items while the fine-tuned conditions emit none, so they have less to generate — but the runs were not instrumented to measure generated token counts directly, so the explanation is not established by the artifacts.

The values across devices are not comparable. The 57–59 s per item for Qwen3-1.7B D at seeds 43/44 reflects a Tesla T4, not a property of method D; the same method on an A30 at seed 42 takes 9.7 s.

The total resource footprint of the reported experiment was approximately 162 GPU-hours of inference across the thirty-two runs and 23 GPU-hours of adapter training across the eight adapters, on five different GPU types. Training one adapter took 0.54 h for OLMo-2-1B on an H100 NVL, 1.0–3.4 h for Qwen3-1.7B depending on device, 4.9 h for Qwen3-8B and 8.8 h for Qwen3.8-27B.

7.8 Human evaluation (RQ2 and RQ3)

The final analysis excludes all test accounts and earlier rubric versions. Fifteen participants consented to the open bilingual study. Ten completed all eight assigned pages, one partial session contributed one page, and four registered participants submitted none, yielding 81 pages for 32 outputs. Each output received two to four pages. The intended three-page coverage was therefore not reached for every output, and 11 of 486 possible dimension judgements are missing. The analysis retains these missing values rather than coding them as neutral scores.

Table 7.13: Human ratings by method, averaged within each brief and then equally across the eight briefs. Scores range from 1 (low) to 5 (high). The last column reports ordinal Krippendorff’s α across the 32 multiply rated outputs.

Dimension	A	B	C	D	α
Chart appropriateness	4.38	4.48	4.02	3.92	0.066
Layout quality	4.40	4.33	3.90	3.98	0.045
Styling and accessibility	4.10	4.33	4.25	4.21	0.579
Interaction design	4.58	4.54	4.13	4.38	0.062
Rationale quality	4.29	4.52	3.76	4.13	0.209
Overall usefulness	4.21	4.52	4.05	4.04	0.167

All method–dimension means lie between 3.76 and 4.58. Retrieval-only (B) has the highest equal-brief mean for chart appropriateness, styling and accessibility, rationale quality, and overall usefulness. Prompt-only (A) is slightly higher for layout quality and interaction design. The largest descriptive B–A difference is +0.31 points for overall usefulness; the layout and interaction differences favouring A are only 0.06 and 0.04 points. The fine-tuned conditions do not exceed B on any dimension. Relative to C, adding retrieval in D is associated with higher interaction (+0.25) and rationale (+0.36) scores, while overall usefulness is essentially unchanged (+0.01 for C over D).

Agreement is the principal qualification. Ordinal α is 0.579 for styling and accessibility but ranges from 0.045 to 0.209 for the other five dimensions. Thus, raters often disagree about the same output, especially on layout, interaction, and chart appropriateness. Together with convenience recruitment, eight briefs, shared raters, and incomplete three-page coverage, this rules out a confirmatory ranking of the four methods. The human data support a bounded descriptive conclusion: participants generally rated all four systems positively, retrieval-only obtained the strongest average profile in this sample, and the automatic reference-agreement penalty for retrieval does not translate into a corresponding decline in perceived usefulness.

Two planned automatic quantities remain outside the executed analysis. Independent chart effectiveness was not computed because its set-valued scorer is not implemented; automatic chart results are therefore agreement with the project’s reference. Retrieval relevance metrics such as Recall@3, MRR@3, and nDCG@3 were also not computed because no independent relevance judgements exist.

Chapter 8

Discussion and Threats to Validity

Chapter 7 reported what the runs produced. This chapter explains what those observations mean, what caused them, and how far they can be trusted. It first relates the findings to the three research questions retained from the exposé. It then examines the measurement decomposition that changed the size and in several cases the sign of the estimated effects, and analyses the two failure modes that dominate the experiment. The following sections discuss why retrieval behaves differently after fine-tuning than before it, what the scale and cross-family observations do and do not license, and which methodological problems arose during the project and how they were handled. The chapter closes with a systematic account of the threats to validity and with the practical implications that the evidence actually supports.

The chapter deliberately keeps two things apart. An explanation of a mechanism inside this experiment — for example, that a specific guideline sentence produced a specific substitution — is supported by the artifacts. A general claim about how retrieval or fine-tuning behaves on structured design tasks is not, and is not made.

8.1 Interpretation of Main Results

RQ1 (Accuracy) — How accurately can the system map dashboard tasks/KPIs to suitable chart types (Top-1/Top-3 where reference labels exist) across prompt-only, RAG, fine-tuning, and fine-tuning+RAG? For Top-1 chart agreement, fine-tuning is the most consistently positive intervention. The C–A contrasts range from +0.36 to +10.58 percentage points across all eight comparable blocks, and seven are statistically significant. Retrieval is more heterogeneous: it improves both Qwen3-1.7B configurations by +2.19 to +3.65 points, has little effect on OLMo’s chart choice, and reduces agreement for the Qwen3-8B and Qwen3.8-27B baselines by 9.49 and 8.76 points, respectively. Adding retrieval after fine-tuning is detrimental in eleven of twelve evaluated contrasts; the sole positive exception is Qwen3.8-27B at +0.73 points and is not statistically significant. Pipeline-level scores must be interpreted together with structure and truncation failures. A defensible Top-3 result cannot be reported because the released reference data do not provide an auditable ranked set of alternative labels.

RQ2 (Actionability & Structure) — Do RAG and/or fine-tuning improve actionability and structure, measured via schema compliance and human rubric ratings? The automatic structure component shows strong model-specific effects. The Qwen3-8B and Qwen3.8-27B baselines satisfy the runtime contract for all evaluated examples in conditions A and B. Unadapted Qwen3-1.7B remains below 1.5 % because it often emits a JSON-encoded string instead of the required object, whereas OLMo reaches roughly 30–33 % strict validity and often exhausts the generation budget before closing the object. Fine-tuning largely repairs the small Qwen model but reduces strict JSON validity in some Qwen3-8B and OLMo settings as adapted outputs become longer under the fixed 512-token limit. The human-rubric part of RQ2 remains unanswered: the repository contains a prepared candidate pool, but no auditable rater export or agreement statistics. Consequently, the thesis makes no empirical claim about human-rated actionability.

RQ3 (Rationale & Robustness) — Do RAG and/or fine-tuning improve heuristic-aligned rationales and robustness under paraphrased or partially specified briefs? Paraphrase stability is high for the strongest baselines: Qwen3-8B is fully stable in the evaluated conditions, and Qwen3.8-27B ranges from 96.7 % to 100 %. The available fine-tuned Qwen3-1.7B rows range from 81 % to 99 %. Clarification behaviour is uncommon overall, reaching at most 3.65 %, and is absent from the completed fine-tuned conditions. Rationale quality is supported only by an automatic lexical-grounding proxy; retrieval relevance was not independently judged. The evidence therefore supports local robustness under the benchmark perturbations but does not justify a broad claim that RAG or fine-tuning improves human-perceived rationale quality.

Model scale, model family, seed variation, and failure modes are used below as supporting analyses for these three questions. They are not treated as additional research questions.

8.2 Effect of Retrieval

Retrieval alone and retrieval after adaptation are two different interventions in this experiment, and their effects differ both in sign and in mechanism.

Before adaptation, the effect depends entirely on what the base model is missing. For OLMo, retrieval supplies structure the model does not have: JSON recovery rises by 15–17 pp across all three seeds while text-level chart agreement stays essentially unchanged (−4.01 to +0.36 pp). The retrieved passages help the model produce more of a well-formed record before the budget runs out, without changing which chart it names. For Qwen3-1.7B the same intervention gives a small positive effect on the chart decision (+2.19 to +3.65 pp) and no effect on format. For the two larger Qwen3 models, which have nothing structural to gain, the only measurable effect is on content — and it is negative.

After adaptation, retrieval competes with what the adapter has learned. The adapter encodes a specific output format and a specific mapping policy derived from the frozen targets; the retrieved passages introduce text that was never present during training and that sometimes recommends a different choice. The D − C contrast at the text level is negative in eleven of twelve blocks, with effect sizes of −3.7 to −8.4 pp for Qwen3-1.7B, −18.98 pp for Qwen3-8B and −27.4 to −36.5 pp for OLMo. The magnitude tracks how fragile the model’s output behaviour already is: the more marginal the model’s capacity for the task, the more damage the additional context does.

Three mechanisms are consistent with the observations, and the study can separate them only partially.

Guideline conflict. This mechanism is directly traceable, and it now replicates. The retrieved chart-selection document states *“Prefer donut over pie: Donut charts are easier to read because the center can show a total value.”* At 27B, retrieval raised pie→donut substitutions from 5 to 28 of 36 pie items, which accounts for almost the entire −8.76 pp B − A difference. At 8B, the same retrieval raised them from 0 to 28, which more than accounts for the −9.49 pp difference. Two independently sized models, two code-state homogeneous contrasts, one sentence in the corpus. The models followed the guidance they were given; the metric penalised them for doing so. This is not a model failure and it is not a retrieval failure. It is a disagreement between the knowledge base and the reference, exposed by a metric that treats the reference as ground truth.

Input-budget competition. Retrieved passages consume part of the 3,584-token input budget. For a model that is already near the output ceiling, less room and a longer prompt make truncation more likely. Qwen3-8B shows this cleanly: truncations rise from 36 under C to 69 under D, and JSON recovery falls by 12.04 pp on a code-state homogeneous pair. OLMo shows the same direction at a higher baseline.

Distraction. For Qwen3-1.7B under D, the error profile changes shape rather than merely degrading: `bar` references are answered with `stacked_bar`, `line`, `scatter` and `pie`. A small model given several partly relevant design rules appears to be pulled toward whichever chart the retrieved text mentions.

The practical reading is narrow and specific to this setting. Retrieval was investigated here as a mechanism for supplying explicit domain guidance at inference time. On this task it helped a weak model produce more usable structure, it did not improve the chart decision at any scale and significantly worsened it at the two larger ones, and it degraded the chart decision further when combined with an adapter trained on the same task. Whether the degradation reflects worse recommendations or merely different ones cannot be settled by reference agreement, and for the donut cases there is concrete reason to think the metric is at least partly at fault.

8.3 Effect of Fine-Tuning and Measurement Choice

The single most consequential finding of this study is not about retrieval or fine-tuning. It is that a metric which requires a valid record before it will score a decision measures both, and reports the result as if it measured only the decision.

The pre-specified top-1 metric reads the primary chart from the parsed runtime object. If the object does not parse, the item is counted as an incorrect chart. This is a defensible design for an end-to-end system evaluation: a recommendation the pipeline cannot read is useless in practice. It is not a defensible basis for the claim “fine-tuning improves chart selection”, because it cannot distinguish a model that chose the wrong chart from a model that chose the right chart inside a record with one type error.

The size of the difference is not marginal. Qwen3-1.7B in the prompt-only condition scores 0.0–0.4 % on the pipeline metric and 86.5–87.2 % once the chart is read from the raw JSON object. A reader of the pipeline number alone would conclude that a 1.7B instruction-tuned model cannot perform this task at all. The correct conclusion is that it performs the analytical part of the task about as well as the OLMo model of comparable size, and fails a single structural convention: it writes `"encoding": "\payment_method_code"` where the contract expects `"encoding": {"x": ..., "y": ..., "aggregate": ...}`.

The decomposition also changes a sign. At Qwen3-8B, the pipeline metric reports C − A as −7.66 pp, which reads as “fine-tuning made chart selection worse”. At the text level the same contrast is +4.38 pp and significant in the opposite direction. Both numbers are correct about different things: the adapter did improve the chart decision, and it did cost the system 36 responses that no longer fit inside the output budget. A single figure cannot carry both statements, and reporting only the first would have been wrong in a way that no amount of statistical care would have caught.

Two lessons follow, and they generalise beyond this project.

The first concerns the design of structured-generation evaluations. Format conformance and content correctness must be measured on separate axes, because interventions affect them differently and because a single combined number can move by eighty percentage points, or change sign, for reasons that have nothing to do with the quality of the decision. The literature already reports that format restrictions themselves change what models produce [Tam et al. \(2024\)](#) and that language models are sensitive to superficial features of prompt format in ways that shift measured performance substantially [Sclar et al. \(2024\)](#). The present study adds an evaluation-side observation to that picture: the *scoring* pipeline can introduce a comparable artifact, independently of the model.

The second concerns the value of retaining raw outputs. The decomposition in Chapter 7 was possible only because every run stored the complete raw model text alongside the parsed object. Had the pipeline stored only the parsed representation, both artifacts would have been undetectable after the fact and would have propagated into every conclusion in this thesis. Raw-output retention should be treated as a requirement of a structured-generation experiment rather than as a debugging convenience.

The three-level decomposition is itself a post-hoc analysis, introduced after inspecting the level-1 results, and Chapter 7 labels it as such throughout. It is not a replacement for the pre-specified metric; it is a diagnostic that explains it. The honest summary is that the pre-specified metric answered a different question from the one the research question asked, and that the raw outputs allowed the intended question to be answered as well.

8.4 Human-Rated Actionability

The exploratory ratings qualify the automatic findings in two ways. First, retrieval-only (B) has the strongest average human profile: its equal-brief means are 4.48 for chart appropriateness, 4.52 for rationale quality, and 4.52 for overall usefulness. This contrasts with the negative B–A reference-agreement effect at 27B. The two measures answer different questions. Automatic agreement asks whether an output reproduces the frozen label; the rubric asks whether participants consider the recommendation appropriate and useful. The divergence is consistent with the identified pie-to-donut substitutions, for which the retrieval corpus and reference encode different but plausible choices.

Second, fine-tuning does not improve the human ratings in this sample. Conditions C and D remain positively rated, but neither exceeds B on any dimension, and their overall-usefulness means are 4.05 and 4.04. This does not establish that fine-tuning harms perceived quality: only eight briefs were rated, assignments form an incomplete block, and agreement is low on five dimensions. It does show that automatic fine-tuning gains should not be translated directly into an actionability claim. RQ2 therefore has a layered answer: fine-tuning can repair structural compliance for the smallest Qwen model, while retrieval-only provides the strongest descriptive human result at 27B.

8.5 Model Scale and Cross-Family Observations

The Qwen3 sweep now covers three scales with a complete A–D matrix at each, and it supports one clear statement about the relationship between capacity and the value of an intervention: **the interventions in this study substitute for capacity, and the substitution is not uniform across what they supply.**

Output-contract compliance is acquired by the base model between 1.7B and 8B. Below that threshold, fine-tuning is the only thing in this study that supplies it. Above it, fine-tuning cannot add compliance and at 8B actively costs it. Chart agreement behaves differently: fine-tuning improves it at every scale, by about ten points at 1.7B and about four at 8B and 27B, so its value diminishes but does not vanish. Retrieval is the intervention that reverses: mildly helpful at the small scale, significantly harmful at both larger ones.

This pattern is worth stating carefully because it can be misread in two ways. It does not say that fine-tuning is unnecessary at larger scales: +4.38 pp on 274 items is a real and statistically detectable improvement at both 8B (Holm-adjusted $p = 0.034$) and 27B ($p = 9.8 \times 10^{-4}$), and the fine-tuned 27B condition also produces the highest macro-F1 in the study (0.762 for C, 0.789 for D against 0.445 for A). It also does not say that a bigger model is always the better choice: the 27B fine-tuned condition took 8.8 GPU-hours to train and 76–81 seconds per item to run, against 4.9 hours and 22.5 seconds for the 8B profile on the same class of hardware.

The 8B and 27B profiles are also a useful internal control on each other. They agree on the base-model quantities to within one percentage point — 91.61 % against 92.34 % prompt-only agreement, −9.49 against −8.76 pp for B – A, the same donut mechanism at the same 28 items — and they disagree on what the common fine-tuning recipe costs. At 27B the adapter adds agreement and macro-F1 with no loss of validity; at 8B it adds agreement but loses 13.87 pp of contract satisfaction and lowers macro-F1. The two adapters differ in training window (1,024 tokens for 27B against 4,096 for 8B) and the two base models differ in context

architecture, so the study cannot say which of these explains the divergence. It can say that “fine-tuning with this recipe is safe at large scale” is not supported: it was safe at one of the two large scales tested.

The cross-family comparison licenses less. OLMo-2-1B-Instruct and Qwen3-1.7B agree on the chart decision at almost identical rates in the prompt-only condition, so the analytical difficulty of the task is apparently similar for both. They diverge sharply on delivering that decision inside the required contract and within the output budget. Because architecture, tokenizer, pretraining corpus, post-training recipe and native context length all differ simultaneously between the two families — the OLMo checkpoint uses multi-head attention with 16 query and 16 key-value heads and a 4,096-token position limit, the Qwen3 profile grouped-query attention with 16 query and 8 key-value heads and a 40,960-token limit — no single factor can be identified as the cause. The correct formulation is that this is a model-family comparison in which several properties change together.

What the cross-family check does establish is worth having. A conclusion drawn from the Qwen3 sweep alone — “QLoRA fine-tuning on this task improves the chart decision at every scale” — happens to survive in OLMo at the text level (+0.36 to +8.76 pp). A conclusion about usable output does not: fine-tuning takes OLMo from 30 % JSON recovery to 2.9 %. The check therefore did what a cross-family control is for: it separated a finding that generalises from one that does not. OLMo is compared conceptually with the smallest Qwen3 profile only; it is not size-matched to the 8B or 27B profiles and no conclusion treats it as such.

8.6 Error Analysis

8.6.1 The encoding type error

For Qwen3-1.7B in conditions A and B across all three seeds, and in conditions C and D at seed 42, essentially every response fails the runtime contract with the same Pydantic error: `kpi_chart_mapping.0.encoding\-\Input\should\be\valid\dictionary`. The model produces a complete, well-formed JSON object with the six required fields, a plausible task type, a correct chart type in about 87 % of cases, and an `encoding` value that is a single column name rather than a channel mapping.

The failure is systematic rather than random: it occurs on 270–274 of 274 items in the affected conditions and on essentially none in the unaffected ones. It disappeared in the seed-43 and seed-44 adapters trained under the later code state, which raise contract compliance to 84–99 %. And no other model in the study shows it at all — Qwen3-8B, Qwen3-8-27B and OLMo never emit a string-valued encoding. This makes it a property of the interaction between the smallest model and the prompt’s field specification rather than an intermittent generation error or a defect in the prompt as such. A 1.7B model appears to read “encoding” as the name of a thing to be filled in with a field, and the prompt’s type description was not sufficient to override that reading.

The methodological consequence for this thesis is that every chart number for the affected conditions must be read at level 2 or 3. The engineering consequence — that the prompt should specify the encoding object explicitly by example, or that constrained decoding should enforce the type Willard and Louf (2023); Geng et al. (2025) — is noted as an implication, not as an executed experiment.

8.6.2 Truncation at the output budget

The second failure mode is quantitatively larger and is not a modelling failure at all. Of the OLMo prompt-only responses at seed 42 that failed to parse, 189 of 191 do not end with a closing brace. Across all conditions, OLMo truncates on 254–274 of 274 items, Qwen3-8B on 0 (A and B), 36 (C) and 69 (D) items, and the current-state Qwen3-1.7B fine-tuned conditions on 0–38 items. The 27B model never truncates.

Inspection of the stored prefixes shows why this matters. A truncated OLMo fine-tuned response opens with a valid `context_summary`, gives the correct `chart_type`, and populates the encoding channels correctly before stopping mid-array inside `having`. The analytical content is present and correct; the response simply did not fit. Text-level chart agreement recovers 87–95 % from these prefixes in the OLMo A, B and C conditions, against pipeline values of 2.6–24.1 %.

The Qwen3-8B row isolates the mechanism cleanly, and it is the clearest evidence in the study that the ceiling is an experimental-design artifact rather than a model property. The same base model produces **zero** truncations in the prompt-only and RAG conditions and **thirty-six** after fine-tuning, rising to **sixty-nine** when retrieval is added on top. Nothing about the model changed except which target format it had learned and how much input budget the prompt consumed.

The cause is the interaction between three fixed design choices. The generation budget was set to 512 new tokens to bound key-value cache growth during 27B inference. The reference format is long: it carries thirteen encoding keys per mapping in addition to the six top-level fields, and mean raw responses run to roughly 1,500–2,300 characters. And fine-tuning teaches the model to reproduce exactly that long format. The three

choices are individually reasonable and jointly produce a systematic ceiling for any model whose tokenizer is less efficient or whose generation is more verbose.

Reporting “fine-tuning destroys OLMo’s output validity” would be a true statement about the measured pipeline and a misleading statement about the model. The accurate statement is that under a 512-token budget, fine-tuning on this verbose target moves the typical response past the budget, and the pipeline records the consequence as a parse failure.

8.6.3 Chart confusions

Where the chart decision can be read, the errors are concentrated in a small number of interpretable substitutions rather than spread over the vocabulary.

For both larger Qwen3 models the dominant substitution under retrieval is **pie** → **donut**. At 27B it rises from 5 items under prompt-only to 28 of the 36 pie items under retrieval; at 8B it rises from **zero** to 28, and appears again at 29 items in the fine-tuning-plus-retrieval condition. The second family is **stacked_bar** → **bar**, at 4–6 items per condition, which is also why the fine-tuned 27B macro-F1 of 0.762 sits well below its 96.7 % top-1: recall on the eleven **stacked_bar** references is only 0.45. The third is **bar** → **histogram**, at 5–9 items in the base-model conditions of both larger models.

For Qwen3-1.7B at seeds 43 and 44, the fine-tuned condition confuses **stacked_bar** → **bar** (10 items) and **pie** → **bar** (4). The retrieval-plus-adapter condition adds a different and larger error family: **bar** → **stacked_bar** (17), **bar** → **line** (10), **bar** → **scatter** (7) and **bar** → **pie** (5). Retrieval moves this model *away* from the dominant reference class in several directions at once, which is consistent with the D – C degradation reported in Chapter 7.

At 8B, fine-tuning produces a different pattern again. Text-level agreement rises by 4.38 pp, but macro-F1 falls from 0.558 to 0.530, and the per-class values show why: F1 on **line** drops from 1.00 to 0.67 and on **stacked_bar** from 0.59 to 0.15, while **bar** and **pie** hold. The adapter moved the model toward the majority class. An aggregate agreement metric rewards that move on a split where three quarters of the references are bars; a class-balanced metric does not. Neither metric settles whether the resulting recommendations are better.

Two of these substitution families deserve separate comment because the reference is not obviously right. A donut and a pie encode the same part-to-whole relationship; a stacked bar and a grouped set of bars can both express a composition. Whether a substitution is an error depends on the analytical task and audience, and visualization effectiveness is task- and distribution-dependent rather than fixed per chart type [Saket et al. \(2019\)](#); [Kim and Heer \(2018\)](#). The human study shows that retrieval-only remained highly rated on chart appropriateness and usefulness, but its small sample and low agreement do not identify which substitutions individual raters preferred.

8.6.4 Alternatives collapse to empty after adaptation

Every fine-tuned condition of every model produced exactly zero items with more than one distinct recommendation. The contrast with the base-model conditions is total: Qwen3-8B offers alternatives on all 274 items under both A and B, and Qwen3.8-27B on 266 and 267 items.

The adapters learned to emit an empty **alternatives** list because that is what the frozen training targets predominantly contain. This is a faithful reproduction of the supervision, and it is an instance of a general property of supervised adaptation that matters for design tasks: the model learns the *distribution* of the targets, including the places where the targets are impoverished. It is also the clearest example in the study of a behavioural change that the aggregate metrics do not capture. A system that proposes one chart and a system that proposes a ranked set of defensible charts are different products, and for a visualization task where more than one encoding is frequently defensible, the second is arguably the more useful one. Fine-tuning on this dataset converts the second into the first, and every agreement metric in Chapter 7 is blind to that conversion. If the intended behaviour is to offer alternatives, the training targets have to contain them.

8.7 Comparison with Related Work and Methodological Lessons

Earlier visualization-recommendation systems formalise perceptual and task constraints, whereas NL-to-visualization benchmarks concentrate on mapping language and data context to a visualization specification [Mackinlay \(1986\)](#); [Dibia and Demiralp \(2019\)](#); [Luo et al. \(2021b\)](#). This thesis extends that evaluation setting to a structured dashboard recommendation with layout, styling, interaction, and rationale fields. The extension is methodological rather than evidential: because those dashboard-level targets are LLM-generated, agreement with them cannot replace an independent quality assessment. The strongest contribution relative to prior work is therefore the provenance-aware decomposition of format conformance, reference agreement, retrieval grounding, and planned human judgement, not a claim that the generated dashboard designs are objectively superior.

Several problems arose during the project that affected methodological validity rather than only implementation. Each is reported with the reason it mattered, the response, and what remains unresolved. Debugging history that had no methodological consequence is not included.

Source-task mismatch between candidate datasets and the target task. Several public visualization resources were examined during dataset construction. Datasets whose primary task depends on a rendered chart image do not expose the query, aggregation, field roles or constraint scope that the target task requires, so they could not supply the analytical core of a brief. nvBench was selected because it provides source-grounded natural-language, schema, query and visualization information aligned with the analytical component of the task Luo et al. (2021b). **Residual limitation:** nvBench is not a dashboard-design corpus, so the dashboard-level fields had to come from somewhere else, which is the origin of the next problem.

Generated supervision for the dashboard-level fields. Six fields — users, context summary, layout, styling, interactions and rationales — are not present in the source corpus and were produced by an LLM under a constrained enrichment procedure. This matters because a model fine-tuned on generated targets learns the conventions of the generator, and agreement with those targets is not independent evidence of quality. The response was to keep the source-grounded analytical fields immutable during enrichment, to record field-level provenance, to validate deterministically, to mark generated records `not_gold`, and to keep the held-out test byte-identical across the freeze steps. **Residual limitation:** generated annotation cannot substitute for expert gold. The evaluation design compensates through an independent effectiveness layer and a human layer, and neither has been executed, so the compensation is currently theoretical.

A strict validator narrower than the training target. The strict response schema forbids extra keys in the encoding object, while the reference encodings carry thirteen additional source-grounded keys. The metric therefore has a construction ceiling of 0 % for any faithful system. This was detected by applying the validator to the 274 reference records during the preparation of Chapter 6 rather than by inspecting the code, and it invalidates any interpretation of strict schema validity as a quality ranking. The response was to verify the ceiling programmatically, publish the check as an artifact, and exclude the metric from every comparative statement. **Residual limitation:** the metric remains in the reported tables for completeness and is a standing invitation to misread; it should be either aligned with the reference contract or removed in future work.

A fixed output budget below the length of the target format. Described in Section 8.3.2. The response was to detect truncation explicitly, quantify it per condition, and re-score the chart decision at a level that survives it. **Residual limitation:** the affected runs were not repeated with a larger budget, so the study cannot report what OLMo or the fine-tuned Qwen3-8B condition would achieve without the ceiling. Every OLMo conclusion and the Qwen3-8B C and D conclusions are conditional on the 512-token budget.

Duplicate concurrent writes to the same output files. A first attempt at the prompt-only and RAG conditions for Qwen3-8B was started twice, and two processes wrote into the same prediction files, producing 548 lines for 274 unique identifiers with conflicting generations. Metrics over such a file mix two executions. The response had three parts: the defect was detected by an identifier-uniqueness check rather than by inspection of the numbers; the affected exports were quarantined and excluded; and both conditions were re-inferred from a single process, which is what Chapter 7 reports. The uniqueness check is now part of the standing completeness criterion for every run. **Residual limitation:** none for the reported results — the Qwen3-8B row is now a complete and clean A–D matrix — but the episode is the reason the 8B A and B runs carry a later git commit than the 8B C and D runs, which makes the C – A contrast at that scale code-state mixed.

Heterogeneous code states across runs. The thirty-two runs were executed over several weeks across four compute environments, and their manifests record different git commits; five of the eight model-by-seed blocks contain at least one run whose commit was not recorded at all. The response was to make code-state comparability an explicit, machine-computed label attached to every reported contrast, at both block and pair level, rather than an unstated assumption. This is what makes it possible to say that the two B – A contrasts carrying the main retrieval finding, and the D – C contrasts at 8B, 27B and OLMo seeds 42 and 43, are code-state homogeneous. **Residual limitation:** nineteen of forty contrasts still involve a run with an unrecorded commit, and this cannot be repaired by analysis. Only re-running the affected conditions under one pinned commit would repair it.

Hardware-determined training precision. The precision helper resolves `precision: auto` against the weakest available device, so two of the three Qwen3-1.7B adapters were trained in fp16 on a Tesla T4 while the rest were trained in bf16. The response was to record the effective precision in the training metadata and to report the three seeds individually rather than as a pooled mean. **Residual limitation:** the three Qwen3-1.7B seeds are not three draws from one procedure, which weakens them as a variance estimate.

Knowledge-base identity across platforms. The knowledge-base digest recorded by the runs differs from the digest of the local repository copy. The cause was determined rather than assumed: converting the

local file from CRLF to LF reproduces the recorded digest exactly, so the retrieved content is identical and only the line terminator differs. **Residual limitation:** a digest that depends on checkout platform is a weak identity. Normalising line endings before hashing would make the artifact identity platform-independent.

8.8 Practical Implications

The implications below follow from the evidence and are stated at the strength it supports.

Below about 8B parameters, fix the output contract before anything else; above it, do not pay for what you already have. The largest single gain observed anywhere in this study was contract compliance at 1.7B, and it was worth eighty percentage points of measured accuracy while the underlying chart decision changed by ten. At 8B and 27B the base model already satisfies the contract, and fine-tuning buys about four points of agreement while risking output validity. Where the contract is the problem, constrained decoding is the obvious alternative to training and is available in the repository but was not enabled for the reported runs Willard and Louf (2023); Geng et al. (2025).

Set the generation budget from the target length, not from memory convenience. A 512-token ceiling against a target that routinely needs more produced the most severe apparent failures in the study, and the Qwen3-8B block shows that they were not model failures: the same base model truncated zero responses without an adapter and sixty-nine with an adapter and retrieval.

Check that the knowledge base and the reference agree before interpreting a retrieval result. The clearest negative retrieval effect in this study was produced by two different models correctly following a single guideline sentence that the reference contradicts. A retrieval corpus is an opinion about design, and if it disagrees with the evaluation reference the experiment measures the disagreement, not the method.

Do not assume that retrieval and fine-tuning compose. In eleven of twelve blocks, adding retrieval to a fine-tuned adapter lowered chart agreement, and at 8B it also cost twelve points of output validity. If both are used, the combination needs its own evaluation rather than an assumption of additivity.

Watch what adaptation removes, not only what it adds. Fine-tuning eliminated the alternatives list in every condition of every model, and no metric in the pre-specified set would have shown it.

Report structured-generation results at more than one level of strictness, and keep the raw outputs. Both recommendations follow directly from Section 8.2.

The human ratings support statements about perceived quality of the textual recommendations in this sample, not a recommendation about which rendered dashboard a practitioner should adopt. That broader claim requires expert review and task-based evaluation of implemented dashboards.

8.9 Threats to Validity

8.9.1 Construct Validity

The reference is not ground truth. Every chart number in this thesis is agreement with the project’s frozen reference. The analytical fields of that reference are source-grounded, but the dashboard-level fields are LLM-generated, and even for the chart type the reference records one defensible choice where several may exist. The donut-versus-pie case, now observed at two model scales, and the stacked-bar-versus-bar case are concrete instances where the reference decides a question that the visualization literature treats as task-dependent Saket et al. (2019); Kim and Heer (2018).

No independent effectiveness criterion was applied. The layer that would score a predicted chart against the set of charts shown effective for the corresponding task in published human-subject studies is specified but not implemented, and every per-item record carries `ll_covered=\not_applicable`.

No human judgement of design quality exists. Layout, styling, accessibility, interaction design, rationale quality and overall usefulness are unmeasured. The automatic metrics do not approximate them, and no claim about them is made.

Aggregate agreement hides a behavioural change. The collapse of the alternatives list after adaptation is invisible to every metric reported in Chapter 7 and was detected only by counting distinct recommendations per item. Other behavioural changes of the same kind may be present and unmeasured.

Grounding is a lexical proxy. The grounding numbers measure content-word overlap between rationale claims and retrieved passages. Overlap is not entailment. Additionally, only six of the sixteen RAG conditions have claim coverage above 70 %, so ten of the reported grounding rates describe a small and non-representative subset of items.

The clarification measure is a regular expression. The missing-information clarification rate is detected by matching phrases such as *clarify*, *insufficient* and *please specify*. It will miss an implicit request and can fire on an unrelated use of a matched word. The near-zero values are consistent across every condition, which makes a purely detector-side explanation unlikely, but the measure is a diagnostic and not a semantic judgement.

8.9.2 Internal Validity

Code-state confounding. Seven of the forty contrasts are code-state homogeneous, fourteen are mixed and nineteen involve a run with an unrecorded commit. A significant contrast from a non-homogeneous pair establishes that the runs differ; it does not establish that the method is the only reason. The contrasts most central to the conclusions are in better shape than the average: B – A at 8B and 27B, and D – C at 8B, 27B and OLMo seeds 42 and 43, are all homogeneous. The C – A contrasts, on which the fine-tuning conclusion rests, are not homogeneous in any block.

Differing training sequence length. Two adapters were trained with a maximum sequence length of 4,096 tokens and two with 1,024, despite sharing every other hyperparameter. Truncating longer training targets changes what the adapter learns. Qwen3-8B and OLMo trained at 4,096, Qwen3-1.7B and Qwen3.8-27B at 1,024, so the 8B-versus-27B comparison of what fine-tuning costs is partly confounded with training window.

Differing inference hardware. Runs executed on five GPU types. This does not affect correctness metrics but makes latency non-comparable across conditions, as Chapter 7 states. The Qwen3-8B block is the one place where all four conditions ran on the same device.

Adapter provenance in two runs. Six of the eight D runs record their source C run identifier explicitly. Two record only the adapter path, which points at the correct condition-C directory for the same model and seed, together with a matching training-configuration hash. This is strong but indirect evidence for those two runs.

8.9.3 External Validity

One task, one output contract, one source corpus. The held-out split contains 274 items drawn from one source corpus, with a chart distribution dominated by `bar` (208 of 274) and only two `scatter` items. Generalisation beyond this distribution is not supported. Task families that were introduced only in the generated training material — distribution, ranking, deviation, flow — do not appear in the test at all and are therefore untested.

Structured recommendations, not rendered dashboards. The system predicts a design object. It does not render it, and no user completed a task with it. Claims about decision quality, time on task, adoption or business value are outside the evidence.

One adaptation configuration. The evidence concerns one QLoRA setting: rank 16, $\alpha = 32$, dropout 0.05, all linear target modules, NF4 4-bit quantization, three epochs, learning rate 2×10^{-4} , cosine schedule. It is not evidence about QLoRA in general, and it is not a comparison between adaptation algorithms.

One retriever and one small corpus. TF-IDF over 41 chunks from three project-authored documents. A different retriever or a larger corpus could produce different retrieval effects, and the donut mechanism shows how directly a single sentence in the corpus can move a metric by nine percentage points.

Four models, asymmetric seed coverage. Two models have three seeds and two have one. The scale trend within Qwen3 rests on single runs at 8B and 27B.

8.9.4 Statistical Conclusion Validity

Pairing is respected; independence across seeds is not assumed. Comparisons are item-level and paired, Holm correction is applied within each block of pairwise tests, and bootstrap intervals resample items with pairing preserved. Seeds are treated as repeated realisations and are never pooled into a sample of independent observations.

No cross-block correction. Holm is applied within a block — one model, one seed, one outcome. Across the whole thesis many blocks are tested, and no global correction is applied. Individual contrasts should therefore be read with their effect sizes and intervals rather than as isolated significance decisions.

Small discordant counts. Several contrasts rest on very few discordant pairs (for example two for the 27B D – C comparison). The exact binomial form of McNemar is used precisely for this reason, but a non-significant result from two discordant pairs carries almost no information and is not evidence of equivalence.

Statistical significance is not practical importance. Several significant contrasts are four or five percentage points on 274 items. Whether such a difference matters for a design assistant is a question the statistics cannot answer.

Variance is not estimated where it cannot be. No standard deviation, confidence interval or stability claim is reported for a single-seed condition, and the reporting layer refuses to compute seed-level intervals from three values.

8.9.5 Human-Evaluation Validity

The human study is limited to eight briefs and four outputs per brief from one model and one seed. Recruitment is an open convenience sample, identities are not independently verified, expertise is optional self-report,

and the bilingual rubric has not undergone an independent equivalence study. The final export contains 81 pages: every output has two to four ratings, but the three-page target is not universal, and “Cannot judge” responses create 11 missing dimension judgements. The design evaluates textual recommendations rather than rendered dashboards and cannot support expert-validity, population-level effects, or task-performance claims. Method blinding, score-blind allocation, equal brief weighting, per-dimension missingness, separate agreement estimates, and exclusion of older protocol versions reduce bias but do not remove these limits. Low ordinal agreement on five dimensions makes small differences between methods unstable.

8.9.6 Reproducibility Limitations

The analysis is fully reproducible; the runs are not fully reproducible. Every number in Chapter 7 can be regenerated from the stored artifacts by one script with no GPU and no network access. The runs themselves depend on upstream model repositories, and the model revision is pinned for two of the four profiles and null for the other two. Sampling is enabled at temperature 0.1 and the runs used five GPU types, so recording a seed documents the configuration without guaranteeing bitwise regeneration.

Dataset and knowledge-base inputs are anchored. The train, validation and test digests are identical in all thirty-two run manifests and were recomputed from the local files. The knowledge-base content is identical across all sixteen RAG runs, subject to the line-ending caveat above.

Engineering correctness is not scientific validity. The repository’s tests establish that the implementation behaves as intended; the frozen-data validation establishes data integrity; the automatic metrics establish task performance against a constructed reference; the multi-seed analysis establishes stochastic stability where three seeds exist; the statistical analysis quantifies uncertainty about observed differences. None of these establishes that a generated dashboard recommendation is a good design. That evidence layer is missing and is named as missing.

Chapter 9

Conclusion and Future Work

This chapter consolidates the findings, answers the three research questions, states the contributions and limitations, and identifies the most direct extensions of the work.

9.1 Overall Findings

The thesis asked whether retrieval-augmented generation, QLoRA fine-tuning, or their combination improves a language model’s ability to turn a short dashboard brief into a structured, machine-readable dashboard-design recommendation. To answer that, it built a frozen and lineage-annotated dataset for a task that no public benchmark covers, implemented four system conditions that share a prompt, a parser, an evaluation path and a held-out split, executed thirty-two complete runs across four base models spanning roughly 1.49B to 27B parameters and two model families, and analysed the outcome with paired item-level statistics computed from stored per-item records.

The design held constant everything that was not the intervention. The same 274 held-out items, verified by digest in all thirty-two run manifests, were used for every condition. Every model has a complete A–D matrix at the primary seed, so the four-way method contrast is available at all four scales. Condition D reused the adapter produced by condition C for the same model, dataset, training configuration and seed rather than training a second one, so that the incremental effect of retrieval after adaptation could be isolated from a second stochastic training trajectory.

Output-contract compliance is acquired by the base model between 1.7B and 8B parameters, and below that threshold it dominates everything else. Qwen3-8B and Qwen3.8-27B satisfy the contract on all 274 items in both base-model conditions. Qwen3-1.7B satisfies it on under 1.5 % of items, failing on one recurring type error, and OLMo-2-1B-Instruct recovers a complete object on 30–33 % of items because almost every response exceeds the 512-token generation budget before it can close.

9.2 Answers to the Research Questions

RQ1 (Accuracy) — How accurately can the system map dashboard tasks/KPIs to suitable chart types (Top-1/Top-3 where reference labels exist) across prompt-only, RAG, fine-tuning, and fine-tuning+RAG? For Top-1 chart agreement, fine-tuning improves all eight comparable model–seed blocks, whereas retrieval helps the small Qwen model, has little effect on OLMo’s chart choice, and harms the two larger Qwen baselines. Retrieval after fine-tuning is detrimental in eleven of twelve contrasts. Top-3 accuracy is not reported because the reference data do not contain an auditable ranked alternative-label set.

RQ2 (Actionability & Structure) — Do RAG and/or fine-tuning improve actionability and structure, measured via schema compliance and human rubric ratings? The larger Qwen baselines are structurally reliable, while Qwen3-1.7B and OLMo expose different serialization and truncation failures. Fine-tuning repairs the small Qwen model but does not improve strict validity uniformly. In the exploratory eight-brief human study at 27B, retrieval-only has the highest equal-brief means for chart appropriateness (4.48), rationale quality (4.52), and overall usefulness (4.52); prompt-only is marginally higher for layout and interaction. Fine-tuned conditions do not improve the human scores. These comparisons are descriptive because coverage is incomplete and agreement is low on five dimensions.

RQ3 (Rationale & Robustness) — Do RAG and/or fine-tuning improve heuristic-aligned rationales and robustness under paraphrased or partially specified briefs? The strongest Qwen baselines are highly stable under controlled paraphrases, while the fine-tuned small-model rows are more variable. Automatic lexical grounding remains only a proxy. Human rationale ratings favour retrieval-only descriptively (4.52), followed by prompt-only (4.29), fine-tuning with retrieval (4.13), and fine-tuning alone (3.76); low agreement ($\alpha = 0.209$) prevents a confirmatory ordering.

Model scale, model family, seed variation, and failure modes support these answers but are not separate research questions.

9.3 Contributions

1. **A frozen, lineage-annotated dataset** for the dashboard-brief-to-recommendation task, with field-level evidence classes, generated material labelled as generated, a byte-identical held-out test across freeze steps, and published manifests and hashes. Contribution: the dataset exists, is auditable and is reproducible from its manifest. Not claimed: that its dashboard-level fields are expert gold.
2. **A configuration-driven implementation of four controlled conditions** with enforced adapter provenance, verified for all eight condition-D runs. Contribution: the C-to-D pairing is documented and checked. Not claimed: that all runs share one code state — they do not, and this is reported per contrast.
3. **A layered evaluation protocol** that separates parse success, contract validity, completeness, chart agreement, robustness and grounding, and that states the evidence class of each. Contribution: the separation is implemented and every reported number carries its class.
4. **A complete A/B/C/D comparison at four model scales across two families**, with paired item-level statistics, Holm-corrected exact tests and bootstrap intervals computed from stored per-item records, and with code-state comparability reported per contrast rather than assumed.
5. **A three-level decomposition of the headline chart metric** that separates chart-decision failures from output-contract failures and generation-budget failures, and that materially changes the interpretation of every method effect in the study, including one sign reversal at 8B. Contribution: the decomposition is reproducible from the artifacts by one script.
6. **A traceable and replicated mechanism for a negative retrieval result.** The -9 point retrieval effect at two independent scales is attributed to one identified sentence in the retrieval corpus, with the substitution counted item by item. Contribution: a negative result explained rather than merely reported.
7. **Two documented measurement artifacts** that would have propagated silently into the conclusions: a strict validator with a construction ceiling of 0 % verified against the 274 reference records, and a generation budget below the length of the training target, isolated cleanly at 8B where the same base model truncates zero responses without an adapter and sixty-nine with an adapter and retrieval.
8. **A completed exploratory human evaluation** with method blinding, score-blind assignment, bilingual rubrics, explicit missingness, equal-brief aggregation, and dimension-specific agreement analysis. The frozen export contains 81 pages for 32 outputs.

9.4 Limitations in Brief

The following are stated as explicitly as the findings above, because a reader is entitled to know the boundary.

The human results describe **perceived quality in a small convenience sample**, not expert-validated design quality or population-level usefulness. Only eight briefs from one model and seed were rated; outputs received two to four pages, and agreement is low on five dimensions.

It makes **no claim that any system selects charts that are effective for real users**. Every chart number is agreement with the project’s frozen reference. The independent effectiveness layer, which would score predictions against sets of charts shown effective in published human-subject studies, is specified and not implemented.

It makes **no claim about retrieval quality**. Retrieval relevance was not measured, because no independent relevance judgements exist. The grounding numbers measure lexical overlap between rationales and retrieved passages, and ten of the sixteen retrieval conditions have claim coverage too low to interpret.

It makes **no claim that QLoRA is superior to other adaptation algorithms**. One QLoRA configuration was used; no algorithm comparison was run.

It makes **no strong causal claim for the fine-tuning contrasts**. Seven of forty contrasts are code-state homogeneous, and none of them is a C – A contrast. Where a pair is not homogeneous, a significant result shows that two runs differ, not that the method is the sole reason.

It makes **no claim about variance at 8B or 27B**, where only one seed was executed.

Finally, it makes **no claim about behaviour outside the observed distribution**: a held-out split of 274 items dominated by comparison/bar tasks, two perturbation families, one guideline corpus of 41 chunks, and one output contract.

9.5 Future Work

The next steps follow from the gaps rather than from a wish list, and they are ordered by how much they would change what the thesis can claim.

Confirm the human findings at larger scale. A preregistered study should extend coverage beyond eight briefs, recruit verified visualization practitioners as well as non-experts, balance assignments, and evaluate rendered dashboards in task-based sessions. This would test whether the descriptive retrieval advantage persists and whether chart substitutions, lost alternatives, and layout choices affect completion time, accuracy, or decision confidence.

Implement the independent chart-effectiveness scorer. The gold table exists in the repository and the design is documented. Scoring predictions as set membership against charts shown effective for the corresponding task and data shape, and reporting covered accuracy and coverage rate separately, would replace agreement with the project’s own reference by a criterion the project did not construct. It would also settle the donut question independently of human raters, if the effectiveness table covers those items.

Re-run inference with a generation budget matched to the target length. The 512-token ceiling produced the most severe apparent failures in the study, and the 8B block shows unambiguously that they are an artifact of that ceiling. Repeating the OLMo conditions and the fine-tuned Qwen3 conditions with a budget sized from the actual distribution of target lengths would establish what those models achieve when the ceiling is not binding. This requires no retraining.

Re-run the affected conditions under one pinned code state. Nineteen of the forty contrasts involve a run whose git commit was not recorded. Re-executing inference for those conditions from one commit, with pinned model revisions, would convert most of the reported contrasts from descriptive to controlled — in particular the C – A contrasts, which carry the fine-tuning conclusion and are homogeneous nowhere. Only inference is required; the existing adapters can be reused, so the cost is a fraction of the original experiment.

Align or retire the strict response schema. The strict contract should either accept the source-grounded encoding keys that the reference carries, in which case it becomes an informative measure, or be removed from the reported metric set.

Add alternatives to the training targets. The alternatives list collapsed to empty because the targets contain empty lists. Supplying defensible alternatives in the supervision would move the fine-tuned systems closer to the behaviour a design assistant should have, and would restore a capability that the base models at 8B and 27B already exhibit and that adaptation currently destroys.

Extend the seed coverage at the larger scales. Three seeds at 8B and 27B would let the scale trend rest on the same evidential footing as the small-model comparison, and would show whether the divergence between the 8B and 27B fine-tuning outcomes is stable or a single-run accident.

Two further directions are worth naming but are further from the current evidence. A matched comparison between QLoRA and one alternative adaptation method — plain LoRA is the closest, since it isolates quantization — would answer whether the choice of adaptation algorithm matters for this task; the trainer already supports the alternatives. And a comparison between constrained decoding and fine-tuning as routes to output-contract compliance would test directly whether the largest measured effect in this study can be obtained without any training at all.

The study shows that structured dashboard recommendation must be evaluated as more than a single accuracy number. Fine-tuning can repair format compliance without proportionally changing the chart decision; retrieval can reduce agreement with a frozen label because its design guidance differs from that label; and a small human study can still rate the retrieved recommendations highly. The combined evidence therefore favours a layered conclusion: report structural validity, reference agreement, robustness, provenance, and human judgement separately, and match each claim to the evidence that actually supports it.

Bibliography

- Allen Institute for AI. OLMo-2-0425-1B-Instruct model card. Hugging Face model repository, 2025. URL <https://huggingface.co/allenai/OLMo-2-0425-1B-Instruct>. Revision 48d788eca847d4d7548f375ad03d3c9312f6139e; Apache-2.0; accessed 2026-09-10.
- Benjamin Bach, Euan Freeman, Alfie Abdul-Rahman, Cagatay Turkay, Saiful Khan, Yulei Fan, and Min Chen. Dashboard design patterns. *IEEE Transactions on Visualization and Computer Graphics*, 29(1):342–352, 2023. doi: 10.1109/TVCG.2022.3209448.
- Elad Ben Zaken, Yoav Goldberg, and Shauli Ravfogel. BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 1–9. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.acl-short.1. URL <https://aclanthology.org/2022.acl-short.1/>.
- Matthew Brehmer and Tamara Munzner. A multi-level typology of abstract visualization tasks. *IEEE Transactions on Visualization and Computer Graphics*, 19(12):2376–2385, 2013. doi: 10.1109/TVCG.2013.124.
- Nghia Bui, Guergana Savova, and Lijing Wang. Assessing the macro and micro effects of random seeds on fine-tuning large language models, 2025. URL <https://arxiv.org/abs/2503.07329>. Accepted at IJCNLP 2025.
- William S. Cleveland and Robert McGill. Graphical perception: Theory, experimentation, and application to the development of graphical methods. *Journal of the American Statistical Association*, 79(387):531–554, 1984. doi: 10.1080/01621459.1984.10478080.
- William G. Cochran. The comparison of percentages in matched samples. *Biometrika*, 37(3-4):256–266, 1950. doi: 10.1093/biomet/37.3-4.256.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, volume 36, pages 10088–10115, 2023. doi: 10.52202/075280-0441. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/1feb87871436031bdc0f2beaa62a049b-Abstract-Conference.html.
- Victor Dibia and Çagatay Demiralp. Data2vis: Automatic generation of data visualizations using sequence-to-sequence recurrent neural networks. *IEEE Computer Graphics and Applications*, 39(5):33–46, 2019. doi: 10.1109/MCG.2019.2924636.
- Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. doi: 10.1214/aos/1176344552.
- Stephen Few. *Information Dashboard Design: The Effective Visual Communication of Data*. O’Reilly Media, 2006. ISBN 9780596100162. URL <https://www.oreilly.com/lpt/pr/1512>.
- Milton Friedman. The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *Journal of the American Statistical Association*, 32(200):675–701, 1937. doi: 10.1080/01621459.1937.10503522.
- Siwei Fu, Kai Xiong, Xiaodong Ge, Siliang Tang, Wei Chen, and Yingcai Wu. Quda: Natural language queries for visual data analytics, 2020. URL <https://arxiv.org/abs/2005.03257>.
- Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021. doi: 10.1145/3458723. URL <https://www.microsoft.com/en-us/research/publication/datasheets-for-datasets/>.
- Saibo Geng, Hudson Cooper, Michał Moskal, Samuel Jenkins, Julian Berman, Nathan Ranchin, Robert West, Eric Horvitz, and Harsha Nori. JSONSchemaBench: A rigorous benchmark of structured outputs for language models, 2025. URL <https://arxiv.org/abs/2501.10868>.
- Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979. URL <https://www.jstor.org/stable/4615733>.
- Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97, pages 2790–2799. PMLR, 2019. URL <https://proceedings.mlr.press/v97/houlsby19a.html>.

- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.
- Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023. doi: 10.1145/3571730.
- JSON Schema. Json schema draft 2020-12. JSON Schema specification, 2022. URL <https://json-schema.org/draft/2020-12>.
- Kushal Kafle, Brian Price, Scott Cohen, and Christopher Kanan. Dvqa: Understanding data visualizations via question answering. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 5648–5656, 2018. URL https://openaccess.thecvf.com/content_cvpr_2018/html/Kafle_DVQA_Understanding_Data_CVPR_2018_paper.html.
- Samira Ebrahimi Kahou, Vincent Michalski, Adam Atkinson, Akos Kadar, Adam Trischler, and Yoshua Bengio. Figureqa: An annotated figure dataset for visual reasoning, 2018. URL <https://arxiv.org/abs/1710.07300>.
- Damjan Kalajdzievski. A rank stabilization scaling factor for fine-tuning with LoRA, 2023. URL <https://arxiv.org/abs/2312.03732>.
- Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.emnlp-main.550. URL <https://aclanthology.org/2020.emnlp-main.550/>.
- Timothy Kassis, Vinayak Agarwal, Yuhuan He, Darshil Patel, and Aubrey M. Brueckner. Scientific agent skills: A library of procedural knowledge for research agents, 2026. URL <https://arxiv.org/abs/2609.00065>. Procedural resource used during manuscript preparation; not evidence for empirical thesis claims.
- Younghoon Kim and Jeffrey Heer. Assessing effects of task and data distribution on the effectiveness of visual encodings. *Computer Graphics Forum*, 37(3):157–167, 2018. doi: 10.1111/cgf.13409.
- Klaus Krippendorff. *Content Analysis: An Introduction to Its Methodology*. SAGE Publications, Thousand Oaks, CA, 4 edition, 2018. ISBN 9781506395661.
- Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, Yuling Gu, Saumya Malik, Victoria Graf, Jena D. Hwang, Jiangjiang Yang, Ronan Le Bras, Øyvind Tafjord, Chris Wilhelm, Luca Soldaini, Noah A. Smith, Yizhong Wang, Pradeep Dasigi, and Hannaneh Hajishirzi. Tulu 3: Pushing frontiers in open language model post-training, 2025. URL <https://arxiv.org/abs/2411.15124>.
- Katherine Lee, Daphne Ippolito, Andrew Nystrom, Chiyuan Zhang, Douglas Eck, Chris Callison-Burch, and Nicholas Carlini. Deduplicating training data makes language models better. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8424–8445. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.acl-long.577. URL <https://aclanthology.org/2022.acl-long.577/>.
- Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3045–3059. Association for Computational Linguistics, 2021. doi: 10.18653/v1/2021.emnlp-main.243. URL <https://aclanthology.org/2021.emnlp-main.243/>.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020. URL <https://arxiv.org/abs/2005.11401>.
- Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4582–4597. Association for Computational Linguistics, 2021. doi: 10.18653/v1/2021.acl-long.353. URL <https://aclanthology.org/2021.acl-long.353/>.
- Haokun Liu, Derek Tam, Mohammed Muqeeth, Jay Mohta, Tenghao Huang, Mohit Bansal, and Colin A. Raffel. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. In *Advances in Neural Information Processing Systems*, volume 35, pages 1950–1965, 2022. doi: 10.52202/068431-0142. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/0cde695b83bd186c1fd456302888454c-Abstract-Conference.html.

- Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. DoRA: Weight-decomposed low-rank adaptation. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pages 32100–32121. PMLR, 2024. URL <https://proceedings.mlr.press/v235/liu24bn.html>.
- Tianqi Luo, Chuhan Huang, Leixian Shen, Boyan Li, Shuyu Shen, Wei Zeng, Nan Tang, and Yuyu Luo. nvbench 2.0: Resolving ambiguity in text-to-visualization through stepwise reasoning. In *Advances in Neural Information Processing Systems*, volume 38, pages 138749–138786, 2025. doi: 10.52202/085713-4172. URL https://papers.nips.cc/paper_files/paper/2025/hash/b5f985cdc1796defe83a6b2f84fe2ab6-Abstract-Datasets_and_Benchmarks_Track.html.
- Yuyu Luo, Jiawei Tang, and Guoliang Li. nvbench: A large-scale synthesized dataset for cross-domain natural language to visualization task, 2021a. URL <https://arxiv.org/abs/2112.12926>.
- Yuyu Luo, Nan Tang, Guoliang Li, Chengliang Chai, Wenbo Li, and Xuedi Qin. Synthesizing natural language to visualization (nl2vis) benchmarks from nl2sql benchmarks. In *Proceedings of the 2021 International Conference on Management of Data*, pages 1235–1247. Association for Computing Machinery, 2021b. doi: 10.1145/3448016.3457261. URL <https://doi.org/10.1145/3448016.3457261>.
- Yuyu Luo, Nan Tang, Guoliang Li, Jiawei Tang, Chengliang Chai, and Xuedi Qin. Natural language to visualization by neural machine translation. *IEEE Transactions on Visualization and Computer Graphics*, 28(1):217–226, 2022. doi: 10.1109/TVCG.2021.3114848.
- Jock Mackinlay. Automating the design of graphical presentations of relational information. *ACM Transactions on Graphics*, 5(2):110–141, 1986. doi: 10.1145/22949.22950.
- Jock D. Mackinlay, Pat Hanrahan, and Chris Stolte. Show me: Automatic presentation for visual analysis. *IEEE Transactions on Visualization and Computer Graphics*, 13(6):1137–1144, 2007. doi: 10.1109/TVCG.2007.70594. URL <https://doi.org/10.1109/TVCG.2007.70594>.
- Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. Chartqa: A benchmark for question answering about charts with visual and logical reasoning. In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 2263–2279. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.findings-acl.177. URL <https://aclanthology.org/2022.findings-acl.177/>.
- Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996.
- Nitesh Methani, Pritha Ganguly, Mitesh M. Khapra, and Pratyush Kumar. Plotqa: Reasoning over scientific plots. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 1527–1536, 2020. URL https://openaccess.thecvf.com/content_WACV_2020/html/Methani_PlotQA_Reasoning_over_Scientific_Plots_WACV_2020_paper.html.
- Rishab Mitra, Arpit Narechania, Alex Endert, and John Stasko. Facilitating conversational interaction in natural language interfaces for visualization. In *2022 IEEE Visualization and Visual Analytics (VIS)*, pages 6–10, 2022. doi: 10.1109/VIS54862.2022.00010. URL <https://doi.org/10.1109/VIS54862.2022.00010>.
- Dominik Moritz, Chenglong Wang, Greg L. Nelson, Halden Lin, Adam M. Smith, Bill Howe, and Jeffrey Heer. Formalizing visualization design knowledge as constraints: Actionable and extensible models in draco. *IEEE Transactions on Visualization and Computer Graphics*, 25(1):438–448, 2019. doi: 10.1109/TVCG.2018.2865240.
- Tamara Munzner. *Visualization Analysis and Design*. CRC Press, 2014. ISBN 9781466508934. doi: 10.1201/b17511.
- Arpit Narechania, Arjun Srinivasan, and John Stasko. NL4dv: A toolkit for generating analytic specifications for data visualization from natural language queries. *IEEE Transactions on Visualization and Computer Graphics*, 27(2):369–379, 2021. doi: 10.1109/TVCG.2020.3030378.
- OLMo Team, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, Nathan Lambert, Dustin Schwenk, Oyvind Tafjord, Taira Anderson, David Atkinson, Faeze Brahman, Christopher Clark, Pradeep Dasigi, Nouha Dziri, Allyson Ettinger, Michal Guerquin, David Heineman, Hamish Ivison, Pang Wei Koh, Jiacheng Liu, Saumya Malik, William Merrill, Lester James V. Miranda, Jacob Morrison, Tyler Murray, Crystal Nam, Jake Poznanski, Valentina Pyatkin, Aman Rangapur, Michael Schmitz, Sam Skjonsberg, David Wadden, Christopher Wilhelm, Michael Wilson, Luke Zettlemoyer, Ali Farhadi, Noah A. Smith, and Hannaneh Hajishirzi. 2 OLMo 2 furious, 2025. URL <https://arxiv.org/abs/2501.00656>.
- Fatemeh Pesaran Zadeh, Juyeon Kim, Jin-Hwa Kim, and Gunhee Kim. Text2chart31: Instruction tuning for chart generation with automatic feedback. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 11459–11480. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.emnlp-main.640. URL <https://aclanthology.org/2024.emnlp-main.640/>.

- Mahima Pushkarna, Andrew Zaldivar, and Oddur Kjartansson. Data cards: Purposeful and transparent dataset documentation for responsible ai. In *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency*, pages 1776–1826. Association for Computing Machinery, 2022. doi: 10.1145/3531146.3533231.
- Qwen Team. Qwen3.8-Max: A new bar for coding and cowork. Qwen blog, August 2026a. URL <https://qwen.ai/blog?id=qwen3.8>. Citation given on the Qwen3.8-72B model card; accessed 2026-09-10.
- Qwen Team. Qwen3.8-72B model card. Hugging Face model repository, 2026b. URL <https://huggingface.co/Qwen/Qwen3.8-72B>. Revision 1d4bf0f2ff6012fd82039f2fa52739d0dd7c60c0; Apache-2.0; accessed 2026-09-10.
- Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. Beyond accuracy: Behavioral testing of nlp models with checklist. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4902–4912. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.acl-main.442. URL <https://aclanthology.org/2020.acl-main.442/>.
- Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. doi: 10.1561/15000000019.
- Subham Sah, Rishab Mitra, Arpit Narechania, Alex Endert, John Stasko, and Wenwen Dou. Generating analytic specifications for data visualization from natural language queries using large language models, 2024. URL <https://arxiv.org/abs/2408.13391>.
- Bahador Saket, Alex Endert, and Cagatay Demiralp. Task-based effectiveness of basic visualizations. *IEEE Transactions on Visualization and Computer Graphics*, 25(7):2505–2512, 2019. doi: 10.1109/TVCG.2018.2829750.
- Alper Sarikaya, Michael Correll, Lyn Bartram, Melanie Tory, and Danyel Fisher. What do we talk about when we talk about dashboards? *IEEE Transactions on Visualization and Computer Graphics*, 25(1):682–692, 2019. doi: 10.1109/TVCG.2018.2864903. URL <https://doi.org/10.1109/TVCG.2018.2864903>.
- Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models’ sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2310.11324>.
- Sungbok Shin, Sanghyun Hong, and Niklas Elmquist. Visualizationary: Automating design feedback for visualization designers using large language models. *IEEE Transactions on Visualization and Computer Graphics*, 31(10):8796–8813, 2025. doi: 10.1109/TVCG.2025.3579700. URL <https://doi.org/10.1109/TVCG.2025.3579700>.
- Luke S. Snyder, Chenglong Wang, and Steven M. Drucker. Challenges & opportunities with llm-assisted visualization retargeting. In *2025 IEEE Visualization and Visual Analytics (VIS)*, pages 141–145, 2025. doi: 10.1109/VIS60296.2025.00034. URL <https://doi.org/10.1109/VIS60296.2025.00034>.
- Karen Spärck Jones. A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1):11–21, 1972. doi: 10.1108/eb026526.
- Zhi Rui Tam, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung-yi Lee, and Yun-Nung Chen. Let me speak freely? a study on the impact of format restrictions on large language model performance. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 1218–1236, Miami, Florida, US, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-industry.91. URL <https://aclanthology.org/2024.emnlp-industry.91/>.
- Yuan Tian, Weiwei Cui, Dazhen Deng, Xinjing Yi, Yurun Yang, Haidong Zhang, and Yingcai Wu. Chartgpt: Leveraging llms to generate charts from abstract natural language. *IEEE Transactions on Visualization and Computer Graphics*, 31(3):1731–1745, 2025. doi: 10.1109/TVCG.2024.3368621.
- Chris van der Lee, Albert Gatt, Emiel van Miltenburg, Sander Wubben, and Emiel Krahmer. Best practices for the human evaluation of automatically generated text. In *Proceedings of the 12th International Conference on Natural Language Generation*, pages 355–368. Association for Computational Linguistics, 2019. doi: 10.18653/v1/W19-8643. URL <https://aclanthology.org/W19-8643/>.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017. URL <https://arxiv.org/abs/1706.03762>.
- Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khoshabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13484–13508. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.acl-long.754. URL <https://aclanthology.org/2023.acl-long.754/>.

- Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew Mingbo Dai, and Quoc V. Le. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations*, 2022. URL <https://arxiv.org/abs/2109.01652>.
- Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945. doi: 10.2307/3001968.
- Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models, 2023. URL <https://arxiv.org/abs/2307.09702>.
- Kanit Wongsuphasawat, Dominik Moritz, Anushka Anand, Jock Mackinlay, Bill Howe, and Jeffrey Heer. Voyager: Exploratory analysis via faceted browsing of visualization recommendations. *IEEE Transactions on Visualization and Computer Graphics*, 22(1):649–658, 2016. doi: 10.1109/TVCG.2015.2467191. URL <https://doi.org/10.1109/TVCG.2015.2467191>.
- Kanit Wongsuphasawat, Zening Qu, Dominik Moritz, Riley Chang, Felix Ouk, Anushka Anand, Jock Mackinlay, Bill Howe, and Jeffrey Heer. Voyager 2: Augmenting visual analysis with partial view specifications. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*, pages 2648–2659, 2017. doi: 10.1145/3025453.3025768. URL <https://doi.org/10.1145/3025453.3025768>.
- World Wide Web Consortium. Web content accessibility guidelines (WCAG) 2.2. W3C Recommendation, 2024. URL <https://www.w3.org/TR/WCAG22/>.
- Zhengkai Wu, Vu Le, Ashish Tiwari, Sumit Gulwani, Arjun Radhakrishna, Ivan Radiček, Gustavo Soares, Xinyu Wang, Zhenwen Li, and Tao Xie. Nl2viz: Natural language to visualization via constrained syntax-guided synthesis. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 972–983, 2022. doi: 10.1145/3540250.3549140. URL <https://doi.org/10.1145/3540250.3549140>.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Gao Gao, Jiaqi Cai, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. doi: 10.48550/arXiv.2505.09388. URL <https://arxiv.org/abs/2505.09388>.
- Ogan M. Yigitbasioglu and Oana Velcu. A review of dashboards in performance management: Implications for design and research. *International Journal of Accounting Information Systems*, 13(1):41–59, 2012. doi: 10.1016/j.accinf.2011.08.002.
- Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3911–3921. Association for Computational Linguistics, 2018. doi: 10.18653/v1/D18-1425. URL <https://aclanthology.org/D18-1425/>.
- Qingru Zhang, Minshuo Chen, Alexander Bukharin, Nikos Karampatziakis, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning. In *International Conference on Learning Representations*, 2023. URL <https://arxiv.org/abs/2303.10512>.
- Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. GaLore: Memory-efficient LLM training by gradient low-rank projection. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pages 61121–61143. PMLR, 2024. URL <https://proceedings.mlr.press/v235/zhao24s.html>.